

Formal Verification Report: Centrifuge Core

- Date: September 11th, 2026
 - Audit Repo: <https://github.com/alexzoid-eth/2026-08-centrifuge-fv-ex>
 - Client Repo: <https://github.com/centrifuge/protocol-internal> (3397d602)
 - Author: [AlexZoid](#)
 - Certora Prover version: 8.16.2
-

Table of Contents

1. [Verification Scope](#)
 2. [Methodology](#)
 - [Setup Scenes](#)
 - [Types of Properties](#)
 - [Types of Assumptions](#)
 3. [Properties](#)
 - [Accounting](#)
 - [Holdings](#)
 - [ShareClassManager](#)
 - [HubRegistry](#)
 - [Spoke](#)
 - [SpokeRegistry](#)
 - [SnapshotQueue](#)
 - [PoolEscrow](#)
 - [MultiAdapter](#)
 - [Hub_LinkedHubCore](#)
 - [HubHandler_LinkedHubCore](#)
 - [Spoke_LinkedSpokeCore](#)
 - [SpokeHandler_LinkedSpokeRegistry](#)
 - [SpokeHandler_LinkedRegistryFactory](#)
 - [MessageDispatcher](#)
 - [PoolEscrowFactory](#)
 4. [Mutation Testing](#)
 5. [Reproducing the Results](#)
 - [Prerequisites](#)
 - [Remote Execution](#)
 - [Local Execution](#)
 - [Running the Suites](#)
 6. [Resources](#)
-

Verification Scope

Centrifuge is a real-world-asset tokenization protocol built as a hub and spoke: one hub chain carries each pool's accounting and administration, spoke chains the custody and user-facing balance-sheet operations; a quorum-gated messaging layer keeps them convergent. The contracts verified, all in [src/core/](#), fall into three groups.

Hub side

1. **Accounting** ([hub/Accounting.sol](#)): the double-entry ledger of per-pool charts of accounts, mutable only inside a transient session that refuses to close unless debits equal credits.
2. **Holdings** ([hub/Holdings.sol](#)): the hub's mirror of spoke asset positions per pool, share class and asset, with carrying value in pool currency against cumulative increase and decrease counters.
3. **ShareClassManager** ([hub/ShareClassManager.sol](#)): the share class registry and hub-side conservation ledger, where per-network issuance and revocation counters must sum to the global total issuance.
4. **HubRegistry** ([hub/HubRegistry.sol](#)): the registry of pools, assets, managers and policies, plus the timelocked authorization ledger the hub access-control model rests on.
5. **Hub** ([hub/Hub.sol](#)): the pool-manager orchestrator over the four contracts above, which must post a paired debit and credit for every accounting mutation.
6. **HubHandler** ([hub/HubHandler.sol](#)): the hub-side entry point for cross-chain messages, applying spoke deltas to Holdings and ShareClassManager, reconciling the burn-and-mint share bridge.

Spoke side

7. **Spoke** ([spoke/Spoke.sol](#)): the balance-sheet orchestrator for users and managers, where every escrow operation must mirror into the snapshot queue.
8. **SpokeRegistry** ([spoke/SpokeRegistry.sol](#)): the spoke's source of truth for pools, share classes, roles, policies, vaults and prices, over an asset-id bijection.
9. **SpokeHandler** ([spoke/SpokeHandler.sol](#)): the spoke-side message applier, deploying escrows, tokens and vaults and routing mutations into the registry.
10. **SnapshotQueue** ([spoke/SnapshotQueue.sol](#)): the accumulator of share and asset deltas pending submission to the hub, carrying the consistency flag the hub trusts.
11. **PoolEscrow** ([spoke/PoolEscrow.sol](#)): the per-pool custody contract, with total and reserved bookkeeping where a withdrawal never pays the reserved part.
12. **PoolEscrowFactory** ([spoke/factories/PoolEscrowFactory.sol](#)): the CREATE2 deployer and address oracle for those escrows, where a pool's escrow address is the same for every caller and unchanged by any call the factory takes.

Messaging

13. **MultiAdapter** ([messaging/MultiAdapter.sol](#)): the quorum layer over bridge adapters, where an inbound payload becomes truth only at threshold votes per chain, pool and session.
14. **MessageDispatcher** ([messaging/MessageDispatcher.sol](#)): the outbound end of the messaging layer, taking a local destination straight into its handler and a remote one into the gateway, where a caller without the ward bit moves no wiring and reaches no neighbour.

Methodology

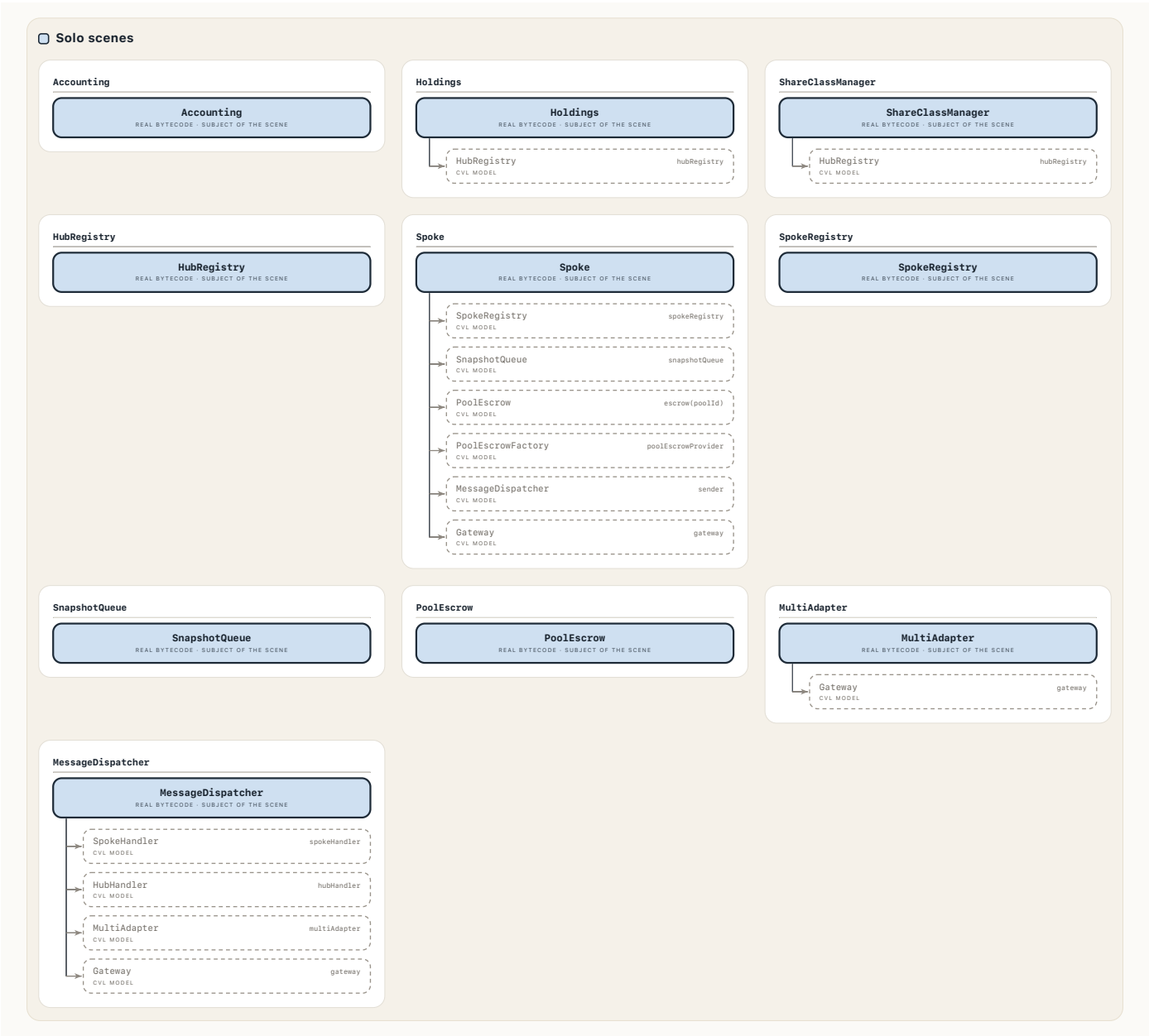
Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to the prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

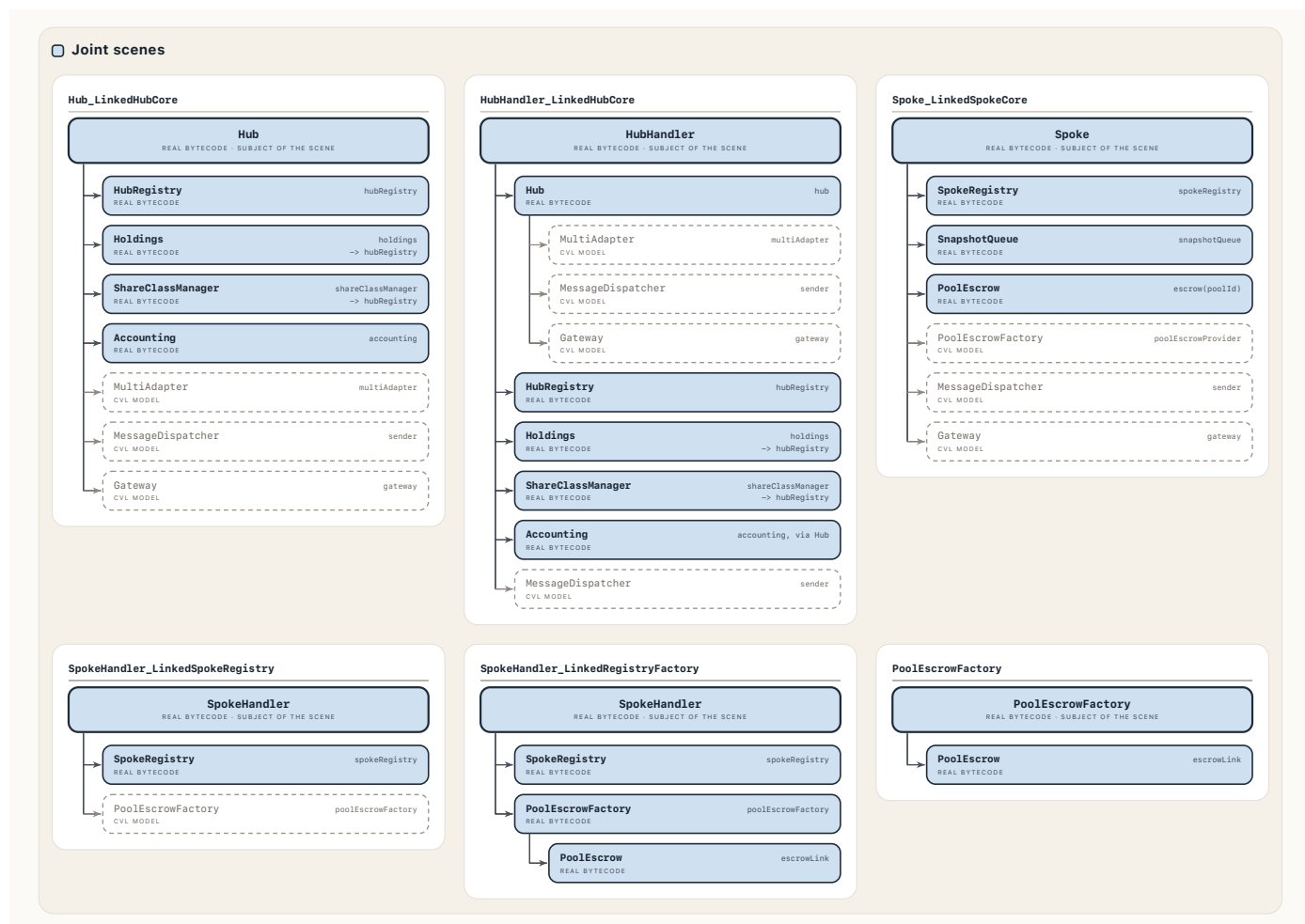
Setup Scenes

A scene is the set of contracts the prover compiles for one verification run. Each is present as **real** bytecode or as a **CVL model**, a stand-in written in the specification language, and every target is real in the scene that verifies it.

A scene splits into a setup and the rules that run over it. The setup ships two files per contract: the specification another scene imports to attach it, and the model that stands in for it. It links the compiled contracts, resolves every outgoing call to a model, a compiled neighbour or out of scope, and rewrites in CVL the internal functions the prover cannot analyse.



A **solo** scene compiles one target and models every neighbour in CVL; it carries whatever valid state that target's storage admits, and every property that needs no second contract.



A **joint** scene compiles several contracts and links them, is usually named `A_LinkedB`, and takes each participant's valid state, where a rule needs it, either as a fact already proven on that contract's own scene or by re-proving it inductively here.

The setup also narrows: one symbolic pool where the scene has that dimension, a multi-pool setup for isolation rules, bounded sets where a property sums (token holders, accounts), every loop unrolled to a fixed count. Each scene below lists its own bounds.

Types of Properties

The official Certora [methodology](#) defines several property categories. A **parametric** category is checked against every external function of the scene, including ones added after the specification.

- **Valid State** (parametric): system-wide invariants that hold in every reachable state; once proven, other properties reuse them as preconditions through `requireInvariant`.
- **State Transitions** (parametric): the moves between valid states, how a valid state may change and in what order.
- **Variable Transitions** (parametric): the permitted before/after relation on a single state variable across any call: monotonic counters, write-once fields, values that only move within a bound.
- **High Level**: properties of one or several specific function calls (share conservation, a deposit and redeem round trip).
- **Reverts**: an external call reverts exactly on its documented preconditions.
- **Reachability**: every rule's states are reachable, so none passes vacuously.

- **Access Control** (parametric): every privileged entry point enforces its ward or role gate.

Types of Assumptions

Each `require` limits which states the prover checks, and its tag says why.

- **SAFE**: excludes no reachable state (`msg.sender != 0`).
- **SCOPE**: narrows a rule to the logic it checks (`receiver != escrowAddress`).
- **UNSAFE**: drops reachable states, for a prover limit or timeout (a loop unrolled to a fixed count).
- **TRUSTED**: guaranteed by a trusted party outside the scene (a valid setup left by the initializer logic).
- **PROVED**: not an assumption: a fact proven elsewhere, reused as a precondition (an invariant via `requireInvariant`).
- **ASSERT**: used in models: a `require` reproducing a revert of the real code, a path the prover drops by default anyway (`InsufficientBalance` in the token model).

Properties

Every verdict below comes from the prover version named in the header.

- **Properties**: every invariant and rule the suite states
- **Proved** : verified by the prover
- **Timeout** : the prover ran the rule and did not decide it within the solver budget the confs carry
- **Mutations** : rules with a mutant the prover killed, meaning the rule holds on the real code and breaks on a deliberately faulted one. An empty cell means no mutant is written for that rule, which is where an undecided rule stands: a kill is a change of verdict, and a rule the prover never decided has none to change
- **Violations** : rules that fail

Contract	Properties	Proved 	Timeout 	Mutations 	Violations 
Accounting	72	72	0	72	0
Holdings	87	87	0	87	0
ShareClassManager	85	85	0	85	0
HubRegistry	105	105	0	105	0
Spoke	94	94	0	94	0
SpokeRegistry	141	141	0	141	0
SnapshotQueue	75	75	0	75	0
PoolEscrow	52	52	0	52	0
MultiAdapter	97	96	1	96	0
Hub_LinkedHubCore	97	97	0	97	0
HubHandler_LinkedHubCore	101	101	0	101	0
Spoke_LinkedSpokeCore	39	39	0	39	0
SpokeHandler_LinkedSpokeRegistry	91	91	0	91	0
SpokeHandler_LinkedRegistryFactory	1	1	0	1	0
MessageDispatcher	3	3	0	3	0
PoolEscrowFactory	4	4	0	4	0
Total	1144	1143	1	1143	0

Accounting

- Every configuration
 - The chart of accounts bounded to 3 symbolic accounts
 - Loops running up to 3 iterations
- Single pool

- One symbolic pool
- Multi pool
 - Two symbolic pools

Valid State

Properties	Status	Mutations
AC-VS-01 <code>absentAccountIsEmpty</code> An account that was never created is empty, so one opened later cannot carry a history.	✓	
AC-VS-02 <code>nullAccountNeverCreated</code> The null account id never exists, so it absorbs no entries and no holding wires to it.	✓	
AC-VS-03 <code>lastUpdatedBetweenEnvFloorAndBlock</code> An existing account's stamp sits above the scene's minimum block time and no later than the current block.	✓	
AC-VS-04 <code>postedHistoryImpliesAnIssuedJournalId</code> An account carrying posting history proves a session was opened and took an id.	✓	

State Transitions

Properties	Status	Mutations
AC-ST-01 <code>stAcAccountOrientationIsPermanent</code> An account keeps the orientation it was opened under.	✓	
AC-ST-02 <code>stAcAccountRowsArePerPool</code> A call moving one pool's account row leaves the other pool's rows untouched.	✓	
AC-ST-03 <code>stAcJournalNumberingIsPerPool</code> A call touching one pool's journal leaves the other pool's journal alone.	✓	

Variable Transitions

Properties	Status	Mutations
AC-VT-01 <code>vtAcAccountDebitNeverDecreases</code> Debit history is permanent: no call writes down what a pool has recorded.	✓	
AC-VT-02 <code>vtAcAccountCreditNeverDecreases</code> Credit history is permanent: no call writes down what a pool has recorded.	✓	
AC-VT-03 <code>vtAcAccountStampNeverRegresses</code> An account's stamp only moves forward, so a row can never be backdated.	✓	
AC-VT-04 <code>vtAcJournalCounterNeverRewinds</code> A pool's journal numbering never rewinds, so a number once spent is never handed out again.	✓	

High Level

Properties	Status	Mutations

AC-HL-01 <code>hlAcSessionClosesOnlyOnEqualPostedValues</code> A session closes only when the values it posted balance on both sides.	✓	
AC-HL-02 <code>hlAcJournalPairDebitsExactlyItsNamedRow</code> A journal pair debits exactly the row and the value it names, and no other row.	✓	
AC-HL-03 <code>hlAcJournalPairCreditsExactlyItsNamedRow</code> A journal pair credits exactly the row and the value it names, and no other row.	✓	
AC-HL-04 <code>hlAcLedgerMovesOnlyThroughAJournal</code> The recorded books move only through a journal; every other call leaves them standing.	✓	
AC-HL-05 <code>hlAcSecondJournalOfAPoolReusesItsJournalId</code> A second journal of the same pool in one transaction is filed under the first journal's id.	✓	
AC-HL-06 <code>hlAcAccountValueReportsTheRowsNetMagnitude</code> The balance reported for an account is its net position, never a gross total.	✓	
AC-HL-07 <code>hlAcAccountValueSignsByTheRowsOrientation</code> An account's reported sign follows the orientation it was opened under.	✓	
AC-HL-08 <code>hlAcReopenedJournalStartsFromEmptyPostedSides</code> A journal reopened after an earlier one starts from empty totals.	✓	

Reverts

Properties	Status	Mutations
AC-RV-01 <code>rvAcUnlockRefusesANonWard</code> A journal opens only for the pool's own ledger authority, and never on the null pool.	✓	
AC-RV-02 <code>rvAcUnlockRefusesASecondOpenWhileAJournalStands</code> A journal already open is never opened a second time.	✓	
AC-RV-03 <code>rvAcCreateAccountRefusesAPaidCallANonWardTheNullIdOrARowThatStands</code> A live account's history cannot be wiped by opening the same account again.	✓	
AC-RV-04 <code>rvAcAddDebitRefusesARowTheLedgerNeverOpened</code> A debit needs a row the ledger opened.	✓	
AC-RV-05 <code>rvAcAddCreditRefusesARowTheLedgerNeverOpened</code> A credit needs a row the ledger opened.	✓	
AC-RV-06 <code>rvAcLockRevertsIffThePostedSidesDiffer</code> A journal closes only when the two sides it posted match.	✓	
AC-RV-07 <code>rvAcAddJournalOffSessionRefusesANonWardOrAnyEntry</code> Nothing can be posted to a pool's books outside an open journal.	✓	
AC-RV-08 <code>rvAcAccountValueRefusesARowTheLedgerNeverOpened</code> The value of an account the ledger never opened is refused, not reported as zero.	✓	
AC-RV-09 <code>rvAcPassingExistsIsReachable</code> Witnesses a read taken in the transaction that opened an account answering and reporting that account.	✓	
AC-RV-10 <code>rvAcJournalPairRefusesUnbalancedLegs</code> An unbalanced pair never reaches the books.	✓	

AC-RV-11 <code>rvAcPassingJournalPairOnABalancedPairIsReachable</code> Witnesses a nonzero balanced pair going through end to end on two rows the ledger already holds.	✓	
AC-RV-12 <code>rvAcAddDebitRefusesAnOutsiderInsideAnOpenJournal</code> An open journal is not an open door: an outsider still cannot charge the books.	✓	
AC-RV-13 <code>rvAcAddCreditRefusesAnOutsiderInsideAnOpenJournal</code> An open journal is not an open door: an outsider still cannot credit the books.	✓	
AC-RV-14 <code>rvAcLockRefusesAnOutsiderOnAnOpenJournal</code> Only the ledger's own authority may close a journal.	✓	
AC-RV-15 <code>rvAcAccountValueAlwaysAnswersForARowThatStands</code> The value of an account the ledger holds always reads back, on either side of its normal balance.	✓	

Reachability

Properties	Status	Mutations
AC-RC-01 <code>rcAcUnlockOpensAJournalIsReachable</code> Witnesses a ward opening a journal on a pool, minting it a fresh id.	✓	
AC-RC-02 <code>rcAcSecondSessionOfAPoolIsReachable</code> Witnesses a pool's journal closed and reopened inside one transaction.	✓	
AC-RC-03 <code>rcAcLastJournalIdOfTheSequenceIsReachable</code> Witnesses a pool one id short of exhaustion opening one more journal.	✓	
AC-RC-04 <code>rcAcEmptyJournalOverAnUntouchedLedgerIsReachable</code> Witnesses an empty journal opening and closing, and still spending one id.	✓	
AC-RC-05 <code>rcAcBalancedPairPostsAndClosesIsReachable</code> Witnesses the ordinary journal end to end: open, a matching debit and credit, close.	✓	
AC-RC-06 <code>rcAcDebitHeavySessionRecoversIsReachable</code> Witnesses that a debit-heavy session is not wedged: the compensating credit still closes it.	✓	
AC-RC-07 <code>rcAcTwoDebitsCloseAgainstOneCreditIsReachable</code> Witnesses a journal of several entries closing on the summed condition.	✓	
AC-RC-08 <code>rcAcZeroValueDebitRestampsTheRowIsReachable</code> Witnesses a zero-value debit restamping the row without moving a total.	✓	
AC-RC-09 <code>rcAcZeroValueCreditRestampsTheRowIsReachable</code> Witnesses a zero-value credit restamping the row without moving a total.	✓	
AC-RC-10 <code>rcAcDebitFillsARowToTheCeilingIsReachable</code> Witnesses one posting carrying a row's debit side from empty to the widest value it holds.	✓	
AC-RC-11 <code>rcAcCreditFillsARowToTheCeilingIsReachable</code> Witnesses one credit carrying a row's credit side from empty to the widest value it holds.	✓	
AC-RC-12 <code>rcAcCeilingPairPostsAndClosesIsReachable</code> Witnesses a journal whose two legs each carry the widest value being settled, not merely opened.	✓	

AC-RC-13 rcAcSaturatedDebitRowStillTakesACreditIsReachable	✓	
Witnesses a row already at its debit ceiling still accepting a credit.		
AC-RC-14 rcAcDebitOvertakesACreditHeavyRowIsReachable	✓	
Witnesses a debit carrying a credit-heavy row back across its own sign.		
AC-RC-15 rcAcCreateDebitNormalAccountIsReachable	✓	
Witnesses a debit-normal row being opened, absent before the call and present after.		
AC-RC-16 rcAcCreateCreditNormalAccountIsReachable	✓	
Witnesses a credit-normal row being opened, stamped and empty on both sides.		
AC-RC-17 rcAcSecondRowOpensBesideALiveOneIsReachable	✓	
Witnesses a row being opened beside a live one without disturbing it.		
AC-RC-18 rcAcFirstRowOfAnEmptyChartIsReachable	✓	
Witnesses the pool's very first account row being opened on an empty chart.		
AC-RC-19 rcAcJournalBatchOfOneEntryASideIsReachable	✓	
Witnesses one entry a side moving both rows the entries name and closing the journal.		
AC-RC-20 rcAcJournalBatchReachesTheWholeChartIsReachable	✓	
Witnesses one batch reaching every row of the pool's chart and still closing.		
AC-RC-21 rcAcEmptyJournalBatchIsReachable	✓	
Witnesses a journal carrying no entry being admitted and still spending one id.		
AC-RC-22 rcAcZeroValueJournalBatchIsReachable	✓	
Witnesses a batch of zero-value entries closing, restamping without moving a total.		
AC-RC-23 rcAcRowsCreatedAndJournalledInOneTransactionIsReachable	✓	
Witnesses a pool going from no chart at all to a closed journal in one transaction.		
AC-RC-24 rcAcFreshRowReadsBackThePostedValueIsReachable	✓	
Witnesses a row opened, posted to and read back in one transaction reporting what was posted.		
AC-RC-25 rcAcDebitNormalRowReportsANegativeValueIsReachable	✓	
Witnesses a debit-normal row credited beyond its debits reporting that magnitude as negative.		
AC-RC-26 rcAcRowDebitedAndCreditedAlikeReadsZeroIsReachable	✓	
Witnesses a row debited and credited alike reading back at zero.		
AC-RC-27 rcAcOpenSessionReportsItsRunningSidesIsReachable	✓	
Witnesses an open session reporting exactly what has been posted to each side so far.		
AC-RC-28 rcAcAuthorityGrantRoundTripIsReachable	✓	
Witnesses an authority grant being real and reversible inside one transaction.		
AC-RC-29 rcAcWardRevokesItselfIsReachable	✓	
Witnesses a ward revoking its own authority.		
AC-RC-30 rcAcFreshRowReadsBackZeroedAndStampedIsReachable	✓	
Witnesses the account getter reporting a freshly opened row as zeroed, oriented and stamped.		

Access Control

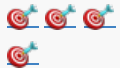
Properties	Status	Mutations
AC-AC-01 <code>acAcDebitHistoryMovesOnlyForAWard</code> No outsider can inflate or erase the charges a pool has booked, outside the direct debit call.	✓	
AC-AC-02 <code>acAcCreditHistoryMovesOnlyForAWard</code> No outsider can inflate or erase the receipts a pool has booked, outside the direct credit call.	✓	
AC-AC-03 <code>acAcAccountStampMovesOnlyForAWard</code> Only the ledger's own authority moves an account's audit stamp, outside the direct posting calls.	✓	
AC-AC-04 <code>acAcAccountOpensOnlyForAWard</code> Only the pool's own ledger authority can open an account, outside the direct ledger calls.	✓	
AC-AC-05 <code>acAcAccountOrientationIsSetOnlyByAWard</code> Only the ledger's own authority decides whether an account counts value up or down, outside the direct ledger calls.	✓	
AC-AC-06 <code>acAcPermissionsMoveOnlyForAWard</code> Only someone already holding the ledger's authority can grant or revoke it, outside the direct ledger calls.	✓	
AC-AC-07 <code>acAcJournalNumberingAdvancesOnlyForAWard</code> Only the ledger's own authority advances a pool's journal numbering, outside the direct ledger calls.	✓	
AC-AC-08 <code>acAcSessionIsLeftOpenOnlyForAWard</code> Only the ledger's own authority can leave the ledger open for posting, outside the direct ledger calls.	✓	

Holdings

- Every configuration
 - Holding rows bounded to 2 symbolic rows
 - The deficit counter pinned to one symbolic network
 - Loops running up to 4 iterations
- Single pool
 - One symbolic pool
 - One symbolic share class
- Multi pool
 - Two symbolic pools
 - One share class per pool
- Multi share class
 - Two share classes in the pinned pool

Valid State

Properties	Status	Mutations
------------	--------	-----------

HO-VS-01 <code>snapshotHookImpliesPoolExists</code> A snapshot hook is attached only to a pool the hub registry knows.	✓	
HO-VS-02 <code>deficitCountMatchesTheShortRows</code> The pool's deficit counter equals the number of holding rows with more withdrawn than deposited.	✓	
HO-VS-03 <code>uninitializedHoldingHasNoValue</code> A holding row that was never initialized has no carrying value.	✓	
HO-VS-04 <code>emptyHoldingCarriesNoValue</code> A holding row with nothing left in it (at least as much taken out as put in) has no carrying value.	✓	
HO-VS-05 <code>snapshotFlagImpliesAStartedNonce</code> A snapshot row marked in sync has recorded at least one update.	✓	
HO-VS-06 <code>accountIdImpliesInitialized</code> Journal account wiring exists only for holding rows that have been initialized.	✓	
HO-VS-07 <code>deficitCountSpansEveryShareClass</code> The pool's deficit counter counts every short row of the pool, whichever class it sits on.	✓	

State Transitions

Properties	Status	Mutations
HO-ST-01 <code>stHoInflowAndOutflowNeverMoveTogether</code> A holding's deposit and withdrawal histories never move in the same call.	✓	
HO-ST-02 <code>stHoOneCallTouchesOneHolding</code> A single call touches a single holding.	✓	
HO-ST-03 <code>stHoInflowNeverLowersValue</code> Money coming into a holding never writes its carrying value down.	✓	
HO-ST-04 <code>stHoOutflowNeverRaisesValue</code> Money leaving a holding never writes its carrying value up.	✓	
HO-ST-05 <code>stHoOpeningAHoldingFundsNothing</code> Opening a holding only gives it a price source, never a position or a carrying value no deposit funded.	✓	
HO-ST-06 <code>stHoSnapshotFlagMovesWithItsNonce</code> A network's snapshot flag flips only together with a one-step advance of its nonce.	✓	
HO-ST-07 <code>stHoHoldingRowIsPerPool</code> A call that moves one pool's holding row leaves every holding row of the other pool alone.	✓	
HO-ST-08 <code>stHoAccountWiringIsPerPool</code> A call that wires one pool's ledger accounts leaves the other pool's wiring alone.	✓	
HO-ST-09 <code>stHoSnapshotRowIsPerPool</code> A call moving one pool's snapshot row leaves the other pool's row for that network alone.	✓	
HO-ST-10 <code>stHoDeficitCountAndHookIgnoreASiblingsFlow</code> Flow booked on one pool's holding row moves neither the sibling pool's deficit counter nor its snapshot hook.	✓	

Variable Transitions

Properties	Status	Mutations
HO-VT-01 <code>vtHoDepositHistoryNeverShrinks</code> A position's deposit history only ever accumulates.	✓	
HO-VT-02 <code>vtHoWithdrawalHistoryNeverShrinks</code> A position's withdrawal history only ever accumulates, an over withdrawal included.	✓	
HO-VT-03 <code>vtHoOpenedHoldingIsNeverClosed</code> A holding the pool has opened can never be closed again.	✓	
HO-VT-04 <code>vtHoSnapshotNonceStepsByAtMostOne</code> A network's snapshot sequence advances one step at a time, never skipped or rewind.	✓	

High Level

Properties	Status	Mutations
HO-HL-01 <code>hlHoDepositHistoryRecordsTheInflowAndSurvivesTheOutflow</code> An inflow is recorded at full size, and the outflow after it leaves that history readable.	✓	
HO-HL-02 <code>hlHoWithdrawalHistoryRecordsTheWholeOutflow</code> An outflow is recorded at full size even beyond what the position carries, the excess staying a claim.	✓	
HO-HL-03 <code>hlHoIncreaseReturnsTheValueItBooked</code> An increase journals exactly the value it adds to the position book.	✓	
HO-HL-04 <code>hlHoDecreaseReturnsTheValueItRemoved</code> A decrease journals exactly the value it takes off the position book.	✓	
HO-HL-05 <code>hlHoUpdateReportsTheRevaluationItWrote</code> A revaluation journals exactly the move it writes, sign included.	✓	
HO-HL-06 <code>hlHoAccountWiringMovesOnlyThroughItsTwoWriters</code> A position's journal wiring moves only through initialize and setAccountId.	✓	
HO-HL-07 <code>hlHoSnapshotHookMovesOnlyThroughItsSetter</code> A pool's NAV automation hook is rewired only by setSnapshotHook.	✓	
HO-HL-08 <code>hlHoPublishingASyncMarkerAlwaysAdvancesTheSequence</code> Every sync marker consumes the sequence number that network was waiting on.	✓	
HO-HL-09 <code>hlHoSnapshotSequenceMovesOnlyThroughItsPublisher</code> A network's snapshot sequence advances only where a marker is published.	✓	

Reverts

Properties	Status	Mutations
HO-RV-01 <code>rvHoInitializeRefusesANonWardANullSourceOrAnOpenRow</code> A holding does not open for a stranger, without a price source, or on a row already open.	✓	
HO-RV-02 <code>rvHoSetAccountIdRefusesANonWardOrAnUnopenedHolding</code>		

A journal account does not wire for a stranger, nor onto a holding never opened.	✓	
HO-RV-03 <code>rvHoUpdateValuationRefusesANonWardANullSourceOrAnUnopenedHolding</code> A holding is reported only by the ledger's own authority, and only at a real price source.	✓	
HO-RV-04 <code>rvHoSetSnapshotHookRefusesANonWardOrAnUnknownPool</code> A pool's NAV hook is attached only by the ledger's own authority, on a pool the registry knows.	✓	
HO-RV-05 <code>rvHoSetSnapshotRefusesANonWardOrAnOutOfOrderNonce</code> A sync marker is published only under the sequence number the network waits for.	✓	
HO-RV-06 <code>rvHoCallOnSyncSnapshotIsNeverRefusedToAward</code> Replaying the sync notification is never refused to a ward, on any network.	✓	
HO-RV-07 <code>rvHoCallOnTransferSnapshotIsNeverRefusedToAward</code> With no automation hook installed, a ward notifying a cross network transfer is never refused, since the hook is called only where a pool has set one.	✓	
HO-RV-08 <code>rvHoIncreaseRefusesANonWard</code> Nobody outside the pool's ward set can book an inflow onto a holding.	✓	
HO-RV-09 <code>rvHoIncreaseRefusesAnInflowPastTheDepositCounter</code> An inflow past the width of the deposit counter is refused rather than wrapped.	✓	
HO-RV-10 <code>rvHoIncreaseBeforeTheRowIsOpenedIsNeverRefusedToAward</code> Custody arriving before a row is opened is never turned away.	✓	
HO-RV-11 <code>rvHoIncreaseOntoAnEmptyValueBookIsNeverRefusedToAward</code> Holdings itself turns away no first deposit onto a position a pool has just opened.	✓	
HO-RV-12 <code>rvHoDecreaseNeverStallsOnAnOversizedOutflow</code> An outflow larger than the position never stalls the message stream.	✓	
HO-RV-13 <code>rvHoUpdateIsNeverRefusedToAwardOnAnOpenHolding</code> Holdings itself turns away no ward revaluing an open holding.	✓	
HO-RV-14 <code>rvHoValuationRefusesAnUnopenedHolding</code> An absent holding refuses its price source read rather than answering the zero address.	✓	
HO-RV-15 <code>rvHoAmountAlwaysAnswers</code> A holding reports its amount in every state, a pool in deficit included.	✓	
HO-RV-16 <code>rvHoCallOnSyncSnapshotRefusesANonWard</code> Replaying the sync notification is closed to anyone outside the ledger's own authority.	✓	
HO-RV-17 <code>rvHoCallOnTransferSnapshotRefusesANonWard</code> Notifying the hook of a cross network transfer is closed to anyone outside that authority.	✓	

Reachability

Properties	Status	Mutations
HO-RC-01 <code>rcHoOpeningThePoolsFirstHoldingIsReachable</code> Witnesses a pool opening the first holding of its share class.	✓	
HO-RC-02 <code>rcHoOpeningWiringAllFourLedgerAccountsIsReachable</code>	✓	

Witnesses one opening call wiring the whole ledger account set of a holding.		
HO-RC-03 <code>rcHoOpeningASecondHoldingIsReachable</code> Witnesses a second asset opened beside a holding the pool already runs.	✓	
HO-RC-04 <code>rcHoBookingADepositIsReachable</code> Witnesses a deposit landing on a live holding and journaling a positive value.	✓	
HO-RC-05 <code>rcHoFillingTheDepositCounterToItsCeilingIsReachable</code> Witnesses a deposit filling a holding's deposit history to the whole width of the field.	✓	
HO-RC-06 <code>rcHoZeroDepositIsReachable</code> Witnesses a deposit of nothing being accepted on a live holding.	✓	
HO-RC-07 <code>rcHoDepositPricedAtNothingIsReachable</code> Witnesses a deposit the price source values at nothing.	✓	
HO-RC-08 <code>rcHoBookingADepositBeforeTheRowIsOpenedIsReachable</code> Witnesses a deposit booked for a row the pool has not opened yet.	✓	
HO-RC-09 <code>rcHoClearingADeficitExactlyIsReachable</code> Witnesses a deposit netting off an earlier over withdrawal exactly.	✓	
HO-RC-10 <code>rcHoBookingAWithdrawalIsReachable</code> Witnesses a withdrawal booked against a funded position.	✓	
HO-RC-11 <code>rcHoWithdrawingPastThePositionIsReachable</code> Witnesses the documented over withdrawal: the position floors at nothing and counts as in deficit.	✓	
HO-RC-12 <code>rcHoClosingThePositionExactlyIsReachable</code> Witnesses a withdrawal sized to the position closing it exactly.	✓	
HO-RC-13 <code>rcHoDeepeningADeficitIsReachable</code> Witnesses a second withdrawal on a row already short deepening the shortfall.	✓	
HO-RC-14 <code>rcHoDeficitCountCarryingASecondShortRowIsReachable</code> Witnesses two rows of one class standing short at the same time.	✓	
HO-RC-15 <code>rcHoBookingAWithdrawalBeforeTheRowIsOpenedIsReachable</code> Witnesses a withdrawal booked for a row the pool has not opened yet.	✓	
HO-RC-16 <code>rcHoRevaluationUpwardIsReachable</code> Witnesses a revaluation marking a holding up.	✓	
HO-RC-17 <code>rcHoRevaluationDownwardIsReachable</code> Witnesses a revaluation marking a holding down.	✓	
HO-RC-18 <code>rcHoRevaluationLeavingThePositionUnmovedIsReachable</code> Witnesses a revaluation of a funded holding that moves nothing.	✓	
HO-RC-19 <code>rcHoRevaluatingAnEmptyHoldingIsReachable</code> Witnesses a revaluation of a live holding that carries no position.	✓	
HO-RC-20 <code>rcHoMarkingANetworkInSyncIsReachable</code> Witnesses a network being marked in sync and its ordering sequence stepping.	✓	
HO-RC-21 <code>rcHoMarkingANetworkOutOfSyncIsReachable</code> Witnesses a network that was in sync being marked out of sync again.	✓	

HO-RC-22 <code>rcHoSyncAndDesyncRoundTripIsReachable</code> Witnesses a network syncing and desyncing inside one transaction.	✓	
HO-RC-23 <code>rcHoSequenceReachingItsCeilingIsReachable</code> Witnesses a network taking the last sync round its ordering sequence can carry.	✓	
HO-RC-24 <code>rcHoAttachingNavAutomationIsReachable</code> Witnesses NAV automation being attached to a pool the hub registry knows.	✓	
HO-RC-25 <code>rcHoAutomationAttachAndDetachRoundTripIsReachable</code> Witnesses automation being attached and taken off again through the same entry point.	✓	
HO-RC-26 <code>rcHoRewiringALedgerAccountIsReachable</code> Witnesses one ledger account of a live holding being rewired.	✓	
HO-RC-27 <code>rcHoSwappingThePriceSourceIsReachable</code> Witnesses a holding's price source being swapped while the position stays put.	✓	
HO-RC-28 <code>rcHoWardGrantAndRevokeRoundTripIsReachable</code> Witnesses authority over the holdings book granted and taken back in one transaction.	✓	
HO-RC-29 <code>rcHoOpeningAndFullyClosingAPositionIsReachable</code> Witnesses a holding opened, funded and closed in one transaction, both histories still on show.	✓	
HO-RC-30 <code>rcHoEnteringAndLeavingADeficitIsReachable</code> Witnesses a row pushed short and lifted back out inside one transaction.	✓	

Access Control

Properties	Status	Mutations
HO-AC-01 <code>acHoWardRightsMoveOnlyForAWard</code> Only an address already holding it can grant or take back the right to write the pool's book.	✓	
HO-AC-02 <code>acHoDepositHistoryMovesOnlyForAWard</code> Only the ledger's own authority moves a position's record of what the pool received.	✓	
HO-AC-03 <code>acHoWithdrawalHistoryMovesOnlyForAWard</code> Only the ledger's own authority moves a position's record of what the pool paid out.	✓	
HO-AC-04 <code>acHoCarryingValueMovesOnlyForAWard</code> A position's carrying value moves only for the ledger's own authority.	✓	
HO-AC-05 <code>acHoPriceSourceMovesOnlyForAWard</code> Only the ledger's own authority names or repoints the price source a position is valued at.	✓	
HO-AC-06 <code>acHoJournalWiringMovesOnlyForAWard</code> Only the ledger's own authority wires the journal accounts a position posts to.	✓	
HO-AC-07 <code>acHoSnapshotHookMovesOnlyForAWard</code> Only the ledger's own authority installs the pool's NAV automation hook.	✓	
HO-AC-08 <code>acHoSyncMarkerMovesOnlyForAWard</code> Only the ledger's own authority flips the marker saying a network's assets and shares are in sync.	✓	

HO-AC-09 <code>acHoSnapshotNonceMovesOnlyForAWard</code> Only the ledger's own authority advances the sequence ordering a network's snapshot updates.	✓	
HO-AC-10 <code>acHoDeficitCountMovesOnlyForAWard</code> Only the ledger's own authority moves the count of positions that are underwater.	✓	

ShareClassManager

- Every configuration
 - Networks bounded to 3 symbolic networks
- Single pool
 - One symbolic pool
 - One symbolic share class
- Multi pool
 - Two symbolic pools
 - One share class per pool, its first
- Multi share class
 - Two share classes in the pinned pool

Valid State

Properties	Status	Mutations
SC-VS-01 <code>shareClassExistsIffCounted</code> The pool's first share class is on record exactly when the pool has ever created a class.	✓	 
SC-VS-02 <code>shareClassExistsIffSaltSet</code> Registration and salt are inseparable: a class is on record exactly when its metadata salt is set.	✓	 
SC-VS-03 <code>uncreatedClassIssuanceZero</code> Shares cannot be issued against a class that was never created.	✓	
SC-VS-04 <code>uncreatedClassNetworkIssuancesZero</code> A never-created class has no issuance recorded on any network.	✓	
SC-VS-05 <code>uncreatedClassNetworkRevocationsZero</code> A never-created class has no revocations recorded on any network.	✓	
SC-VS-06 <code>uncreatedClassPriceZero</code> A never-created class carries no share price.	✓	
SC-VS-07 <code>uncreatedClassPriceStampZero</code> A never-created class carries no price timestamp.	✓	
SC-VS-08 <code>existingClassPoolRegistered</code> A share class only exists inside a pool the hub registry knows.	✓	
SC-VS-09 <code>totalIssuanceIsNetworkNetSum</code> Total issuance is exactly the issued-minus-revoked net summed over the bounded network set.	✓	  

SC-VS-10 <code>priceStampNotFuture</code> Price timestamps never run ahead of the chain clock.	✓	
SC-VS-11 <code>saltPrefixIsPool</code> Every class salt is branded with its owning pool: the salt's leading bytes are the pool id.	✓	
SC-VS-12 <code>classSaltConsumed</code> The salt stored on a class is marked consumed in the global registry, so it can never be reused.	✓	
SC-VS-13 <code>classSaltSetImpliesRegisteredPerClass</code> A class carries a metadata salt only once it is on record, so no salt sits on a class that was never minted.	✓	
SC-VS-14 <code>registeredClassIndexWithinCount</code> A class on record was minted at an index the counter has already reached.	✓	
SC-VS-15 <code>poolSaltPrefixIsOwnPool</code> Each pool's class salt is branded with THAT pool, so no pool's salt carries a sibling's id.	✓	
SC-VS-16 <code>classSaltsDistinct</code> Two classes of one pool never carry the same salt, so a salt names at most one class.	✓	
SC-VS-17 <code>classSaltConsumedPerClass</code> The salt a class carries is marked consumed in the global registry, so it can never be reused.	✓	
SC-VS-18 <code>classWithinCountIsRegistered</code> A counter reaching a class's index means that class was minted.	✓	
SC-VS-19 <code>registeredClassCarriesASalt</code> A class on record always carries its salt, so no minted class is saltless.	✓	
SC-VS-20 <code>classTotalIssuanceIsNetworkNetSum</code> Each class's total issuance is exactly its own issued-minus-revoked net.	✓	
SC-VS-21 <code>classUncreatedRowsZero</code> A class that was never minted carries nothing at all.	✓	
SC-VS-22 <code>nullSaltNeverConsumed</code> The null salt is never marked consumed, so the empty salt can never block a real one.	✓	

State Transitions

Properties	Status	Mutations
SC-ST-01 <code>stScNetworkIssuedAndRevokedTotalsNeverMoveTogether</code> A network's issued and revoked totals never move in the same call.	✓	
SC-ST-02 <code>stScOneCallBooksOneNetwork</code> A call books shares for a single network, so a cross-chain transfer stays auditable leg by leg.	✓	
SC-ST-03 <code>stScCreationClaimsAFreeIdentity</code> A new class takes an unclaimed salt and marks it spent, so two classes never share an address.	✓	
SC-ST-04 <code>stScClassRowIsPerPool</code> A call that moves one pool's class row leaves the other pool's row untouched.	✓	

SC-ST-05 <code>stScNetworkLedgerIsPerPool</code> A call that books shares for one pool's class on a network leaves the other pool's ledger for that network alone.	✓	
SC-ST-06 <code>stScClassRowIsPerClass</code> A call that moves one class's row leaves the pool's other class untouched.	✓	
SC-ST-07 <code>stScNetworkLedgerIsPerClass</code> A call that books shares for one class on a network leaves the pool's other class alone on that network.	✓	
SC-ST-08 <code>stScClassCountStepsByOne</code> The class counter moves by at most one per call, with a second class within reach and not only on the pool's first.	✓	
SC-ST-09 <code>stScClassSaltIsWriteOnce</code> A class's metadata salt never changes once set.	✓	

Variable Transitions

Properties	Status	Mutations
SC-VT-01 <code>vtScNetworkIssuedTotalNeverDecreases</code> What a network has been sent is a running total no entry point may lower.	✓	
SC-VT-02 <code>vtScNetworkRevokedTotalNeverDecreases</code> What a network has given up is a running total no entry point may lower.	✓	

High Level

Properties	Status	Mutations
SC-HL-01 <code>hlScCrossChainTransferConservesClassSupply</code> Moving shares between chains cannot mint or burn them: the class supply ends where it started.	✓	
SC-HL-02 <code>hlScCrossChainTransferBooksTheRevokeLegOnTheOrigin</code> The origin of a transfer is charged with exactly what left it.	✓	
SC-HL-03 <code>hlScCrossChainTransferBooksTheIssueLegOnTheTarget</code> The target of a transfer is credited with exactly what arrived.	✓	
SC-HL-04 <code>hlScIssueRevokeRoundTripRestoresClassSupply</code> Issuing and then revoking the same amount on one network is neutral for every holder.	✓	
SC-HL-05 <code>hlScIssueRevokeRoundTripKeepsTheIssueOnRecord</code> A network's issued history is cumulative: a later revocation never walks it back.	✓	
SC-HL-06 <code>hlScIssueRevokeRoundTripKeepsTheRevocationOnRecord</code> A revocation is booked as a revocation, never as a negative issuance.	✓	
SC-HL-07 <code>hlScRevocationNeverExceedsClassSupply</code> The class supply is floored at zero, keeping a meaningful denominator under every price.	✓	
SC-HL-08 <code>hlScClassIdentityMovesOnlyOnCreation</code> Only addShareClass moves the salt registry, the class count, the record and the class salt.	✓	

SC-HL-09 <code>hlScCreationHandsOutThePreviewedId</code> A pool's first class carries exactly the id the preview announced.	✓	
SC-HL-10 <code>hlScSharePriceMovesOnlyOnRepricing</code> Only a repricing moves the published price and the stamp that dates it.	✓	
SC-HL-11 <code>hlScRepricingStoresTheGivenPriceAndStamp</code> A repricing publishes exactly the price and the stamp it was handed.	✓	
SC-HL-12 <code>hlScCreationMatchesTheIndexedPreview</code> A minted class carries exactly the id the index preview names for its position.	✓	

Reverts

Properties	Status	Mutations
SC-RV-01 <code>rvScAddShareClassRefusesACreationThatDoesNotCheckOut</code> A share class is created once, by the manager's own authority, on a salt nobody had claimed.	✓	
SC-RV-02 <code>rvScUpdateSharePriceRefusesANonWardAnUnknownClassOrAFutureStamp</code> A share price is recorded only for a known class, stamped no later than the current block.	✓	
SC-RV-03 <code>rvScUpdateSharesRefusesANonWardOrAnUnknownClass</code> A supply update does not land for a stranger, nor on a class the book never recorded.	✓	
SC-RV-04 <code>rvScIssuanceRefusesANetworkInRevocationDeficit</code> A network in revocation deficit reads as a refusal, never as a wrapped positive.	✓	
SC-RV-05 <code>rvScPreviewNextShareClassIdRefusesAPoolWhoseClassCounterIsFull</code> A pool whose class counter is full is refused a preview rather than quoted a used id.	✓	
SC-RV-06 <code>rvScPassingExistsIsReachable</code> A read taken in the transaction that created a class answers and reports that class.	✓	
SC-RV-07 <code>rvScRelyRefusesANonWard</code> The roll of parties that may operate the pool's classes is extended by a sitting ward alone.	✓	
SC-RV-08 <code>rvScDenyRefusesANonWard</code> The roll of parties that may operate the pool's classes is trimmed by a sitting ward alone.	✓	
SC-RV-09 <code>rvScIssuanceAlwaysAnswersANetworkThatIsNotInDeficit</code> A network that is not in revocation deficit always reads back, a flat one included.	✓	

Reachability

Properties	Status	Mutations
SC-RC-01 <code>rcScFirstClassCreationIsReachable</code> Witnesses a pool minting its very first share class and consuming the salt in the same call.	✓	
SC-RC-02 <code>rcScPreviewThenCreateHandsOutTheAnnouncedId</code> Witnesses a pool's first class being handed the id the preview announced, the preview then moving on.	✓	
SC-RC-03 <code>rcScExistenceProbeFlipsOnCreation</code> Witnesses the existence probe of a pool's first class flipping from false to true.	✓	

SC-RC-04 <code>rcScPricePublishedAtTheCurrentBlockIsReachable</code> Witnesses a freshly computed share price being published under the current block.	✓	
SC-RC-05 <code>rcScBackdatedPriceKeepsItsOwnStamp</code> Witnesses a price computed earlier being published under its own stamp.	✓	
SC-RC-06 <code>rcScOlderPriceReplacesTheStoredOne</code> Witnesses a stale price overwriting a fresher one.	✓	
SC-RC-07 <code>rcScPriceCanBeWrittenDownToZero</code> Witnesses a class being written down to zero without being removed.	✓	
SC-RC-08 <code>rcScIssuanceOnANetworkIsReachable</code> Witnesses shares issued on a network advancing that network's leg and the class supply.	✓	
SC-RC-09 <code>rcScRevocationOnANetworkIsReachable</code> Witnesses shares revoked on a network advancing its revoked leg as the supply falls.	✓	
SC-RC-10 <code>rcScOneNetworkRoundTripIsReachable</code> Witnesses a network round trip returning the class supply to where it started.	✓	
SC-RC-11 <code>rcScCrossNetworkTransferLeavesTheThirdNetworkUntouched</code> Witnesses a cross network transfer leaving the class supply unchanged and a third network untouched.	✓	
SC-RC-12 <code>rcScEveryModelledNetworkCanCarrySupply</code> Witnesses every modelled network carrying supply at once, the total being the sum of the legs.	✓	
SC-RC-13 <code>rcScClassSupplyCanReachItsStoredCeiling</code> Witnesses an empty class being issued straight up to the ceiling its supply can hold.	✓	
SC-RC-14 <code>rcScClassCanBeFullyRetired</code> Witnesses a class being retired in full by a single revocation.	✓	
SC-RC-15 <code>rcScRetiredClassCanBeRefilled</code> Witnesses a fully retired class being issued into again.	✓	
SC-RC-16 <code>rcScZeroMovementIsAdmittedAndBooksNothing</code> Witnesses a zero share movement being admitted and booking nothing.	✓	
SC-RC-17 <code>rcScPerNetworkViewReportsTheNetAfterAnIssuance</code> Witnesses the per network view reporting what a network was sent minus what was taken back.	✓	
SC-RC-18 <code>rcScNetworkRevocationsCanExceedItsIssuances</code> Witnesses a network revoking past its own issuances while the class supply stays positive.	✓	
SC-RC-19 <code>rcScRelyGrantsAuthority</code> Witnesses authority being granted: a party holding no seat is relied on and holds one afterwards.	✓	
SC-RC-20 <code>rcScDenyWithdrawsAuthority</code> Witnesses authority being withdrawn: a party that already held a seat is denied and holds none afterwards.	✓	
SC-RC-21 <code>rcScRelyThenDenyRoundTripIsReachable</code> Witnesses a seat granted and taken back in one transaction.	✓	

Access Control

Properties	Status	Mutations
SC-AC-01 <code>acScOnlyAWardMovesTheWardSet</code> Only an address already holding it can grant or withdraw the power to operate a pool's classes.	✓	
SC-AC-02 <code>acScOnlyAWardMovesTheClassCounter</code> Only the manager's own authority moves the number of share classes a pool has issued.	✓	
SC-AC-03 <code>acScOnlyAWardMovesTheClassRoster</code> Only the manager's own authority puts a share class on the pool's roster or takes it off.	✓	
SC-AC-04 <code>acScOnlyAWardMovesTheSaltRegistry</code> Only the manager's own authority moves the registry of consumed salts.	✓	
SC-AC-05 <code>acScOnlyAWardMovesTheClassSalt</code> Only the manager's own authority moves the salt recorded on a share class.	✓	
SC-AC-06 <code>acScOnlyAWardMovesTheClassSupply</code> Only the manager's own authority moves the supply of a share class.	✓	
SC-AC-07 <code>acScOnlyAWardMovesANetworkIssuedTotal</code> Only the manager's own authority moves the shares a network is on record as having issued.	✓	
SC-AC-08 <code>acScOnlyAWardMovesANetworkRevokedTotal</code> Only the manager's own authority moves the shares a network is on record as having revoked.	✓	
SC-AC-09 <code>acScOnlyAWardMovesTheSharePrice</code> Only the manager's own authority moves the price per share of a class.	✓	
SC-AC-10 <code>acScOnlyAWardMovesThePriceTimestamp</code> Only the manager's own authority moves the timestamp saying how fresh a class price is.	✓	

HubRegistry

- Single pool
 - One symbolic pool
- Multi pool
 - Two symbolic pools

Valid State

Properties	Status	Mutations
HR-VS-01 <code>assetDecimalsCapped</code> No asset carries more precision than the 18-decimal registration cap.	✓	
HR-VS-02 <code>unregisteredAssetHasZeroDecimals</code> An unregistered asset row holds no leftover precision.	✓	
HR-VS-03 <code>nullAssetNeverRegistered</code>	✓	

The null asset id never enters the registry.		
HR-VS-04 <code>poolCurrencyRegistered</code> The pool's denomination currency is always a registered asset.	✓	
HR-VS-05 <code>managerImpliesPoolExists</code> Nobody holds manager rights on a pool before that pool is created.	✓	
HR-VS-06 <code>zeroAddressNeverManager</code> The zero address is never a pool manager.	✓	
HR-VS-07 <code>policyImpliesPoolExists</code> A policy is only ever installed on a created pool.	✓	
HR-VS-08 <code>nonceImpliesPoolExists</code> The policy nonce only ever advances on a created pool.	✓	
HR-VS-09 <code>policyImpliesNonzeroNonce</code> An installed policy sits at nonce one or higher.	✓	
HR-VS-10 <code>bridgingHookImpliesPoolExists</code> A bridging hook is only ever wired to a created pool.	✓	
HR-VS-11 <code>requestManagerImpliesPoolExists</code> A hub request manager, on any network, is only ever wired to a created pool.	✓	
HR-VS-12 <code>pendingAuthAboveEnvFloor</code> A pending authorization always matures above the scene's minimum block time.	✓	
HR-VS-13 <code>pendingNeverUnderNullPolicy</code> No authorization is ever scheduled in the namespace of an uninstalled policy.	✓	
HR-VS-14 <code>pendingNeverUnderZeroNonce</code> The zero-nonce namespace never holds a pending authorization.	✓	
HR-VS-15 <code>pendingNeverAheadOfNonce</code> Namespaces ahead of the pool's current policy nonce hold no pending entries.	✓	
HR-VS-16 <code>pendingUnderCurrentNonceMatchesPolicy</code> Within the current nonce tenure only the installed policy's own namespace holds pending entries.	✓	
HR-VS-17 <code>nullAuthIdNeverPending</code> The null authorization id never carries a pending entry: every grant is keyed by a derived id.	✓	

State Transitions

Properties	Status	Mutations
HR-ST-01 <code>stHrPolicySwapOpensANewTenure</code> Swapping a pool's policy always opens a new tenure, inheriting no schedule.	✓	
HR-ST-02 <code>stHrNewTenureStartsWithNoSchedule</code> A tenure starts with an empty schedule, so nothing waits already matured when a new policy takes over.	✓	
HR-ST-03 <code>stHrSchedulingMaturesAheadAndInTheLiveTenure</code> A newly scheduled authorization matures strictly later than the moment it was scheduled.	✓	

HR-ST-04 <code>stHrDischargeLeavesTheTenureStanding</code> An authorization leaves the ledger only out of the sitting tenure, and discharging it leaves that tenure standing.	✓	
HR-ST-05 <code>stHrPendingMovesOnlyOffOrOntoZero</code> A pending grant appears from nothing or clears to nothing, never restamped in place.	✓	
HR-ST-06 <code>stHrRegisteredAssetKeepsItsRow</code> A registered asset stays registered and its decimals never move.	✓	
HR-ST-07 <code>stHrCurrencyDecimalsAreStable</code> Re-denominating a pool never changes the precision of its unit of account.	✓	
HR-ST-08 <code>stHrCurrencyNeverClears</code> A pool that holds a currency never has it cleared, so a created pool can never silently become uncreated.	✓	
HR-ST-09 <code>stHrPoolRowIsPerPool</code> A call that moves one pool's registry row leaves the other pool's row untouched.	✓	
HR-ST-10 <code>stHrManagerRightsArePerPool</code> A call that grants or revokes manager rights on one pool leaves the other pool's roster alone.	✓	
HR-ST-11 <code>stHrRequestManagerIsPerPool</code> A call that wires one pool's request manager on a network leaves the other pool's wiring on that network alone.	✓	
HR-ST-12 <code>stHrPendingAuthIsPerPool</code> Moving one pool's authorization namespace leaves every namespace of the other pool alone.	✓	
HR-ST-13 <code>stHrOperatorSeatNeedsItsOwnPoolsCurrency</code> Nobody is handed the operator seat of a pool that never gains a currency of its own.	✓	

Variable Transitions

Properties	Status	Mutations
HR-VT-01 <code>vtHrManagerRosterMovesOneAccountAtATime</code> One call moves the manager rights of at most one account.	✓	
HR-VT-02 <code>vtHrTenureCounterAdvancesByAtMostOne</code> A pool's authorization tenure advances one step at a time and never rewinds.	✓	
HR-VT-03 <code>vtHrRequestManagerMovesOneNetworkAtATime</code> Wiring a pool's request manager on one network never re-routes another.	✓	
HR-VT-04 <code>vtHrAuthLedgerMovesOneEntryAtATime</code> One call moves the timelock entry of at most one privileged call.	✓	

High Level

Properties	Status	Mutations
HR-HL-01 <code>hlHrAuthorizationIsSpentOnlyOnce</code>		

One veto window buys at most one privileged call: a grant is spent by what it funds.	✓	
HR-HL-02 <code>hlHrConsumeSpendsOnlyALiveEntry</code> A consumption that succeeds spent a grant that was really on the ledger.	✓	
HR-HL-03 <code>hlHrConsumeWaitsOutTheVetoDelay</code> The veto window cannot be skipped: a consumption never fires before the grant it spends has matured.	✓	
HR-HL-04 <code>hlHrConsumeRefusesASTaleGrant</code> A stale grant fails closed against the expiry window its consuming policy names.	✓	
HR-HL-05 <code>hlHrCanceledAuthorizationNeverFires</code> A veto is final: an authorization that has been canceled can never fund a consumption afterwards.	✓	
HR-HL-06 <code>hlHrAuthLedgerMovesOnlyThroughItsThreeWriters</code> The authorization ledger moves only through its own three entry points.	✓	
HR-HL-07 <code>hlHrManagerRightsMoveOnlyThroughTheirTwoWriters</code> Pool manager rights move only through pool registration and the roster update.	✓	
HR-HL-08 <code>hlHrScheduledCallMaturesAfterExactlyThePolicyDelay</code> A scheduled call waits exactly as long as the pool's own policy asks.	✓	
HR-HL-09 <code>hlHrPoolCreationSeatsTheAccountItNames</code> Creating a pool hands the manager seat to the account the creation names.	✓	
HR-HL-10 <code>hlHrTokenPrecisionMatchesTheAssetRow</code> The precision read answers with the precision the registry recorded for that asset.	✓	
HR-HL-11 <code>hlHrSetPolicyInstallsExactlyTheAddressItNames</code> Installing a rule book leaves the pool running under the address the call named.	✓	

Reverts

Properties	Status	Mutations
HR-RV-01 <code>rvHrRegisterAssetRefusesANonWardAnOversizedPrecisionOrARRepeat</code> An asset is registered once, by the registry's own authority.	✓	
HR-RV-02 <code>rvHrRegisterPoolRefusesAnUnusableOrRepeatedOpening</code> A pool is created once, by the registry's own authority.	✓	
HR-RV-03 <code>rvHrUpdateManagerRefusesANonWardAnAbsentPoolOrANullSeat</code> A manager seat moves only by the registry's own authority, on a pool that exists.	✓	
HR-RV-04 <code>rvHrUpdateCurrencyRefusesANonWardAnAbsentPoolOrARescaling</code> A pool is re-denominated only into a registered currency at the precision it already accounts in.	✓	
HR-RV-05 <code>rvHrSetPolicyRefusesANonWardOrAnAbsentPool</code> A policy is installed only by the registry's own authority, on a pool that exists.	✓	
HR-RV-06 <code>rvHrSetHubRequestManagerRefusesANonWardOrAnAbsentPool</code> A request manager is wired only by the registry's own authority, on a pool that exists.	✓	

HR-RV-07 <code>rvHrSetBridgingHookIsNeverRefusedToAWardOnALivePool</code> A live pool's bridging hook can always be wired, and unwired, by the registry's authority.	✓	
HR-RV-08 <code>rvHrInitiateAuthorizationRefusesANonWardAnInPolicyCallOrATakenSlot</code> A call goes on the timelock only where the pool's policy rates it out of policy.	✓	
HR-RV-09 <code>rvHrCancelAuthorizationRefusesANonWardOrAnEmptySlot</code> A cancel of nothing writes no audit entry, and only the registry's authority vetoes.	✓	
HR-RV-10 <code>rvHrConsumeAuthorizationIsNeverRefusedToThePolicyInsideItsWindow</code> A matured grant is spendable by the pool's policy for the whole of its window, first instant included.	✓	
HR-RV-11 <code>rvHrTokenDecimalsRefusesAnAssetTheRegistryDoesNotCarry</code> An unknown asset never reads back as zero decimals; the read refuses instead.	✓	
HR-RV-12 <code>rvHrAuthIdAlwaysAnswers</code> The identifier naming a pending authorization is computable in every state.	✓	
HR-RV-13 <code>rvHrRetiredGrantIsNeverConsumed</code> Installing a policy retires the grants the outgoing tenure scheduled, re-installs included.	✓	
HR-RV-14 <code>rvHrRelyRefusesANonWard</code> The registry's ward roll is extended by a sitting ward alone.	✓	
HR-RV-15 <code>rvHrDenyRefusesANonWard</code> The registry's ward roll is trimmed by a sitting ward alone.	✓	
HR-RV-16 <code>rvHrRegisterAssetIsNeverRefusedToAWardOnAFreshAssetWithinTheCap</code> An asset at any precision the cap admits can always be brought onto the registry.	✓	

Reachability

Properties	Status	Mutations
HR-RC-01 <code>rcHrRegisterAssetIsReachable</code> Witnesses an unknown asset joining the registry and carrying its own precision into the asset row.	✓	
HR-RC-02 <code>rcHrRegisterAssetFromAnEmptyRegistryIsReachable</code> Witnesses the first asset of a deployment being registered while the registry still holds nothing.	✓	
HR-RC-03 <code>rcHrRegisterAssetBesideAnExistingOneIsReachable</code> Witnesses a second asset joining the registry while the asset already on it keeps its own row untouched.	✓	
HR-RC-04 <code>rcHrRegisterAssetAtThePrecisionCapIsReachable</code> Witnesses the documented 18 decimal cap being inclusive, an asset registering at exactly the cap.	✓	
HR-RC-05 <code>rcHrRegisterAssetWithZeroPrecisionIsReachable</code> Witnesses a zero precision asset being legal, registering with a decimals of zero.	✓	
HR-RC-06 <code>rcHrRegisterPoolIsReachable</code> Witnesses a pool being created on a registered currency with its first manager seated in the same call.	✓	

HR-RC-07 <code>rcHrRegisterPoolOnAZeroPrecisionCurrencyIsReachable</code> Witnesses a pool being denominated in an asset that carries no precision at all.	✓	
HR-RC-08 <code>rcHrRegisterAssetThenRegisterPoolIsReachable</code> Witnesses a deployment brought up from nothing: an asset registered and a pool denominated in it.	✓	
HR-RC-09 <code>rcHrUpdateManagerSeatsAManagerIsReachable</code> Witnesses a vacant manager seat being filled on a live pool.	✓	
HR-RC-10 <code>rcHrUpdateManagerRevokesASeatIsReachable</code> Witnesses a held manager seat being taken back.	✓	
HR-RC-11 <code>rcHrManagerSeatRoundTripIsReachable</code> Witnesses a manager seat being granted and taken back again inside one transaction.	✓	
HR-RC-12 <code>rcHrUpdateCurrencyRedenominatesThePoolIsReachable</code> Witnesses a live pool being re-denominated onto another asset of the same precision.	✓	
HR-RC-13 <code>rcHrSetBridgingHookAttachesIsReachable</code> Witnesses a bridging hook being attached to a pool that had none.	✓	
HR-RC-14 <code>rcHrSetBridgingHookDetachesIsReachable</code> Witnesses an attached bridging hook being taken off again.	✓	
HR-RC-15 <code>rcHrBridgingHookRoundTripIsReachable</code> Witnesses a bridging hook being attached and taken off again inside one transaction.	✓	
HR-RC-16 <code>rcHrSetHubRequestManagerWiresANetworkIsReachable</code> Witnesses a request manager being wired for a network the pool had not wired yet.	✓	
HR-RC-17 <code>rcHrSetPolicyInstallsTheFirstPolicyIsReachable</code> Witnesses a pool that never carried a policy having its first one installed.	✓	
HR-RC-18 <code>rcHrSetPolicyUninstallsWithTheZeroAddressIsReachable</code> Witnesses a policy being uninstalled with the zero address, the tenure advancing all the same.	✓	
HR-RC-19 <code>rcHrSetPolicyAtTheLastNamespaceIsReachable</code> Witnesses a pool's very last namespace still being installable, one tenure below the nonce ceiling.	✓	
HR-RC-20 <code>rcHrOrphanedScheduleCanBeRemadeAfterAPolicySwapIsReachable</code> Witnesses a policy swap orphaning a grant, so the same call can be put on the clock again.	✓	
HR-RC-21 <code>rcHrInitiateAuthorizationIsReachable</code> Witnesses an out of policy call being put on the clock, maturing strictly later than the moment it was scheduled.	✓	
HR-RC-22 <code>rcHrInitiateAuthorizationWithTheEmptyPayloadIsReachable</code> Witnesses the shortest schedulable payload, an empty calldata blob, going onto the clock.	✓	
HR-RC-23 <code>rcHrInitiateAuthorizationAtTheMaturityCeilingIsReachable</code> Witnesses a maturity stamp reaching the ceiling its stored width allows.	✓	
HR-RC-24 <code>rcHrCancelAuthorizationIsReachable</code> Witnesses a pending grant being withdrawn before it ever matures, which is what the veto window is for.	✓	

HR-RC-25 <code>rcHrConsumeAuthorizationIsReachable</code> Witnesses a matured grant being fired by the pool's own installed policy, which clears it off the ledger.	✓	
HR-RC-26 <code>rcHrConsumeAuthorizationLateInsideTheWindowIsReachable</code> Witnesses a late consumption still firing while it is inside the window its policy named.	✓	
HR-RC-27 <code>rcHrConsumeAuthorizationAtTheMaturityInstantIsReachable</code> Witnesses the lower edge of the veto window being inclusive, a consumption firing at the exact maturity instant.	✓	
HR-RC-28 <code>rcHrConsumeAuthorizationAtTheExpiryEdgeIsReachable</code> Witnesses the upper edge of the window being inclusive, a consumption firing exactly expiry seconds past maturity.	✓	
HR-RC-29 <code>rcHrRelyIsReachable</code> Witnesses authority being granted to an account that held none.	✓	
HR-RC-30 <code>rcHrDenyIsReachable</code> Witnesses authority being revoked from an account that held it.	✓	
HR-RC-31 <code>rcHrAuthorityRoundTripIsReachable</code> Witnesses a grant of authority being fully reversible inside one transaction.	✓	
HR-RC-32 <code>rcHrAssetRegistrationViewFlipsIsReachable</code> Witnesses the public registration view turning an unknown asset into a known one across a registration.	✓	
HR-RC-33 <code>rcHrPoolExistenceViewFlipsOnRegistrationIsReachable</code> Witnesses the pool existence view turning from false to true across a registration.	✓	
HR-RC-34 <code>rcHrPolicyViewsReportTheInstallIsReachable</code> Witnesses the policy views reporting the address just installed and the tenure that install opened.	✓	
HR-RC-35 <code>rcHrAuthIdChangesAcrossAPolicySwapIsReachable</code> Witnesses one payload answering to two different authorization ids on either side of a policy change.	✓	

Access Control

Properties	Status	Mutations
HR-AC-01 <code>acHrWardRightsMoveOnlyForAWard</code> Only an address already holding registry rights can hand them out or take them back.	✓	
HR-AC-02 <code>acHrAssetRowMovesOnlyForAWard</code> Only the registry's own authority registers an asset and sets the precision every price reads.	✓	
HR-AC-03 <code>acHrPoolCurrencyMovesOnlyForAWard</code> Only the registry's own authority creates a pool or points it at a currency.	✓	
HR-AC-04 <code>acHrManagerRightsMoveOnlyForAWard</code> Only the registry's own authority grants or withdraws the right to manage a pool.	✓	
HR-AC-05 <code>acHrPolicyRowMovesOnlyForAWard</code> Only the registry's own authority installs the contract that vets a pool's privileged calls.	✓	

HR-AC-06 <code>acHrBridgingHookMovesOnlyForAWard</code> Only the registry's own authority wires a pool's bridging hook.	✓	
HR-AC-07 <code>acHrRequestManagerMovesOnlyForAWard</code> Only the registry's own authority points the request manager a pool answers to.	✓	
HR-AC-08 <code>acHrAuthorizationIsScheduledOnlyByAWard</code> Only the registry's own authority can start the veto clock on a privileged call.	✓	
HR-AC-09 <code>acHrPendingAuthorizationMovesOnlyForAWardOrThePolicy</code> A timelock entry moves only for the registry's authority or the pool's own installed policy.	✓	

Spoke

- One symbolic pool
- One symbolic share class
- One ERC20 asset row in the escrow
- ERC20 and ERC6909 tokens modeled in CVL (standard, up to 3 holders, uint128 balances, 6 to 18 decimals, no share token hooks)

State Transitions

Properties	Status	Mutations
SP-ST-01 <code>stSpQueuedMoveIsBackedByTheEscrow</code> A move queued for the hub is backed by exactly one move of the escrow's spendable balance.	✓	

Variable Transitions

Properties	Status	Mutations
SP-VT-01 <code>vtSpBridgeNeverAnnouncesAnEmptyTransfer</code> The spoke never announces a cross-chain share transfer of nothing.	✓	
SP-VT-02 <code>vtSpBridgeNeverAnnouncesBackToThisNetwork</code> A cross-chain share transfer is never announced back to this same network.	✓	
SP-VT-03 <code>vtSpAssetRegistrationNeverAnnouncesTheNullId</code> Every asset registration the spoke announces carries a real asset number.	✓	
SP-VT-04 <code>vtSpAssetRegistrationRespectsTheDecimalsCeiling</code> An announced registration never claims a precision above the protocol ceiling.	✓	
SP-VT-05 <code>vtSpOneCallHandsOverAtMostOneMessage</code> One call on the spoke costs the pool at most one outbound message.	✓	

High Level

Properties	Status	Mutations
SP-HL-10 <code>hlSpBridgeDebitsTheNamedOwner</code> A bridge transfer takes the shares from the owner the caller named.	✓	

SP-HL-11 <code>hlSpBridgeBurnsTheBridgedShares</code> The shares a bridge transfer takes are destroyed on this network.	✓	
SP-HL-12 <code>hlSpBridgeParksNoSharesOnTheSpoke</code> A bridge transfer parks no shares on the spoke.	✓	
SP-HL-13 <code>hlSpBridgeAnnouncesExactlyOneTransfer</code> Every bridge transfer announces itself to the destination chain exactly once.	✓	
SP-HL-14 <code>hlSpBridgeAnnouncesTheBurnedAmount</code> The outbound announcement carries the amount the spoke burned.	✓	
SP-HL-15 <code>hlSpBridgeAnnouncesTheNamedReceiver</code> The outbound announcement carries the receiver the caller named.	✓	
SP-HL-32 <code>hlSpBridgeSpendsTheOwnersApproval</code> A bridge transfer spends the approval the owner granted, not a forced transfer.	✓	
SP-HL-33 <code>hlSpBridgeReportsNoShareMovementToTheHub</code> A bridge transfer tells the hub nothing about the share count.	✓	
SP-HL-34 <code>hlSpNoteDepositMovesNoTokens</code> Counting assets that reached the escrow by another route moves no tokens.	✓	

Reverts

Properties	Status	Mutations
SP-RV-01 <code>rvSpFileRefusesANonWardOrAnUnknownName</code> The spoke is never rewired by an outsider, nor under a name it does not wire.	✓	
SP-RV-02 <code>rvSpDepositRefusesANonManager</code> Paying assets into a pool's escrow is closed to anyone the pool has not seated as a manager.	✓	
SP-RV-03 <code>rvSpNoteDepositRefusesANonManager</code> Crediting assets that arrived outside a deposit needs the manager seat.	✓	
SP-RV-04 <code>rvSpWithdrawRefusesANonManager</code> Paying assets out of a pool's escrow is closed to anyone the pool has not seated as a manager.	✓	
SP-RV-05 <code>rvSpWithdrawReservedRefusesANonManager</code> Releasing a hold and paying it out needs the manager seat.	✓	
SP-RV-06 <code>rvSpReserveRefusesANonManager</code> Placing a hold on a pool's assets needs the manager seat.	✓	
SP-RV-07 <code>rvSpUnreserveRefusesANonManager</code> Releasing a hold needs the manager seat.	✓	
SP-RV-08 <code>rvSpSubmitQueuedAssetsRefusesANonManager</code> Reporting a pool's queued asset movements to the hub needs the manager seat.	✓	
SP-RV-09 <code>rvSpIssueRefusesANonManager</code> Minting share tokens needs the manager seat.	✓	
SP-RV-10 <code>rvSpRevokeRefusesANonManager</code>	✓	

Burning share tokens needs the manager seat.		
SP-RV-11 <code>rvSpWithdrawSharesRefusesANonManager</code> Moving share tokens parked in the pool escrow needs the manager seat.	✓	
SP-RV-12 <code>rvSpSubmitQueuedSharesRefusesANonManager</code> Reporting a pool's queued share movements to the hub needs the manager seat.	✓	
SP-RV-13 <code>rvSpTransferSharesFromRefusesANonManager</code> The forced transfer entry needs the manager seat.	✓	
SP-RV-14 <code>rvSpBridgeRefusesAStrangerToTheOwner</code> A bridge transfer may be sent only by the named owner or by a ward acting for it.	✓	
SP-RV-15 <code>rvSpBridgeRefusesANonBridgerOwner</code> Shares leave the chain only for an owner the pool has registered as a bridger.	✓	
SP-RV-16 <code>rvSpBridgeRefusesAZeroAmount</code> A bridge transfer of nothing is refused.	✓	
SP-RV-17 <code>rvSpBridgeRefusesALocalDestination</code> A bridge transfer addressed to this very network is refused.	✓	
SP-RV-18 <code>rvSpRequestRefusesAnyCallerButTheRegisteredManager</code> Forwarding a request to the hub is open to the pool's registered request manager alone.	✓	
SP-RV-19 <code>rvSpRelyRefusesANonWard</code> Spoke authority is handed on only by an account that already holds it.	✓	
SP-RV-20 <code>rvSpDenyRefusesANonWard</code> Spoke authority is withdrawn only by an account that holds it.	✓	
SP-RV-21 <code>rvSpSubmitQueuedAssetsIsNeverRefusedToASeatedManager</code> With no rule book installed, a seated manager attaching no value hands the pool's queued asset flows over without a ward seat.	✓	
SP-RV-22 <code>rvSpSubmitQueuedSharesIsNeverRefusedToASeatedManager</code> With no rule book installed, a seated manager attaching no value hands the pool's queued share net over without a ward seat.	✓	
SP-RV-23 <code>rvSpManagerCallIsNeverRefusedToAnyCaller</code> The manager call is open to every caller, as the interface says it is by choice.	✓	

Reachability

Properties	Status	Mutations
SP-RC-01 <code>rcSpRegisterAssetMintsAFreshIdForAnUnknownErc20</code> Witnesses an ERC20 the network has never seen being announced and minting its first asset id.	✓	
SP-RC-02 <code>rcSpRegisterAssetReannouncesTheStoredIdIsReachable</code> Witnesses a repeat announcement reaching the hub with the id minted the first time.	✓	
SP-RC-03 <code>rcSpRegisterAssetMintsAFreshIdForAnErc6909Tranche</code> Witnesses an ERC6909 tranche being announced through the per id metadata branch.	✓	
SP-RC-04 <code>rcSpRegisterAssetAtTheDecimalsCeilingIsReachable</code>		

Witnesses a token sitting exactly at the decimals ceiling still being announced.	✓	
SP-RC-05 <code>rcSpFirstDepositIntoAnEmptyRowIsReachable</code> Witnesses the first deposit into a pool, every unit spendable and one increase queued.	✓	
SP-RC-06 <code>rcSpDepositIntoAHoldingRowIsReachable</code> Witnesses a later deposit into a row that already holds custody.	✓	
SP-RC-07 <code>rcSpDepositOfThePayersWholeBalanceIsReachable</code> Witnesses a payer emptied by a single deposit.	✓	
SP-RC-08 <code>rcSpDepositOfAnErc6909TrancheIsReachable</code> Witnesses a deposit of an ERC6909 tranche, moving units by token id.	✓	
SP-RC-09 <code>rcSpNoteDepositCreditsCustodyWithoutMovingTokens</code> Witnesses the accounting only credit: custody is booked while the token balance never moves.	✓	
SP-RC-10 <code>rcSpZeroAmountDepositIsReachable</code> Witnesses a zero amount deposit accepted and still reaching the queue.	✓	
SP-RC-11 <code>rcSpWithdrawPaysTheReceiverIsReachable</code> Witnesses a normal payout: custody falls, the receiver is paid the tokens and one decrease is queued for the hub.	✓	
SP-RC-12 <code>rcSpWithdrawEmptiesTheFreeBalanceExactlyIsReachable</code> Witnesses a pool taking out everything it may spend in one call.	✓	
SP-RC-13 <code>rcSpWithdrawLeavesTheHeldPartBehindIsReachable</code> Witnesses a payout of exactly the unheld part while the hold stays where it was.	✓	
SP-RC-14 <code>rcSpWithdrawToTheCallerThemselvesIsReachable</code> Witnesses a manager paying a withdrawal out to their own account.	✓	
SP-RC-15 <code>rcSpFirstHoldOnARowIsReachable</code> Witnesses the first hold on a row, custody untouched and a decrease queued.	✓	
SP-RC-16 <code>rcSpHoldOnTheWholeFreeBalanceIsReachable</code> Witnesses a row being held in full, leaving the pool nothing to spend on that asset.	✓	
SP-RC-17 <code>rcSpReleaseOfTheWholeHoldIsReachable</code> Witnesses a hold released in full and the increase queued for the hub.	✓	
SP-RC-18 <code>rcSpWithdrawReservedPaysHeldFundsOutIsReachable</code> Witnesses held funds paid out, the hold consumed and nothing reported twice.	✓	
SP-RC-19 <code>rcSpIssueMintsSharesIsReachable</code> Witnesses shares minted for an investor and one issuance queued.	✓	
SP-RC-20 <code>rcSpZeroShareIssuanceIsReachable</code> Witnesses a zero share issuance accepted and still queued.	✓	
SP-RC-21 <code>rcSpRevokeBurnsTheCallersSharesIsReachable</code> Witnesses shares revoked, destroyed, and one revocation queued.	✓	
SP-RC-22 <code>rcSpRevokeDownToAnEmptyClassIsReachable</code> Witnesses a share class wound down to nothing in one call.	✓	

SP-RC-23 <code>rcSpWithdrawSharesHandsCustodyOutIsReachable</code> Witnesses shares leaving the pool escrow for an investor, supply and report unmoved.	✓	
SP-RC-24 <code>rcSpTransferSharesFromMovesAHoldingIsReachable</code> Witnesses a manager moving shares between two holders.	✓	
SP-RC-25 <code>rcSpSubmitQueuedAssetsReportsToTheHubIsReachable</code> Witnesses the asset side of a pool's report reaching the hub.	✓	
SP-RC-26 <code>rcSpSubmitQueuedSharesWithNoExtraGasIsReachable</code> Witnesses the share side of a pool's report reaching the hub with no extra gas budget.	✓	
SP-RC-27 <code>rcSpBridgeDrivenByTheOwnerIsReachable</code> Witnesses the origin leg of a bridge driven by the share owner themselves.	✓	
SP-RC-28 <code>rcSpBridgeDrivenByAWardForTheOwnerIsReachable</code> Witnesses a ward bridging shares on a named owner's behalf, the owner's own balance paying for it.	✓	
SP-RC-29 <code>rcSpBridgeOfTheWholeHoldingIsReachable</code> Witnesses an account bridging out entirely in one call.	✓	
SP-RC-30 <code>rcSpBridgeThroughTheShortEntryPointIsReachable</code> Witnesses the short bridging entry point working with no remote gas budget.	✓	
SP-RC-31 <code>rcSpPaidRequestIsRelayedIsReachable</code> Witnesses an investor request relayed to the hub in paid mode.	✓	
SP-RC-32 <code>rcSpUnpaidRequestIsRelayedIsReachable</code> Witnesses an investor request relayed to the hub under the deferred payment flag, so the request entry point is live in both of its payment modes.	✓	
SP-RC-33 <code>rcSpManagerCallFromAnUnseatedCallerIsReachable</code> Witnesses a caller holding no seat relaying a manager call to the hub side.	✓	
SP-RC-34 <code>rcSpRelyGrantsAFreshWardIsReachable</code> Witnesses a ward extending spoke authority to an account that held none, the grant half of the permission pair.	✓	
SP-RC-35 <code>rcSpRelyThenDenyRoundTripIsReachable</code> Witnesses the full authority round trip: a seat granted here can be taken back again in the same transaction.	✓	
SP-RC-36 <code>rcSpDepositThenWithdrawRoundTripIsReachable</code> Witnesses a complete custody round trip, both legs reported to the hub.	✓	
SP-RC-37 <code>rcSpHoldThenPayOutIsReachable</code> Witnesses funds held and then paid to a third party, the hub told once.	✓	
SP-RC-38 <code>rcSpIssueThenWithdrawSharesIsReachable</code> Witnesses the claim path end to end, only the mint reported to the hub.	✓	
SP-RC-39 <code>rcSpDepositThenSubmitQueuedAssetsIsReachable</code> Witnesses a pool filling its asset queue and reporting it in the same transaction, the pairing the queue exists for.	✓	
SP-RC-40 <code>rcSpHoldThenReleaseRoundTripIsReachable</code> Witnesses a hold placed and lifted in one transaction, both legs reported.	✓	

SP-RC-41 <code>rcSpWithdrawOfAnErc6909TrancheIsReachable</code> Witnesses a payout of an ERC6909 tranche, handing units back by token id.	✓	
SP-RC-42 <code>rcSpFileRepointsTheGatewayIsReachable</code> Witnesses a ward repointing the route every outbound message and its payment travel.	✓	
SP-RC-43 <code>rcSpFileRepointsTheMessageSenderIsReachable</code> Witnesses a ward repointing the contract that composes outbound messages.	✓	
SP-RC-44 <code>rcSpHoldBeyondTheRowsCustodyIsReachable</code> Witnesses a hold larger than everything the row carries, a forward booking.	✓	

Access Control

Properties	Status	Mutations
SP-AC-01 <code>acSpWardListMovesOnlyForAWard</code> The list of privileged accounts changes only at the direction of an account already on it.	✓	 
SP-AC-02 <code>acSpGatewayIsRepointedOnlyByAWard</code> Only a ward repoints the route every outbound message and its payment travels.	✓	
SP-AC-03 <code>acSpMessageSenderIsRepointedOnlyByAWard</code> Only a ward repoints the contract that composes the spoke's outbound messages.	✓	
SP-AC-04 <code>acSpQueuedAssetDeltasComeOnlyFromAManager</code> Asset movements the hub will later settle are queued through the spoke only by a pool manager.	✓	
SP-AC-05 <code>acSpQueuedShareDeltasComeOnlyFromAManager</code> Share movements the hub will later settle are queued through the spoke only by a pool manager.	✓	
SP-AC-06 <code>acSpEscrowCustodyMovesOnlyForAManager</code> Custody of the pool's escrowed assets moves through the spoke only for a pool manager.	✓	
SP-AC-07 <code>acSpQueueSubmissionsLeaveOnlyForAManager</code> Through the spoke, only a pool manager submits the pool's queued position to the hub.	✓	
SP-AC-08 <code>acSpShareSupplyGrowsThroughTheSpokeOnlyForAManager</code> Through the spoke, only a pool manager brings new shares into existence.	✓	
SP-AC-09 <code>acSpUnauthorizedCannotReduceAHolderBalance</code> Through the spoke, a third party who is neither the holder, their approver nor a manager takes no tokens.	✓	
SP-AC-10 <code>acSpApprovalToTheSpokeIsSpentOnlyByItsOwnerOrAWard</code> The approval a holder grants the spoke is spent only by that holder or by a ward.	✓	
SP-AC-11 <code>acSpBridgeStartsOnlyForABridgerOrAWard</code> Shares leave this network only for a listed bridger or a ward.	✓	
SP-AC-12 <code>acSpInvestorRequestsLeaveOnlyForTheRequestManager</code> Through the spoke, an investor request reaches the hub only for the pool's own request manager.	✓	

SpokeRegistry

- One symbolic pool
- One symbolic share class

Valid State

Properties	Status	Mutations
SR-VS-01 <code>poolCreatedAtNotFuture</code> The pool's creation record always points at a block that has already happened.	✓	
SR-VS-02 <code>poolCreatedAboveEnvFloor</code> A created pool's stamp sits above the scene's minimum block time.	✓	
SR-VS-03 <code>shareTokenIffRegistrar</code> A share class comes online with its token and registrar together, neither half alone.	✓	
SR-VS-04 <code>shareClassImpliesActivePool</code> A share class only lives inside a pool that was actually created.	✓	
SR-VS-05 <code>tokenRowScIdImpliesPoolId</code> The token lookup row is written whole: a class half never appears without its pool reference.	✓	
SR-VS-06 <code>shareTokenHasCanonicalReverseRow</code> The registered share token resolves back to its pool and class, so token lookups agree with the class registry.	✓	
SR-VS-07 <code>onlyCurrentShareTokenHoldsALookupRow</code> On the pinned class, the only live token lookup row is that class's own share token.	✓	
SR-VS-08 <code>nullTokenRowPoolIdZero</code> The zero address never owns a token lookup row.	✓	
SR-VS-09 <code>nullTokenRowScIdZero</code> The token lookup row the zero address would own names no share class.	✓	
SR-VS-10 <code>requestManagerImpliesActivePool</code> A request manager can only be installed on a created pool.	✓	
SR-VS-11 <code>managerImpliesActivePool</code> Manager rights exist only on a created pool.	✓	
SR-VS-12 <code>bridgerImpliesActivePool</code> Bridger rights exist only on a created pool.	✓	
SR-VS-13 <code>policyImpliesNonzeroNonce</code> An installed policy always carries a nonzero policy nonce.	✓	
SR-VS-14 <code>policyNonceImpliesActivePool</code> The policy nonce only starts moving on a created pool.	✓	
SR-VS-15 <code>nullAuthIdRowEmpty</code> The null authorization id never carries a count: every grant is keyed by a derived id.	✓	
SR-VS-16 <code>noPolicyEverMeansNoAuthorizations</code> While the pinned pool has never installed a policy, no authorization is recorded against it.	✓	
SR-VS-17 <code>assetForwardResolvesBack</code>	✓	

Every registered asset id resolves through the reverse map back to itself.		
SR-VS-18 <code>assetReverseResolvesBack</code> Every asset id filed under an asset and token id pair records exactly that pair.	✓	
SR-VS-19 <code>registeredAssetIdWellFormed</code> Registered asset ids carry a real origin chain and a counter value the issuer has already reached.	✓	
SR-VS-20 <code>unregisteredAssetRowEmpty</code> An unregistered asset id holds no leftover token id.	✓	
SR-VS-21 <code>nullAssetNeverRegistered</code> The zero asset address is never registered under any token id.	✓	
SR-VS-22 <code>sharePriceStampImpliesShareClass</code> A share price timestamp can only exist once the share class does.	✓	
SR-VS-23 <code>sharePriceValueImpliesShareClass</code> A share price value can only exist once the share class does.	✓	
SR-VS-24 <code>assetPriceStampImpliesShareClass</code> An asset price timestamp can only exist once the share class does.	✓	
SR-VS-25 <code>assetPriceValueImpliesShareClass</code> An asset price value can only exist once the share class does.	✓	
SR-VS-26 <code>assetPriceStampImpliesRegisteredAsset</code> An asset price timestamp can only exist for an asset the registry knows.	✓	
SR-VS-27 <code>assetPriceValueImpliesRegisteredAsset</code> An asset price value can only exist for an asset the registry knows.	✓	
SR-VS-28 <code>linkedVaultIsRegistered</code> A vault can only be linked after it was registered.	✓	
SR-VS-29 <code>unregisteredVaultPoolIdZero</code> An unregistered vault carries no pool reference.	✓	
SR-VS-30 <code>unregisteredVaultScIdZero</code> An unregistered vault carries no share class reference.	✓	
SR-VS-31 <code>unregisteredVaultAssetIdZero</code> An unregistered vault carries no asset id.	✓	
SR-VS-32 <code>unregisteredVaultTokenIdZero</code> An unregistered vault carries no token id.	✓	
SR-VS-33 <code>registeredVaultMatchesAssetRegistry</code> A registered vault's asset identity always agrees with the asset registry row it was checked against.	✓	
SR-VS-34 <code>registeredVaultSitsOnALiveShareClass</code> Every vault this scene registers sits on the pinned pool's live share class.	✓	
SR-VS-35 <code>nullVaultNeverRegistered</code> The zero address is never a registered vault.	✓	
SR-VS-36 <code>noAuthorizationsUnderZeroNonce</code>	✓	

The zero-nonce namespace never holds a grant.		
SR-VS-37 <code>noAuthorizationsAheadOfTenure</code> Namespaces of tenures the pool has not opened yet are empty.	✓	
SR-VS-38 <code>currentTenureAuthorizationsCarryTheInstalledPolicy</code> Inside the sitting tenure only the installed policy's own namespace holds grants.	✓	
SR-VS-39 <code>onlyADeployedContractOwnsATokenRow</code> An address carrying no code never owns a live token lookup row.	✓	

State Transitions

Properties	Status	Mutations
SR-ST-01 <code>stSrRegisteredVaultStartsUnlinked</code> A vault enters the registry unlinked, so registering one never by itself starts routing investor flows through it.	✓	
SR-ST-02 <code>stSrPolicySwapAdvancesTheInstallCounter</code> Changing a pool's policy always advances its install counter, retiring the outgoing grants.	✓	
SR-ST-03 <code>stSrGrantsLandOnlyOnTheInstalledPolicy</code> An authorization is only ever granted under the policy and counter in force at the call.	✓	
SR-ST-04 <code>stSrPriceStampsNeverRewind</code> Price stamps only ever move forward, so a stale cross-chain price can never overwrite a fresher one.	✓	
SR-ST-05 <code>stSrAuthorizationLedgerStepsByOneInTheStandingNamespace</code> The authorization ledger moves by exactly one, and only on the tenure in force.	✓	
SR-ST-06 <code>stSrRegisteredVaultKeepsItsIdentity</code> A registered vault's identity is never rewritten.	✓	
SR-ST-07 <code>stSrPoolEntryStampIsWrittenOnce</code> A pool's entry stamp, once written, never moves (the PoolAlreadyAdded latch).	✓	

Variable Transitions

Properties	Status	Mutations
SR-VT-01 <code>vtSrShareTokenNeverClears</code> A share class that has come online never goes dark: it always keeps a share token.	✓	
SR-VT-02 <code>vtSrPolicyTenureCounterStepsForwardByOne</code> The pool's policy tenure counter stands still or advances by one, never rewinding.	✓	
SR-VT-03 <code>vtSrAssetCounterStepsForwardByOne</code> The asset numbering counter advances one number at a time and never rewinds.	✓	
SR-VT-04 <code>vtSrIssuedAssetNumberKeepsItsBinding</code> An asset number, once issued, always names the same token and token id.	✓	
SR-VT-05 <code>vtSrRegisteredAssetKeepsItsNumber</code> A token that already carries an asset number keeps that number.	✓	

SR-VT-06 <code>vtSrPoolRolesMoveOneAccountAtATime</code> One call moves the pool standing of at most one account.	✓	
--	---	---

High Level

Properties	Status	Mutations
SR-HL-01 <code>hlSrConsumptionSpendsExactlyOneGrant</code> One grant funds one consumption and nothing more.	✓	
SR-HL-02 <code>hlSrPolicySwapOrphansOutstandingGrants</code> Swapping the pool's policy orphans every outstanding grant.	✓	
SR-HL-03 <code>hlSrAuthorizationLedgerCountsGrants</code> The authorization ledger is a counter, not a flag: grants accumulate and revoke one at a time.	✓	
SR-HL-04 <code>hlSrCreatedAssetPairResolvesToIssuedId</code> The asset number handed back at registration is the number the pair is filed under.	✓	
SR-HL-05 <code>hlSrCreatedAssetIdResolvesToItsAsset</code> The number handed back at registration resolves to the asset it was issued for.	✓	
SR-HL-06 <code>hlSrCreatedAssetIdResolvesToItsTokenId</code> The number handed back at registration resolves to the token id it was issued for.	✓	
SR-HL-07 <code>hlSrAuthorizationLedgerMovesOnlyThroughItsThreeWriters</code> Grants are minted and spent only through the three authorization entries.	✓	
SR-HL-08 <code>hlSrAssetQuoteIsStoredExactlyAsSent</code> An accepted asset quote leaves the class holding exactly what the hub sent, value and stamp.	✓	

Reverts

Properties	Status	Mutations
SR-RV-01 <code>rvSrAddPoolRefusesANonWardOrAPoolAlreadyOpen</code> A pool does not open for a stranger, nor twice on one the registry already carries.	✓	
SR-RV-02 <code>rvSrAddShareClassRefusesARepeatOrAnUnusableRegistration</code> A share class is created once, by a ward, against a deployed token with a real registrar.	✓	
SR-RV-03 <code>rvSrLinkTokenRefusesANonWardADeadPoolOrAnUnusableToken</code> A class is relinked only by a ward, and only to a deployed token with a real registrar.	✓	
SR-RV-04 <code>rvSrSetRequestManagerRefusesANonWardOrAPoolNeverOpened</code> A request manager is installed only by a ward, on a pool that was opened.	✓	
SR-RV-05 <code>rvSrUpdateManagerRefusesANonWardOrAPoolNeverOpened</code> The pool manager role moves only by a ward, on a pool that was opened.	✓	
SR-RV-06 <code>rvSrUpdateBridgerRefusesANonWardOrAPoolNeverOpened</code> The bridger role moves only by a ward, on a pool that was opened.	✓	
SR-RV-07 <code>rvSrSetPolicyRefusesANonWardOrAPoolNeverOpened</code> A policy is installed only by a ward, on a pool that was opened.	✓	

SR-RV-08 <code>rvSrAuthorizeRefusesANonWardOrAPoolWithNoPolicy</code> An authorization is recorded only by a ward, on a pool with a policy installed.	✓	
SR-RV-09 <code>rvSrUnauthorizeRefusesANonWardOrAGrantThatDoesNotStand</code> An authorization is revoked only by a ward, and only where the standing tenure holds one.	✓	
SR-RV-10 <code>rvSrConsumeAuthorizationRefusesAnyCallerButThePolicyOrAnEmptySlot</code> An authorization is consumed only by the pool's installed policy, so no grant is spent twice.	✓	
SR-RV-11 <code>rvSrRegisterVaultRefusesARepeatOrAMismatchedRegistration</code> A vault is recorded once, by a ward, on a live class and for the asset its id resolves to.	✓	
SR-RV-12 <code>rvSrLinkVaultRefusesAnUnknownOrAlreadyLinkedVault</code> A vault is linked only by a ward, and only where its recorded identity matches.	✓	
SR-RV-13 <code>rvSrUnlinkVaultRefusesAnUnroutedOrMismatchedVault</code> A vault is unlinked only by a ward, and only where its recorded identity matches.	✓	
SR-RV-14 <code>rvSrCreateAssetIdRefusesANonWardANullAssetOrAPairAlreadyNumbered</code> An asset is numbered once, by a ward, and never the null address.	✓	
SR-RV-15 <code>rvSrUpdatePricePoolPerShareRefusesANonWardADeadClassOrAnOlderStamp</code> A share price is written only by a ward, with a stamp at least as fresh as the one on file.	✓	
SR-RV-16 <code>rvSrUpdatePricePoolPerAssetRefusesANonWardAnUnknownAssetOrAnOlderStamp</code> An asset price is written only by a ward, with a stamp at least as fresh as the one on file.	✓	
SR-RV-17 <code>rvSrIdToAssetFailClosedRefusesAnUnregisteredId</code> The fail closed asset lookup never serves a null asset.	✓	
SR-RV-18 <code>rvSrIdToAssetLookupAlwaysAnswers</code> The plain asset lookup answers for every id, registered or not.	✓	
SR-RV-19 <code>rvSrAssetToIdFailClosedRefusesAnUnregisteredPair</code> The fail closed reverse lookup never serves a null id.	✓	
SR-RV-20 <code>rvSrAssetToIdLookupAlwaysAnswers</code> The plain reverse lookup answers for every asset and token id pair.	✓	
SR-RV-21 <code>rvSrPricePoolPerShareCheckedRefusesANeverComputedPrice</code> A checked share read refuses a never computed price rather than serving a blank slot.	✓	
SR-RV-22 <code>rvSrPricePoolPerAssetCheckedRefusesANeverComputedPrice</code> A checked asset read refuses a never computed price rather than serving a blank slot.	✓	
SR-RV-23 <code>rvSrRelyRefusesANonWard</code> Registry authority is handed on only by an account that already holds it.	✓	
SR-RV-24 <code>rvSrDenyRefusesANonWard</code> Registry authority is withdrawn only by an account that holds it.	✓	

Reachability

Properties	Status	Mutations
SR-RC-01 <code>rcSrAddPoolIsReachable</code> Witnesses a pool being opened and taking the current block time as its entry stamp.	✓	

SR-RC-02 <code>rcSrAddPoolAtTheClockFloorIsReachable</code> Witnesses a pool opened at the earliest block time the scene admits.	✓	
SR-RC-03 <code>rcSrPoolReadsActiveRightAfterAddIsReachable</code> Witnesses the pool reading back as active through its own getter in the transaction that opened it.	✓	
SR-RC-04 <code>rcSrAddShareClassIsReachable</code> Witnesses a share class coming online with its token and the reverse lookup written.	✓	
SR-RC-05 <code>rcSrTokenSwapRetiringTheOutgoingTokenIsReachable</code> Witnesses a class moved onto a second token, the outgoing one retired in the same call.	✓	
SR-RC-06 <code>rcSrRegistrarSwapKeepingTheIncumbentTokenIsReachable</code> Witnesses a registrar swapped while the class keeps the token it already serves.	✓	
SR-RC-07 <code>rcSrSetRequestManagerIsReachable</code> Witnesses a request manager being installed on a live pool, so investor requests get an address to route to.	✓	
SR-RC-08 <code>rcSrClearingTheRequestManagerIsReachable</code> Witnesses the request manager row being cleared, so a pool can switch requests off again.	✓	
SR-RC-09 <code>rcSrGrantingOneManagerSeatIsReachable</code> Witnesses the manager seat being granted to one account while a second account is left without it.	✓	
SR-RC-10 <code>rcSrRevokingOneOfTwoManagerSeatsIsReachable</code> Witnesses one of two manager seats being revoked while the other holder keeps its own.	✓	
SR-RC-11 <code>rcSrGrantingTheBridgerSeatAloneIsReachable</code> Witnesses the bridger seat granted to an account that holds no manager seat.	✓	
SR-RC-12 <code>rcSrFirstPolicyInstallIsReachable</code> Witnesses the first policy being installed on a pool, opening the pool's first authorization tenure.	✓	
SR-RC-13 <code>rcSrPolicyReplacementIsReachable</code> Witnesses one policy being replaced by another, with the tenure counter advancing on the swap.	✓	
SR-RC-14 <code>rcSrClearingThePolicyIsReachable</code> Witnesses the policy being cleared to the zero address, which still advances the tenure counter.	✓	
SR-RC-15 <code>rcSrInstallThenClearPolicyIsReachable</code> Witnesses a policy installed and cleared inside one transaction, the counter advancing twice.	✓	
SR-RC-16 <code>rcSrAuthorizeIsReachable</code> Witnesses a matured Hub authorization recorded against the sitting policy.	✓	
SR-RC-17 <code>rcSrTwoOutstandingGrantsAreReachable</code> Witnesses the same call authorized twice and the ledger counting both.	✓	
SR-RC-18 <code>rcSrUnauthorizeEmptyingTheRowIsReachable</code> Witnesses a grant being revoked before anyone spends it, taking the ledger row back to empty.	✓	

SR-RC-19 <code>rcSrPolicyConsumingAGrantIsReachable</code>	Witnesses the installed policy spending a grant made for it, taking the ledger row back to empty.		
SR-RC-20 <code>rcSrConsumingOneOfTwoGrantsIsReachable</code>	Witnesses one of two outstanding grants being spent, leaving the second standing for a later call.		
SR-RC-21 <code>rcSrPolicySwapPuttingGrantsBeyondRevocationIsReachable</code>	Witnesses a revocation after a policy swap landing on the standing tenure's row.		
SR-RC-22 <code>rcSrFirstAssetNumberIsReachable</code>	Witnesses the very first asset number being minted, with both lookup legs resolving to the pair it was issued for.		
SR-RC-23 <code>rcSrSecondAssetPairTakingItsOwnNumberIsReachable</code>	Witnesses a second asset pair taking a number of its own.		
SR-RC-24 <code>rcSrNumberingAMultiTokenAssetPairIsReachable</code>	Witnesses one token of a multi token asset contract being numbered, so an ERC6909 holding can take an id of its own.		
SR-RC-25 <code>rcSrAssetNumberOnTheSmallestOriginChainIsReachable</code>	Witnesses a number issued on the smallest legal origin chain namespace.		
SR-RC-26 <code>rcSrCheckedAssetLookupsAfterNumberingAreReachable</code>	Witnesses both checked lookups answering the moment a pair is numbered.		
SR-RC-27 <code>rcSrRegisterVaultIsReachable</code>	Witnesses a vault registered against a live class and a known ERC20 asset, out of service.		
SR-RC-28 <code>rcSrRegisterMultiTokenVaultIsReachable</code>	Witnesses a vault registered against one token of a multi token asset contract.		
SR-RC-29 <code>rcSrLinkVaultIsReachable</code>	Witnesses a registered vault being put into service, so investor flows start routing through it.		
SR-RC-30 <code>rcSrUnlinkVaultIsReachable</code>	Witnesses a live vault being taken out of service while it stays registered, so its identity is kept for a later relink.		
SR-RC-31 <code>rcSrWithdrawingAndRestoringVaultServiceIsReachable</code>	Witnesses service withdrawn from a vault and restored again in one transaction.		
SR-RC-32 <code>rcSrLinkedVaultReadingBackThroughItsViewsIsReachable</code>	Witnesses a freshly linked vault reading back as linked and registered through its own getters.		
SR-RC-33 <code>rcSrFirstSharePriceIsReachable</code>	Witnesses a class taking its first share price, stored together with the stamp it was computed at.		
SR-RC-34 <code>rcSrSharePriceMovingToALaterStampIsReachable</code>	Witnesses a stored share quote moved forward to a later stamp.		
SR-RC-35 <code>rcSrSharePriceCorrectionAtTheSameStampIsReachable</code>	Witnesses a share quote corrected at the stamp it already carries.		

SR-RC-36 <code>rcSrFirstAssetPriceIsReachable</code> Witnesses the first quote for one registered asset of a live class being stored with its stamp.	✓	
SR-RC-37 <code>rcSrCheckedSharePriceReadAfterAWriteIsReachable</code> Witnesses the checked share read answering with the quote just written.	✓	
SR-RC-38 <code>rcSrCheckedAssetPriceReadAfterAWriteIsReachable</code> Witnesses the checked asset read answering with the quote just written.	✓	
SR-RC-39 <code>rcSrFreshWardOpeningAPoolIsReachable</code> Witnesses an account gaining registry authority and using it at once.	✓	
SR-RC-40 <code>rcSrDenyLeavingTheCallersOwnSeatIsReachable</code> Witnesses one holder's registry authority withdrawn while the caller keeps its own.	✓	
SR-RC-41 <code>rcSrWholePoolBringUpIsReachable</code> Witnesses a pool brought from unopened all the way up to a vault in service.	✓	
SR-RC-42 <code>rcSrRevokingOneOfTwoBridgerSeatsIsReachable</code> Witnesses one of two bridger seats withdrawn while the other holder keeps its own.	✓	

Access Control

Properties	Status	Mutations
SR-AC-01 <code>acSrWardSetMovesOnlyForAWard</code> Control of the registry is handed out or taken away only by an address that already holds it.	✓	
SR-AC-02 <code>acSrPoolEntryStampMovesOnlyForAWard</code> Only a ward brings a pool onto this spoke.	✓	
SR-AC-03 <code>acSrShareClassBindingMovesOnlyForAWard</code> Only a ward points a share class at its token and registrar.	✓	
SR-AC-04 <code>acSrTokenLookupRowMovesOnlyForAWard</code> Only a ward writes or clears a token's lookup row.	✓	
SR-AC-05 <code>acSrRequestManagerMovesOnlyForAWard</code> Only a ward installs a pool's request manager.	✓	
SR-AC-06 <code>acSrManagerRightsMoveOnlyForAWard</code> Only a ward grants or withdraws pool manager rights.	✓	
SR-AC-07 <code>acSrBridgerRightsMoveOnlyForAWard</code> Only a ward grants or withdraws bridger rights.	✓	
SR-AC-08 <code>acSrPolicyInstallMovesOnlyForAWard</code> Only a ward seats a pool's policy or advances its install counter.	✓	
SR-AC-09 <code>acSrAuthorizationGrantedOnlyByAWard</code> An authorization is granted only by a ward.	✓	
SR-AC-10 <code>acSrAuthorizationSpentOrRevokedOnlyByAWardOrTheInstalledPolicy</code> An outstanding authorization is spent or revoked only by a ward or the installed policy.	✓	
SR-AC-11 <code>acSrAssetRegistryMovesOnlyForAWard</code> Only a ward mints an asset number or moves the asset bijection.	✓	

SR-AC-12 <code>acSrVaultRegistrationMovesOnlyForAWard</code> Only a ward enters a vault in the registry.	✓	
SR-AC-13 <code>acSrVaultLinkFlagMovesOnlyForAWard</code> Only a ward links or unlinks a vault.	✓	
SR-AC-14 <code>acSrSharePriceMovesOnlyForAWard</code> Only a ward moves a share class price.	✓	
SR-AC-15 <code>acSrAssetPriceMovesOnlyForAWard</code> Only a ward moves an asset price.	✓	

SnapshotQueue

- Every configuration
 - Queue rows bounded to 3 symbolic rows on 3 distinct assets
- Single pool
 - One symbolic pool
 - One symbolic share class
- Multi pool
 - Two symbolic pools
 - One share class per pool

Valid State

Properties	Status	Mutations
SQ-VS-01 <code>zeroDeltaIsNonPositive</code> A fully netted share queue reads as a plain zero, so a flush never reports a positive zero share update to the hub.	✓	
SQ-VS-02 <code>queuedAssetCounterMatchesOutstandingRows</code> The queued asset counter counts exactly the asset rows still waiting to flush.	✓	

State Transitions

Properties	Status	Mutations
SQ-ST-01 <code>stSqAssetFlushEmptiesTheRowAndSpendsOneOrdinal</code> Handing an asset queue to the hub empties it, strikes it off the count and advances the nonce.	✓	
SQ-ST-02 <code>stSqShareFlushHandsOverTheWholeNet</code> Spending a nonce on the share queue always hands over the whole outstanding net, never part of it.	✓	
SQ-ST-03 <code>stSqOpeningAnAssetRowSpendsNoOrdinalAndMovesNoShareNet</code> Opening an asset queue neither advances the nonce nor touches the share net.	✓	
SQ-ST-04 <code>stSqAssetFlowTakesOneSideOnly</code> A queued flow is recorded in one direction only, so a deposit is never also reported as a withdrawal of the same asset.	✓	

SQ-ST-05 <code>stSqNonceAdvancesByAtMostOne</code> The flush nonce advances by at most one per call, so the hub is never made to skip a report number.	✓	
SQ-ST-06 <code>stSqShareRowIsPerPool</code> A call that moves one pool's share row leaves the other pool's share row alone.	✓	
SQ-ST-07 <code>stSqAssetRowIsPerPool</code> A call that moves one pool's asset row leaves every asset row of the other pool alone.	✓	
SQ-ST-08 <code>stSqQueueSidesDoNotCrossPools</code> The share side of one pool and the asset side of another never move together.	✓	

Variable Transitions

Properties	Status	Mutations
SQ-VT-01 <code>vtSqAssetRowsMoveOneAtATime</code> A call touches the queue of a single asset, never another asset's waiting amounts.	✓	

High Level

Properties	Status	Mutations
SQ-HL-01 <code>hlSqShareFlushReportsATruthfulSnapshot</code> The share report tells the hub the class is in sync exactly when every asset queue is handed over.	✓	
SQ-HL-02 <code>hlSqAssetFlushReportsATruthfulSnapshot</code> The asset report claims a snapshot only when nothing else is still waiting.	✓	
SQ-HL-03 <code>hlSqAssetPayloadCarriesTheNetMagnitude</code> The hub is handed the net of everything queued for the asset, not the gross.	✓	
SQ-HL-04 <code>hlSqAssetPayloadPointsTowardTheLargerSide</code> The flushed asset payload points in the direction of the larger gross side.	✓	
SQ-HL-05 <code>hlSqSharePayloadCarriesTheNetMagnitude</code> A share flush hands the hub the magnitude of the signed sum, never the gross traffic.	✓	
SQ-HL-06 <code>hlSqSharePayloadCarriesTheNetSign</code> The flushed share payload reads as an issuance exactly when the queued net is positive.	✓	
SQ-HL-07 <code>hlSqSecondAssetFlushReportsNothing</code> Flushing consumes the asset queue, so no asset flow is ever submitted to the hub twice.	✓	
SQ-HL-08 <code>hlSqSecondShareFlushReportsNothing</code> Flushing consumes the share queue, so no share delta is ever applied by the hub twice.	✓	
SQ-HL-09 <code>hlSqConsecutiveAssetFlushesSpendConsecutiveOrdinals</code> Back-to-back asset flushes file under consecutive ordinals, with no gap and no repeat.	✓	
SQ-HL-10 <code>hlSqConsecutiveShareFlushesSpendConsecutiveOrdinals</code> Back-to-back share flushes file under consecutive ordinals, with no gap and no repeat.	✓	
SQ-HL-11 <code>hlSqZeroAmountQueueingCannotStallTheSnapshot</code> A zero amount call queues nothing and can never wedge the class out of sync.	✓	

SQ-HL-12 <code>hlSqOnlyAFlushSpendsAnOrdinal</code> Only the two flush entries spend a flush ordinal.	✓	
--	---	---

Reverts

Properties	Status	Mutations
SQ-RV-01 <code>rvSqQueueAssetsRefusesANonWard</code> An asset flow is queued by the queue's own authority alone.	✓	
SQ-RV-02 <code>rvSqQueueSharesRefusesANonWard</code> A share delta is queued by the queue's own authority alone.	✓	
SQ-RV-03 <code>rvSqFlushAssetsRefusesANonWard</code> An asset queue is handed to the hub by the queue's own authority alone.	✓	
SQ-RV-04 <code>rvSqFlushSharesRefusesANonWard</code> The share queue is handed to the hub by the queue's own authority alone.	✓	
SQ-RV-05 <code>rvSqRelyRefusesANonWard</code> A seat over the queue is granted by a sitting ward alone.	✓	
SQ-RV-06 <code>rvSqDenyRefusesANonWard</code> A seat over the queue is revoked by a sitting ward alone.	✓	
SQ-RV-07 <code>rvSqFlushAssetsAlwaysAdmitsAWardWhileOrdinalsRemain</code> A pool's queued asset flows always reach the hub while the ordinal stream has numbers left.	✓	
SQ-RV-08 <code>rvSqFlushSharesAlwaysAdmitsAWardWhileOrdinalsRemain</code> A pool's queued share net always reaches the hub while the ordinal stream has numbers left.	✓	

Reachability

Properties	Status	Mutations
SQ-RC-01 <code>rcSqQueueAssetsCanOpenADepositRow</code> Witnesses a ward opening a fresh deposit queue for an asset.	✓	
SQ-RC-02 <code>rcSqQueueAssetsCanOpenAWithdrawalRow</code> Witnesses a ward opening a fresh withdrawal queue for an asset.	✓	
SQ-RC-03 <code>rcSqQueueAssetsCanAccumulateASecondDeposit</code> Witnesses a second deposit accumulating gross without counting the asset outstanding twice.	✓	
SQ-RC-04 <code>rcSqQueueAssetsCanHoldBothSidesOfOneRow</code> Witnesses one asset row carrying a deposit and a withdrawal at the same time.	✓	
SQ-RC-05 <code>rcSqQueueAssetsCanLeaveThreeAssetsOutstandingAtOnce</code> Witnesses three assets of one class outstanding at the same time.	✓	
SQ-RC-06 <code>rcSqQueueAssetsCanAbsorbTheFullAccumulatorWidth</code> Witnesses a single queued flow as large as the accumulator itself.	✓	

SQ-RC-07 <code>rcSqQueueAssetsCanMakeAClassOutstandingWithOneUnit</code> Witnesses a single unit of an asset making a class outstanding.	✓	
SQ-RC-08 <code>rcSqZeroAmountQueueLeavesTheSnapshotAvailable</code> Witnesses a zero amount call being a complete no-op.	✓	
SQ-RC-09 <code>rcSqQueueSharesCanOpenAnIssuanceNet</code> Witnesses a ward opening a net issuance on a drained share queue.	✓	
SQ-RC-10 <code>rcSqQueueSharesCanOpenAREvocationNet</code> Witnesses a ward opening a net revocation on a drained share queue.	✓	
SQ-RC-11 <code>rcSqQueueSharesCanGrowAnIssuanceNet</code> Witnesses issuances in one direction piling up into a single queued net.	✓	
SQ-RC-12 <code>rcSqQueueSharesCanShrinkAnIssuanceNet</code> Witnesses a smaller revocation shrinking a waiting issuance and leaving it an issuance.	✓	
SQ-RC-13 <code>rcSqQueueSharesCanCancelAnIssuanceNetExactly</code> Witnesses a matching revocation draining the queue to a plain zero.	✓	
SQ-RC-14 <code>rcSqQueueSharesCanAbsorbTheFullNetWidth</code> Witnesses a single queued share delta as large as the net cell itself.	✓	
SQ-RC-15 <code>rcSqZeroShareQueueLeavesTheNetPayloadIntact</code> Witnesses a zero share call disturbing nothing.	✓	
SQ-RC-16 <code>rcSqALargerRevocationCanFlipTheQueuedNet</code> Witnesses one large revocation flipping the queued net to the other side.	✓	
SQ-RC-17 <code>rcSqAssetFlushCanHandOverAndReportASnapshot</code> Witnesses the last outstanding asset row handed over with the snapshot flag set.	✓	
SQ-RC-18 <code>rcSqAssetFlushWithASiblingRowOutstandingCanReportNoSnapshot</code> Witnesses a partial hand over reporting the class as not in sync.	✓	
SQ-RC-19 <code>rcSqAssetFlushWithSharesOutstandingCanReportNoSnapshot</code> Witnesses the asset side refusing to claim sync while the share net still waits.	✓	
SQ-RC-20 <code>rcSqAssetFlushCanNetTowardTheDepositSide</code> Witnesses a two-sided row handed over as a single deposit surplus.	✓	
SQ-RC-21 <code>rcSqAssetFlushCanNetTowardTheWithdrawalSide</code> Witnesses a two-sided row handed over as a single withdrawal surplus.	✓	
SQ-RC-22 <code>rcSqEmptyAssetFlushStillSpendsAnOrdinal</code> Witnesses a flush of an empty asset row still spending a flush ordinal.	✓	
SQ-RC-23 <code>rcSqSecondAssetFlushCanReportNothingUnderTheNextOrdinal</code> Witnesses a second flush of the same row reporting nothing under the next ordinal.	✓	
SQ-RC-24 <code>rcSqAssetFlushCanCarryTheFullAccumulatorWidth</code> Witnesses the whole width of a queued deposit leaving in a single payload.	✓	
SQ-RC-25 <code>rcSqOffsettingRowCanFlushZeroYetClearTheCount</code> Witnesses a row whose two sides cancel handed over as a zero payload and struck off.	✓	
SQ-RC-26 <code>rcSqShareFlushCanHandOverAnIssuanceAndReportASnapshot</code> Witnesses a waiting issuance handed to the hub with the snapshot flag set.	✓	

SQ-RC-27 <code>rcSqShareFlushCanHandOverARevocationAndReportASnapshot</code> Witnesses a waiting revocation handed over as a snapshot, leaving the share queue drained.	✓	
SQ-RC-28 <code>rcSqShareFlushWithAnAssetRowOutstandingCanReportNoSnapshot</code> Witnesses the share side refusing to claim sync while an asset row still waits.	✓	
SQ-RC-29 <code>rcSqEmptyShareFlushStillSpendsAnOrdinal</code> Witnesses a flush of a drained share queue still spending a flush ordinal.	✓	
SQ-RC-30 <code>rcSqShareFlushCanCarryTheFullNetWidth</code> Witnesses the whole width of a queued share net leaving in a single payload.	✓	
SQ-RC-31 <code>rcSqAnOutOfSyncClassCanBeBroughtBackIntoSync</code> Witnesses a class with both queues loaded brought back into sync in one sequence.	✓	
SQ-RC-32 <code>rcSqAnAssetRowCanBeReusedAfterAFlush</code> Witnesses an asset row being reusable after being handed over.	✓	
SQ-RC-33 <code>rcSqRelyCanGrantTheQueueSeat</code> Witnesses that a ward can hand a seat over the queue to an account that held none.	✓	
SQ-RC-34 <code>rcSqDenyCanRemoveAnotherAccountsQueueSeat</code> Witnesses that a ward can take the seat back from another account, so a granted authority over the queue is revocable.	✓	
SQ-RC-35 <code>rcSqAFreshlyReliedAccountCanMoveTheQueue</code> Witnesses a freshly granted seat being usable at once.	✓	
SQ-RC-36 <code>rcSqAWardCanDenyItself</code> Witnesses a ward handing in its own seat.	✓	
SQ-RC-37 <code>rcSqQueuedSharesViewCanReadBackAFreshNet</code> Witnesses the public share queue view reporting the net a queue call just wrote.	✓	
SQ-RC-38 <code>rcSqQueuedAssetsViewCanReadBackAFreshFlow</code> Witnesses the public asset queue view reporting the gross side a queue call just wrote.	✓	
SQ-RC-39 <code>rcSqSecondShareFlushCanReportNothingUnderTheNextOrdinal</code> Witnesses a second share flush reporting nothing under the next ordinal.	✓	

Access Control

Properties	Status	Mutations
SQ-AC-01 <code>acSqOnlyAWardMovesTheShareNet</code> Only the queue's own authority moves the netted share position reported to the hub.	✓	
SQ-AC-02 <code>acSqOnlyAWardMovesAnAssetRow</code> Only the queue's own authority moves an asset's queued deposit and withdrawal totals.	✓	
SQ-AC-03 <code>acSqOnlyAWardMovesTheOutstandingAssetCount</code> Only the queue's own authority moves the count of asset queues still waiting.	✓	
SQ-AC-04 <code>acSqOnlyAWardSpendsAFlushOrdinal</code> A flush ordinal is spent only by a ward, so an outsider cannot put the hub's ordering of queue updates out of step.	✓	

[SQ-AC-05](#) `acSqOnlyAWardMovesTheWardBit`

Only a sitting ward grants or revokes authority over the queue.



PoolEscrow

- Every configuration
 - One symbolic share class
 - Two symbolic reservers
 - Two symbolic reasons
- Multi row
 - Two asset rows; the other confs pin one

Valid State

Properties	Status	Mutations
PE-VS-01 <code>reservedMatchesBucketSum</code> A row's stored earmark is exactly the sum of the holds booked against it.		
PE-VS-02 <code>everyAssetRowMatchesItsBucketSum</code> Every asset row's stored earmark is exactly the sum of its own per-reserver holds.		

State Transitions

Properties	Status	Mutations
PE-ST-01 <code>stPePayoutBoundedByFreeBalance</code> A single call pays out at most what was free of earmarks when it started, so a fully earmarked holding pays out nothing.		
PE-ST-02 <code>stPeOneCallTouchesOneBucket</code> One entry touches one reservation bucket, so a hold under one reason never moves what another reserver holds.		
PE-ST-03 <code>stPeDepositNeverLowersTheCustodyTotal</code> Booking custody in never lowers what the row says the escrow holds.		

High Level

Properties	Status	Mutations
PE-HL-01 <code>hlPeAvailableBalanceIsTheUnreservedCustody</code> The payout view reports custody minus the aggregate hold, floored at zero.		
PE-HL-02 <code>hlPeSweepingTheFreeBalanceLeavesTheEarmarks</code> Paying out everything the view calls free drains custody to exactly the earmarked amount.		
PE-HL-03 <code>hlPeReserveThenReleaseRestoresTheBucket</code> Reserving and releasing the same amount under one key hands the bucket back exactly.		
PE-HL-04 <code>hlPeTakeInThenPayOutRestoresTheCustody</code> Booking custody in and paying the same amount out returns the total to where it started.		
PE-HL-05 <code>hlPeCustodyRoundTripTouchesNoHold</code>		

Custody traffic moves no earmark: a booking and a payout leave the aggregate hold alone.	✓	
PE-HL-06 <code>hlPeAnyHoldCapsEveryPayout</code> A hold under any key immediately caps what anyone can pay out, down to nothing.	✓	

Reverts

Properties	Status	Mutations
PE-RV-01 <code>rvPeDepositRefusesANonWard</code> Custody is booked in only by the escrow's own authority.	✓	
PE-RV-02 <code>rvPeWithdrawRefusesANonWardOrAPayoutBeyondTheFreeBalance</code> A payout is taken by the escrow's own authority alone, and only out of the balance left free.	✓	
PE-RV-03 <code>rvPePassingWithdrawAtTheAdvertisedFreeBalanceIsReachable</code> Witnesses a ward attaching no value paying out exactly the nonzero amount the free balance view advertises.	✓	
PE-RV-04 <code>rvPeReserveRefusesANonWard</code> A hold is placed only by the escrow's own authority.	✓	
PE-RV-05 <code>rvPeUnreserveRefusesANonWardOrAShortBucket</code> A release is taken only up to what the named bucket holds, so one key never drains another.	✓	
PE-RV-06 <code>rvPeRelyRefusesANonWard</code> The ward roll is extended by a sitting ward alone.	✓	
PE-RV-07 <code>rvPeDenyRefusesANonWard</code> The ward roll is trimmed by a sitting ward alone.	✓	
PE-RV-08 <code>rvPeAvailableBalanceOfAlwaysAnswers</code> The free balance view answers in every state the books can reach.	✓	
PE-RV-09 <code>rvPeAuthTransferToRefusesANonWard</code> Assets leave custody on the escrow's own authority alone, whatever token standard they follow.	✓	
PE-RV-10 <code>rvPeRecoverTokensRefusesANonWard</code> The recovery path for ether and plain tokens answers to the escrow's own authority alone.	✓	
PE-RV-11 <code>rvPeRecoverTokensByTokenIdRefusesANonWard</code> The recovery path naming a token id answers to the escrow's own authority alone.	✓	
PE-RV-12 <code>rvPeUnreserveAlwaysAdmitsAKeyReleasingItsWholeBucket</code> A key always gets back the whole of what it placed on hold, so nothing is earmarked forever.	✓	
PE-RV-13 <code>rvPeDenyAlwaysAdmitsAWardStandingItselfDown</code> A ward always stands its own seat down, the step every escrow takes when it is handed over.	✓	

Reachability

Properties	Status	Mutations

PE-RC-01 <code>rcPeDepositBooksCustodyOnAnErc20Row</code> Witnesses a booking going through on an ERC20 row and widening the row's tracked total.	✓	
PE-RC-02 <code>rcPeDepositBooksTheFullBookWidth</code> Witnesses one booking crediting an empty row with the entire width of the custody book.	✓	
PE-RC-03 <code>rcPeDepositBooksAnErc6909Row</code> Witnesses a booking going through on an ERC6909 row, the token id carried by the row key.	✓	
PE-RC-04 <code>rcPeDepositReopensAFullyEncumberedRow</code> Witnesses a booking onto a row earmarked to its last unit, making the advertised free balance positive again.	✓	
PE-RC-05 <code>rcPeWithdrawEmptiesAnUnencumberedRow</code> Witnesses a payout taking a row that carries no earmark down to an empty book in one call.	✓	
PE-RC-06 <code>rcPeWithdrawSweepsTheAdvertisedFreeBalance</code> Witnesses every free unit leaving in one payout while an earmark stands.	✓	
PE-RC-07 <code>rcPeWithdrawLeavesTheRowStillPaying</code> Witnesses a partial payout from an earmarked row that leaves the row still advertising a positive free balance.	✓	
PE-RC-08 <code>rcPeWithdrawBooksAPayoutToANullReceiver</code> Witnesses a payout booked to the null address, the entry being accounting only.	✓	
PE-RC-09 <code>rcPeReserveOpensABucket</code> Witnesses a hold opening a fresh bucket and raising the row's aggregate earmark with it.	✓	
PE-RC-10 <code>rcPeReserveBooksABucketForAnUnauthorizedKey</code> Witnesses a ward opening a hold under a key belonging to neither the caller nor any ward.	✓	
PE-RC-11 <code>rcPeUnreserveEmptiesAForeignBucket</code> Witnesses a ward emptying a hold booked under another party's key.	✓	
PE-RC-12 <code>rcPeTwoReserversHoldOneRowAtOnce</code> Witnesses two parties holding the same row at once under one reason, the aggregate earmark carrying both buckets.	✓	
PE-RC-13 <code>rcPeOneReserverHoldsTwoReasons</code> Witnesses one party holding two buckets on the same row, one per reason code.	✓	
PE-RC-14 <code>rcPeReserveBeyondCustodyFreezesTheRow</code> Witnesses a row that paid out past its own custody reporting nothing free rather than wrapping.	✓	
PE-RC-15 <code>rcPeReserveEncumbersTheWholeFreeBalance</code> Witnesses a hold of exactly the advertised free balance, which earmarks the row to its last unit.	✓	
PE-RC-16 <code>rcPeUnreserveClearsTheRowsEarmark</code> Witnesses a party's whole hold being released, leaving the row carrying no earmark at all.	✓	
PE-RC-17 <code>rcPeUnreserveLeavesTheBucketStillHolding</code> Witnesses a partial release, the bucket and the row's aggregate both falling while the key keeps a standing hold.	✓	

PE-RC-18 <code>rcPeReleasingOneBucketLeavesTheOtherHeld</code> Witnesses one party's hold being released in full while another party's hold on the same row stands untouched.	✓	
PE-RC-19 <code>rcPeReleaseLetsAnOverEncumberedRowPayOutAgain</code> Witnesses a row wedged by holds beyond its custody being released back into a state that pays out.	✓	
PE-RC-20 <code>rcPeHoldPayoutReleaseWidensTheFreeBalance</code> Witnesses a hold narrowing a payout and then handing the same amount back to the free balance once it is released.	✓	
PE-RC-21 <code>rcPeRelyGrantsTheWardSeat</code> Witnesses a ward handing the seat to an account that held none.	✓	
PE-RC-22 <code>rcPeDenyRevokesAnotherWard</code> Witnesses a ward stripping another account's seat while keeping its own.	✓	
PE-RC-23 <code>rcPeFreshlyReliedAccountBooksADeposit</code> Witnesses an account relied in one call moving the books in the next call of the same block.	✓	
PE-RC-24 <code>rcPeBucketReadsBackTheHold</code> Witnesses the public bucket book reporting back exactly the hold that was just placed.	✓	

Access Control

Properties	Status	Mutations
PE-AC-01 <code>acPeOnlyWardMovesTheCustodyTotal</code> Only the escrow's own authority moves the accounted custody total, outside the ward-gated movers.	✓	
PE-AC-02 <code>acPeOnlyWardMovesTheAggregateHold</code> Only the escrow's own authority moves the aggregate hold, outside the ward-gated movers.	✓	
PE-AC-03 <code>acPeOnlyWardMovesAReservationBucket</code> Only the escrow's own authority moves a reservation bucket, outside the ward-gated movers.	✓	
PE-AC-04 <code>acPeOnlyWardMovesAWardSeat</code> Only an account already on the roll can extend or trim it, outside the ward-gated movers.	✓	

MultiAdapter

- One symbolic pool
- One symbolic share class
- Adapters bounded to 8 symbolic adapters
- One symbolic remote chain
- Loops running up to 8 iterations

Valid State

Properties	Status	Mutations
------------	--------	-----------

MA-VS-01 <code>activeSetSessionCurrentOrCleared</code> The outbound route carries the current session id or is cleared.	✓	
MA-VS-02 <code>clearedActiveSetEmpty</code> A cleared route holds no adapter roster.	✓	
MA-VS-03 <code>activeSetMirrorsSessionList</code> The roster a send fans out to is a verbatim copy of the session's own roster.	✓	
MA-VS-04 <code>sessionsAboveCounterEmpty</code> Sessions past the current session id hold no roster, so no adapter set can be staged ahead of the live one.	✓	
MA-VS-05 <code>blockedAboveCounterEmpty</code> Sessions past the current session id hold no stash; only a session that once existed can be blocked.	✓	
MA-VS-06 <code>sessionZeroNeverConfigured</code> Session zero is the unconfigured sentinel: it never holds a roster and is never blocked.	✓	
MA-VS-07 <code>registeredAdapterWellFormed</code> Every registered adapter row is well formed, threshold within quorum and quorum within the cap.	✓	
MA-VS-08 <code>adapterQuorumEqualsListLength</code> Every registered adapter records the session's roster size as its quorum.	✓	
MA-VS-09 <code>sessionThresholdUniform</code> One threshold per session, so the quorum gate never depends on which adapter's row happens to be read.	✓	
MA-VS-10 <code>listedAdapterHoldsItsIndex</code> Each listed adapter holds its own one-based roster position, so every adapter votes into its own lane.	✓	
MA-VS-11 <code>adapterIdPointsAtItsOwnSlot</code> A registered adapter's id points back at its own roster slot, so the ids form a permutation of the roster positions.	✓	
MA-VS-12 <code>blockedExcludesLive</code> A session is live or blocked, never both, so an unblock can never double-deliver a roster.	✓	
MA-VS-13 <code>blockedThresholdWithinList</code> A stash keeps a workable threshold within its roster size.	✓	
MA-VS-14 <code>blockedAtCounterWasActive</code> A stash held at the current session id is always flagged as previously active.	✓	
MA-VS-15 <code>blockedCounterClearsActiveSet</code> Blocking the current session leaves no outbound route, so sends reject while it sits in the stash.	✓	
MA-VS-16 <code>blockedListDistinct</code> Stashed rosters are duplicate free, so an unblock can always reinstall without wedging the session in blocked state.	✓	
MA-VS-17 <code>stampedSessionNeverStashed</code> A stamped route means the session it names is not sitting in the stash.	✓	

State Transitions

Properties	Status	Mutations
MA-ST-01 <code>stMaSessionAdvanceRestampsTheRoute</code> Rotating a pool's adapter set changes the session id and re-stamps the route in the same call.	✓	
MA-ST-02 <code>stMaBlockStashesTheLiveRosterWhole</code> Taking a session out of service preserves everything needed to put it back.	✓	
MA-ST-03 <code>stMaBlockingASupersededSessionKeepsTheRoute</code> Retiring a superseded session leaves the route carrying traffic untouched.	✓	
MA-ST-04 <code>stMaUnblockRestoresExactlyWhatWasStashed</code> Putting a session back reinstates exactly the roster and the threshold that were taken out.	✓	
MA-ST-05 <code>stMaUnblockReopensTheRouteOnlyForTheSessionInService</code> A restored session takes over outbound traffic only if it was the one in service.	✓	

Variable Transitions

Properties	Status	Mutations
MA-VT-01 <code>vtMaSessionIdNeverRegresses</code> The session counter of a route only ever moves forward.	✓	
MA-VT-02 <code>vtMaSessionIdAdvancesByOneAtMost</code> No call carries the session counter past the id quoted for the next configuration.	✓	
MA-VT-03 <code>vtMaRosterMoveSparesOtherSessions</code> Installing or retiring one session's adapter set leaves every other session's roster alone.	✓	

High Level

Properties	Status	Mutations
MA-HL-01 <code>hlMaExecuteSpendsATallyOnlyAtTheThreshold</code> A tally is spent only when its confirmations meet the session's threshold.	✓	
MA-HL-02 <code>hlMaExecuteSparesTheSecondLaneOutsideTheQuorum</code> An execution leaves the second vote lane where it stood when the quorum stops short of it.	✓	
MA-HL-03 <code>hlMaExecuteSparesTheThirdLaneOutsideTheQuorum</code> An execution leaves the third vote lane where it stood when the quorum stops short of it.	✓	
MA-HL-04 <code>hlMaExecuteForwardsExactlyOneMessage</code> An approved execution hands the gateway exactly one message.	✓	
MA-HL-05 <code>hlMaExecuteForwardsOnTheArrivalChain</code> The gateway hears an executed message on the chain lane it arrived from.	✓	
MA-HL-06 <code>hlMaHandleForwardsAtMostOnce</code> One adapter delivery triggers at most one forwarding.	✓	

MA-HL-07 <code>hlMaVoteNeverReachesTheGateway</code> The proof path records and never executes.	✓	
MA-HL-08 <code>hlMaSessionIdMovesOnlyByRotation</code> The session id scoping every roster, stash and vote row moves only through a rotation.	✓	
MA-HL-09 <code>hlMaRotationKeepsEarlierSessionAdapterIds</code> A rotation leaves the adapter id of a session already minted where it stood.	✓	
MA-HL-10 <code>hlMaRotationKeepsEarlierSessionQuorums</code> A rotation leaves the quorum of a session already minted where it stood.	✓	
MA-HL-11 <code>hlMaRotationKeepsEarlierSessionThresholds</code> A rotation leaves the threshold of a session already minted where it stood.	✓	
MA-HL-12 <code>hlMaBlockThenUnblockKeepsTheThreshold</code> A block and restore can never lower the bar a message must clear.	✓	
MA-HL-13 <code>hlMaSendFansOutOncePerRosterMember</code> One send performs exactly one adapter dispatch per member of the live roster.	✓	
MA-HL-14 <code>hlMaSendCarriesTheRequestedChain</code> An outbound dispatch leaves for the chain the send names.	✓	
MA-HL-15 <code>hlMaSendCarriesTheRefundAddressItWasGiven</code> An outbound dispatch names the refund address the send was given.	✓	
MA-HL-16 <code>hlMaSendWrapsThePayloadWithTheLiveSessionId</code> An outbound payload is wrapped with the session id the route serves.	✓	
MA-HL-17 <code>hlMaSendLeavesTheLiveRouteWhereItStands</code> An outbound dispatch leaves the roster it fanned out over exactly where it stood.	✓	

Reverts

Properties	Status	Mutations
MA-RV-01 <code>rvMaFileRefusesANonWardOrAnUnknownName</code> The route is never rewired by an outsider, nor under a name it does not wire.	✓	
MA-RV-02 <code>rvMaUpdateManagerRefusesANonWard</code> Only a ward enrolls or removes a pool transport manager.	✓	
MA-RV-03 <code>rvMaSetAdaptersRefusesWithoutTransportAuthority</code> An outsider can never swap the bridges that carry a pool's traffic.	✓	
MA-RV-04 <code>rvMaSetAdaptersRefusesAnUnreachableThreshold</code> A consensus bar its own roster can never reach is rejected.	✓	
MA-RV-05 <code>rvMaSetAdaptersRefusesAZeroThresholdOverALiveRoster</code> A live roster asking for no consensus at all is rejected.	✓	
MA-RV-06 <code>rvMaSetAdaptersRefusesARepeatedAdapter</code> A roster naming one adapter twice is rejected.	✓	
MA-RV-07 <code>rvMaSetAdaptersRefusesAnUnexpectedSessionId</code> An install must name the route's next session id, so the two endpoints cannot desync.	✓	

MA-RV-08 <code>rvMaSetAdaptersOnAFreshRosterAlwaysGoesThroughForAWard</code> The documented recovery rotation is always available to a ward attaching no value.	✓	
MA-RV-09 <code>rvMaBlockSessionRefusesAnOutsiderOrASessionWithNoRoster</code> No outsider can black out a route, and a session holding no roster cannot be blacked out.	✓	
MA-RV-10 <code>rvMaUnblockSessionRefusesWithoutTransportAuthority</code> A set governance retired stays retired against an outsider.	✓	
MA-RV-11 <code>rvMaUnblockSessionRefusesWithoutASTashedSession</code> Only a session actually held out of service can be restored.	✓	
MA-RV-12 <code>rvMaUnblockSessionOfASingleStashAlwaysGoesThroughForAWard</code> A route retired by mistake can always be put back in service by a ward.	✓	
MA-RV-13 <code>rvMaHandleRefusesASTrangerSubmission</code> A delivery is refused unless the caller is that adapter or a manager of the payload's pool.	✓	
MA-RV-14 <code>rvMaVoteRefusesASTrangerSubmission</code> A vote is refused unless the caller is that adapter or a manager of the payload's pool.	✓	
MA-RV-15 <code>rvMaExecuteRefusesASTrangerSubmission</code> An execution is refused unless the caller is that adapter or a manager of the payload's pool.	✓	
MA-RV-16 <code>rvMaExecuteRefusesAnUnregisteredAdapter</code> An execution is never attributed to an address no session of the route ever registered.	✓	
MA-RV-17 <code>rvMaHandleRefusesAnUnregisteredSender</code> An unregistered address cannot deliver a message on its own behalf.	✓	
MA-RV-18 <code>rvMaVoteRefusesAnUnregisteredSender</code> An unregistered address cannot cast a confirmation on its own behalf.	✓	
MA-RV-19 <code>rvMaExecuteRefusesAnUnregisteredSender</code> An unregistered address cannot spend a tally on its own behalf.	✓	
MA-RV-20 <code>rvMaSendRefusesANonWard</code> Only a ward hands the route an outbound message.	✓	
MA-RV-21 <code>rvMaSendRefusesWithoutALiveRoster</code> A route with no adapters refuses outbound traffic instead of dropping it.	✓	
MA-RV-22 <code>rvMaEstimateRefusesWithoutALiveRoster</code> A route with no adapters refuses to quote a message rather than quoting zero.	✓	
MA-RV-23 <code>rvMaQuorumAlwaysAnswers</code> The roster size a route votes against is readable for every route.	✓	
MA-RV-24 <code>rvMaThresholdAlwaysAnswers</code> The consensus bar of a route is readable for every route.	✓	
MA-RV-25 <code>rvMaNextActiveSessionIdRefusesARouteWhoseCounterIsSpent</code> The id an install must name is readable until the route's counter is spent, then refused.	✓	

Reachability

Properties	Status	Mutations

MA-RC-01 rcMaFirstRosterInstallIsReachable	Witnesses a pool transport route configured for the first time, one adapter at the lowest bar.	✓	
MA-RC-02 rcMaRosterCapInstallIsReachable	Witnesses a roster installed at the protocol cap of eight adapters under unanimity.	✓	
MA-RC-03 rcMaIntermediateThresholdInstallIsReachable	Witnesses a roster installed at a bar strictly between one adapter and unanimity.	✓	
MA-RC-04 rcMaManagerHaltsARouteWithTheEmptySetIsReachable	Witnesses a transport manager halting its own pool's route by installing the empty adapter set.	✓	
MA-RC-05 rcMaBlockingTheSessionInServiceIsReachable	Witnesses the session in service being taken out, its roster moved into the stash and the outbound route left cleared.	✓	
MA-RC-06 rcMaBlockThenUnblockRestoresTheRouteIsReachable	Witnesses a session taken out of service and put back, roster and route returning intact.	✓	
MA-RC-07 rcMaBlockingASupersededSessionKeepsTheRouteIsReachable	Witnesses a superseded session retired while the route carrying traffic stays where it stands.	✓	
MA-RC-08 rcMaUnblockingASupersededSessionKeepsTheRouteIsReachable	Witnesses a superseded session coming back without taking the live route off its session.	✓	
MA-RC-09 rcMaSendFansOutOverTheLiveRosterIsReachable	Witnesses an outbound payload handed to the adapter in service, badged and with its refund address.	✓	
MA-RC-10 rcMaSendResumesAfterABlockAndUnblockIsReachable	Witnesses outbound traffic resuming on a route taken out of service and put back.	✓	
MA-RC-11 rcMaSecondDeliveryCompletesTheQuorumIsReachable	Witnesses a first delivery recording a proof and a second completing a two of two quorum.	✓	
MA-RC-12 rcMaExecuteReleasesThePayloadAfterTwoProofsIsReachable	Witnesses a tally built by two proofs and then spent by an explicit execution.		
MA-RC-13 rcMaThresholdOneLetsOneAdapterForwardAloneIsReachable	Witnesses one adapter of a two adapter roster releasing a payload alone.	✓	
MA-RC-14 rcMaExecuteDrivesASilentLaneNegativeIsReachable	Witnesses a vote lane driven below zero, the lanes being a running ledger and not a count.	✓	
MA-RC-15 rcMaSupersededSessionStillDeliversIsReachable	Witnesses an in flight message of a superseded session still resolving after a rotation.	✓	
MA-RC-16 rcMaManagerSeatGrantAndRevokeIsReachable	Witnesses a pool transport manager seat granted to a fresh account and taken back.	✓	
MA-RC-17 rcMaFileRepointsBothWiringTargetsIsReachable	Witnesses both wiring targets being repointed.	✓	
MA-RC-18 rcMaWardSeatGrantAndRevokeIsReachable	Witnesses a ward seat granted to an account outside the ledger and revoked again.	✓	

Access Control

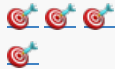
Properties	Status	Mutations
MA-AC-01 <code>acMaOnlyWardMovesTheWardSet</code> Only an existing ward can grant or revoke the ward bit.	✓	
MA-AC-02 <code>acMaOnlyWardModifiesManagerRights</code> Only a ward can enrol or remove a pool's transport manager.	✓	
MA-AC-03 <code>acMaOnlyWardRepointsTheGateway</code> Only a ward can repoint the gateway, so no outsider can redirect executed messages to a destination of its own.	✓	
MA-AC-04 <code>acMaOnlyWardRepointsTheMessageProperties</code> Only a ward can repoint the message model that says which pool a message belongs to.	✓	
MA-AC-05 <code>acMaOnlyWardFansOutASend</code> Only a ward can make the protocol dispatch outbound traffic.	✓	
MA-AC-06 <code>acMaOnlyWardOrManagerAdvancesTheSession</code> Only a ward or the pool's transport manager can advance the session.	✓	
MA-AC-07 <code>acMaOnlyWardOrManagerMovesTheLiveRoute</code> Only a ward or the pool's transport manager can move the roster a send fans out to.	✓	
MA-AC-08 <code>acMaOnlyWardOrManagerMovesASessionRoster</code> Only a ward or the pool's transport manager can rewrite a session's roster.	✓	
MA-AC-09 <code>acMaOnlyWardOrManagerMovesAnAdapterRegistration</code> Only a ward or the pool's transport manager can move an adapter's registration.	✓	
MA-AC-10 <code>acMaOnlyWardOrManagerMovesTheBlockedStash</code> Only a ward or the pool's transport manager can take a session out of service or restore it.	✓	
MA-AC-11 <code>acMaOnlyAdapterOrManagerMovesAVoteTally</code> Consensus cannot be forged from outside the adapter set; the ward bit alone buys nothing here.	✓	
MA-AC-12 <code>acMaOnlyAdapterOrManagerReachesTheGateway</code> A caller outside the adapter set and the pool's managership hands the gateway nothing.	✓	

Hub_LinkedHubCore

- The chart of accounts bounded to 3 symbolic accounts
- One symbolic pool
- One symbolic share class
- Networks bounded to 3 symbolic networks
- Holding rows bounded to 2 symbolic rows
- The deficit counter pinned to one symbolic network
- Loops running up to 4 iterations

Valid State

--	--	--

Properties	Status	Mutations
HU-VS-01 <code>journalDebitsEqualCredits</code> Double-entry bookkeeping holds at rest over the pool's bounded account set.	✓	
HU-VS-02 <code>holdingAccountIdsExist</code> Every accounting account a holding references exists in the ledger.	✓	
HU-VS-03 <code>initializedHoldingHasAccounts</code> An initialized holding carries all four of its journal accounts, so both legs of every update have a home.	✓	
HU-VS-04 <code>initializedHoldingAssetRegistered</code> A holding is only ever initialized for an asset the registry knows.	✓	
HU-VS-05 <code>registeredPoolIsLocal</code> Once the pool holds a currency it was created on this hub's own network.	✓	
HU-VS-06 <code>initializedHoldingImpliesShareClassExists</code> A holding is only ever initialized on a share class the pool has actually minted.	✓	
HU-VS-07 <code>ledgerAccountImpliesRegisteredPool</code> A ledger row exists only on a pool the registry has created.	✓	

State Transitions

Properties	Status	Mutations
HU-ST-01 <code>stHuPolicySwapAdvancesTheAuthorizationNamespace</code> Changing the rule book a pool runs under advances the namespace its pending authorizations are filed in.	✓	
HU-ST-02 <code>stHuLedgerMoveIsFiledUnderAFreshJournal</code> Every hub posting is filed under a journal identity minted for it.	✓	
HU-ST-03 <code>stHuValueMoveIsBookedExactly</code> Revaluing a holding books the whole difference and nothing beyond it.	✓	
HU-ST-04 <code>stHuPriceSourceSwapNeverRewiresTheAccounts</code> Switching the price source of a live holding never rewires the accounts its entries are posted to.	✓	

Variable Transitions

Properties	Status	Mutations
HU-VT-01 <code>vtHuAuthorizationTenureStepsByAtMostOne</code> A pool's authorization tenure stands still or advances by one, never rewinding.	✓	
HU-VT-02 <code>vtHuJournalSequenceStepsByAtMostOne</code> A pool's journal numbering stands still or advances by one, never rewinding.	✓	
HU-VT-03 <code>vtHuShareClassSequenceStepsByAtMostOne</code> A pool's share class numbering stands still or advances by one, never rewinding.	✓	

High Level

Properties	Status	Mutations
HU-HL-01 <code>hlHuAmountRoundTripKeepsEveryAccountNet</code> A deposit posting and an equal withdrawal posting cancel on every account's net balance.	✓	
HU-HL-02 <code>hlHuValueRoundTripKeepsTheHoldingAccountNetWhenGainAndLossAreOtherAccounts</code> An equal revaluation up and down returns the holding's own account to its starting net.	✓	
HU-HL-03 <code>hlHuAccountWiringMovesOnlyThroughWiringEntries</code> A holding's posting slots are rewired only by the two wiring entries.	✓	
HU-HL-04 <code>hlHuValuationMovesOnlyThroughItsInstallers</code> The valuation quoting every revaluation of a holding is installed only by its two entries.	✓	
HU-HL-05 <code>hlHuSnapshotHookMovesOnlyThroughItsSetter</code> The pool's snapshot hook is repointed only by its dedicated setter.	✓	
HU-HL-06 <code>hlHuLedgerTotalsMoveOnlyThroughJournaling</code> The pool's ledger totals move only through the five journaling entries.	✓	

Reverts

Properties	Status	Mutations
HU-RV-01 <code>rvHuFileRefusesANonWardOrAnUnknownName</code> The hub is never rewired by an outsider, nor under a name it does not wire.	✓	
HU-RV-02 <code>rvHuCreatePoolRefusesARepeatOrAnUnusableOpening</code> A pool is never opened twice, by an outsider, on a foreign network, or without an operator and a known currency.	✓	
HU-RV-03 <code>rvHuSetPolicyRefusesAnUnauthorisedInstall</code> A rule book is never installed on a live pool by a caller holding neither seat.	✓	
HU-RV-04 <code>rvHuInitiateAuthorizationRefusesAnUnauthorisedOrDuplicateSchedule</code> An out of policy call is never put on the clock by an outsider, without a rule book, without a delay, or over a slot already taken.	✓	
HU-RV-05 <code>rvHuCancelAuthorizationRefusesANonManagerOrAnEmptySlot</code> A pending authorization is never withdrawn by an outsider, nor invented where none stands.	✓	
HU-RV-06 <code>rvHuUpdateHubManagerRefusesANonManagerOrANullSeat</code> An operator seat is never moved by an outsider, on an absent pool, or onto the null address.	✓	
HU-RV-07 <code>rvHuUpdateCurrencyRefusesARescalingRedenomination</code> A pool's unit of account is never swapped for an unknown currency or one that would rescale its books.	✓	
HU-RV-08 <code>rvHuCreateAccountRefusesARepeatOrAnUnusableOpening</code> A live row's debit and credit history can never be zeroed by re-creating the row.	✓	
HU-RV-09 <code>rvHuSetHoldingAccountIdRefusesAnAbsentRowOrHolding</code> A holding's postings are never pointed at an absent account or onto a holding never opened.	✓	
HU-RV-10 <code>rvHuUpdateHoldingValuationRefusesANullSourceOrClosedHolding</code>	✓	

A live holding is never left without a way to value it, and a closed one is never given one.		
HU-RV-11 <code>rvHuManagerCallRefusesAMisdeclaredValue</code> A manager call carrying the wrong native value is refused.	✓	
HU-RV-12 <code>rvHuRequestCallbackRefusesAnyCallerButTheRegisteredManager</code> A request result never goes back to the spoke for anyone but the pool's own registered request manager.	✓	
HU-RV-13 <code>rvHuPassingPolicyReadIsReachable</code> A pool's rule book stays readable the moment it is installed.	✓	
HU-RV-14 <code>rvHuPricePoolPerAssetAlwaysAnswers</code> Asking a pool what an asset is worth always answers, a holding never opened included.	✓	
HU-RV-15 <code>rvHuInitializeHoldingRefusesARepeatOrAnUnusableOpening</code> A holding opens once, for a manager, on a live class with a price source and four accounts.	✓	

Reachability

Properties	Status	Mutations
HU-RC-01 <code>rcHuCreatePoolSeatsADelegateIsReachable</code> Witnesses governance opening a pool and handing the operator seat to a third party at once.	✓	
HU-RC-02 <code>rcHuCreatePoolSeatsTheOpeningWardIsReachable</code> Witnesses the ward that opens a pool naming itself its operator.	✓	
HU-RC-03 <code>rcHuCreatePoolAtTheSmallestCurrencyIsReachable</code> Witnesses a pool opened in the smallest usable currency id.	✓	
HU-RC-04 <code>rcHuPoolBirthThenDelegationIsReachable</code> Witnesses a pool opened and a second operator appointed in one transaction.	✓	
HU-RC-05 <code>rcHuSecondOperatorAppointmentIsReachable</code> Witnesses a sitting operator appointing a second one alongside itself, so the operator set of a live pool can grow.	✓	
HU-RC-06 <code>rcHuOperatorSeatRoundTripIsReachable</code> Witnesses an operator seat being granted and withdrawn again in one transaction, so appointment is not a one way door.	✓	
HU-RC-07 <code>rcHuOperatorCanStandItselfDownIsReachable</code> Witnesses an operator withdrawing its own seat.	✓	
HU-RC-08 <code>rcHuGovernanceInstallsTheFirstRuleBookIsReachable</code> Witnesses governance installing a pool's first rule book without holding an operator seat.	✓	
HU-RC-09 <code>rcHuOperatorSwapsTheRuleBookIsReachable</code> Witnesses an operator swapping a standing rule book, the tenure advancing with it.	✓	
HU-RC-10 <code>rcHuRuleBookCanBeRemovedIsReachable</code> Witnesses a pool's rule book cleared with the null address, the tenure advancing all the same.	✓	
HU-RC-11 <code>rcHuPrivilegedCallSchedulingIsReachable</code> Witnesses an operator putting a privileged call on the schedule under the standing rule	✓	

book.		
HU-RC-12 <code>rcHuScheduledCallWithdrawalIsReachable</code> Witnesses a scheduled call withdrawn in the transaction that scheduled it.	✓	
HU-RC-13 <code>rcHuScheduledCallAtTheMaturityCeilingIsReachable</code> Witnesses the longest maturity the schedule field can hold being reached.	✓	
HU-RC-14 <code>rcHuFirstShareClassIsReachable</code> Witnesses a pool's very first share class added through the hub, the salt consumed with it.	✓	
HU-RC-15 <code>rcHuDebitNormalRowCanBeOpenedIsReachable</code> Witnesses an asset or expense row opened debit normal and stamped with its opening block.	✓	
HU-RC-16 <code>rcHuCreditNormalRowCanBeOpenedIsReachable</code> Witnesses an equity or liability row opened credit normal.	✓	
HU-RC-17 <code>rcHuAccountsThenHoldingIsReachable</code> Witnesses the whole opening walk of a holding in one transaction, wired to four posting slots.	✓	
HU-RC-18 <code>rcHuHoldingOpeningValueBookedAsPrincipalIsReachable</code> Witnesses a holding's whole opening value booked as principal under one journal identity.	✓	
HU-RC-19 <code>rcHuHoldingOpeningWithNothingToBookIsReachable</code> Witnesses a holding opened on an empty row, costing the ledger no journal identity.	✓	
HU-RC-20 <code>rcHuHoldingWithARepeatedSlotIsReachable</code> Witnesses one ledger row shared between two of a holding's four posting slots.	✓	
HU-RC-21 <code>rcHuHoldingMarkUpIsBookedIsReachable</code> Witnesses the first mark up of a live holding, booked on both sides under one identity.	✓	
HU-RC-22 <code>rcHuHoldingMarkDownToZeroIsReachable</code> Witnesses a holding written all the way down to zero on the price source's word alone.	✓	
HU-RC-23 <code>rcHuRevaluationLeavingTheBooksUntouchedIsReachable</code> Witnesses a revaluation that lands on the value already carried, spending nothing.	✓	
HU-RC-24 <code>rcHuDepositPostingIsReachable</code> Witnesses a deposit booked, the holding's row debited and its counterpart credited.	✓	
HU-RC-25 <code>rcHuWithdrawalPostingIsReachable</code> Witnesses a withdrawal booked as the exact mirror of a deposit.	✓	
HU-RC-26 <code>rcHuAmountRoundTripIsReachable</code> Witnesses a deposit and an equal withdrawal filed under one journal identity.	✓	
HU-RC-27 <code>rcHuZeroAmountPostingIsAcceptedIsReachable</code> Witnesses a zero amount posting being accepted and moving nothing.	✓	
HU-RC-28 <code>rcHuLargestAmountPostingIsReachable</code> Witnesses the largest amount the posting field can carry booked in a single call.	✓	
HU-RC-29 <code>rcHuValueGainPostingIsReachable</code> Witnesses a revaluation gain booked under one fresh journal identity.	✓	
HU-RC-30 <code>rcHuValueLossPostingIsReachable</code>		

Witnesses a revaluation loss booked the other way about.	✓	
HU-RC-31 <code>rcHuZeroValuePostingIsAcceptedIsReachable</code> Witnesses a zero revaluation being accepted and moving nothing.	✓	
HU-RC-32 <code>rcHuMinimalBalancedJournalIsReachable</code> Witnesses the smallest committing batch: one debit and one credit of equal value.	✓	
HU-RC-33 <code>rcHuWiderBalancedJournalIsReachable</code> Witnesses a wider batch committing, two entries a side with the sums meeting.	✓	
HU-RC-34 <code>rcHuEmptyJournalIsReachable</code> Witnesses an empty journal opening and closing, spending an identity while posting nothing.	✓	
HU-RC-35 <code>rcHuZeroValuedJournalPairIsReachable</code> Witnesses a zero valued pair committing and re-stamping live rows without moving them.	✓	
HU-RC-36 <code>rcHuSnapshotHookRoundTripIsReachable</code> Witnesses NAV automation switched on and then off again in one transaction.	✓	
HU-RC-37 <code>rcHuShareClassAnnouncementIsReachable</code> Witnesses a share class announced to a spoke in exactly one outbound message.	✓	
HU-RC-38 <code>rcHuPaidRequestCallbackIsReachable</code> Witnesses the pool's request manager pushing a paid callback out to its spoke.	✓	
HU-RC-39 <code>rcHuUnpaidRequestCallbackIsReachable</code> Witnesses a pool's registered request manager pushing a callback to its spoke in unpaid mode.	✓	
HU-RC-40 <code>rcHuWardAuthorityRoundTripIsReachable</code> Witnesses hub authority extended to a new party and withdrawn again in one transaction.	✓	
HU-RC-41 <code>rcHuWardCanStandItselfDownIsReachable</code> Witnesses a ward removing its own authority.	✓	
HU-RC-42 <code>rcHuUnopenedHoldingPricesAtParIsReachable</code> Witnesses the asset price view answering at par for a holding that was never opened.	✓	

Access Control

Properties	Status	Mutations
HU-AC-01 <code>acHuWardSetMovesOnlyForAWard</code> The hub's trusted set is widened and narrowed only by an address it already trusts.	✓	
HU-AC-02 <code>acHuWiringMovesOnlyForAWard</code> The hub's messaging wiring and its fee hook are repointed only by a ward.	✓	
HU-AC-03 <code>acHuPoolCurrencyMovesOnlyForAWardOrManager</code> The currency a pool keeps its books in is set and changed only by a hub ward or a pool manager.	✓	
HU-AC-04 <code>acHuManagerSeatMovesOnlyForAWardOrManager</code> A pool's manager seat is granted and revoked only by a hub ward or a manager the pool trusts.	✓	

HU-AC-05 <code>acHuPoolPolicyMovesOnlyForAWardOrManager</code> A pool's rule book and its authorization namespace move only for a hub ward or a pool manager.	✓	
HU-AC-06 <code>acHuAuthorizationLedgerMovesThroughTheHubOnlyForAPoolManager</code> A pool's timelocked authorization ledger is written only by a manager of that pool.	✓	
HU-AC-07 <code>acHuBridgingHookMovesOnlyForAPoolManager</code> A pool's bridging hook is repointed only by a manager of that pool.	✓	
HU-AC-08 <code>acHuRequestManagerSeatMovesOnlyForAPoolManager</code> The contract a pool accepts investment requests from is set only by a manager of that pool.	✓	
HU-AC-09 <code>acHuSnapshotHookMovesOnlyForAPoolManager</code> The hook a pool drives its NAV automation from is repointed only by a manager of that pool.	✓	
HU-AC-10 <code>acHuHoldingValuationMovesOnlyForAPoolManager</code> The price source a holding is marked at is installed and swapped only by a pool manager.	✓	
HU-AC-11 <code>acHuHoldingAccountWiringMovesOnlyForAPoolManager</code> The ledger accounts a holding posts into are wired only by a manager of the pool.	✓	
HU-AC-12 <code>acHuHoldingValueMovesThroughTheHubOnlyForAPoolManager</code> The carried value a pool reports for a holding moves only for a manager of that pool.	✓	
HU-AC-13 <code>acHuShareClassRegisterMovesOnlyForAPoolManager</code> A pool's share class register and consumed salts move only for a manager of that pool.	✓	
HU-AC-14 <code>acHuSharePriceMovesOnlyForAPoolManager</code> A share class price and the moment it was computed move only for a manager of the pool.	✓	
HU-AC-15 <code>acHuAccountCreationNeedsAPoolManager</code> A ledger account comes into existence only for a manager of the pool.	✓	
HU-AC-16 <code>acHuAccountDebitMovesThroughTheHubOnlyForAWardOrManager</code> The charges booked against a pool's account move only for a hub ward or a pool manager.	✓	
HU-AC-17 <code>acHuAccountCreditMovesThroughTheHubOnlyForAWardOrManager</code> The receipts booked against a pool's account move only for a hub ward or a pool manager.	✓	
HU-AC-18 <code>acHuJournalCounterMovesThroughTheHubOnlyForAWardOrManager</code> The counter naming each of a pool's journals advances only for a hub ward or a pool manager.	✓	
HU-AC-19 <code>acHuRequestCallbackNeedsTheRegisteredRequestManager</code> A request fulfilment is sent only for the request manager the pool registered on that network.	✓	
HU-AC-20 <code>acHuOutboundInstructionNeedsAPoolManagerOrARequestFulfilment</code> An instruction leaves for another network only for a pool manager or on a request fulfilment.	✓	

HubHandler_LinkedHubCore

- The chart of accounts bounded to 3 symbolic accounts
- One symbolic pool

- One symbolic share class
- Networks bounded to 3 symbolic networks
- Holding rows bounded to 2 symbolic rows
- The deficit counter pinned to one symbolic network
- Loops running up to 4 iterations

Valid State

Properties	Status	Mutations
HH-VS-01 <code>postedDebitsCoverCarriedValue</code> The value carried on any two of a pool's holdings never exceeds what the ledger has posted to the debit side of any three of its accounts.	✓	

State Transitions

Properties	Status	Mutations
HH-ST-01 <code>stHhReportedAssetMoveIsBookedExactly</code> An asset movement reported from a network is booked at the value the holding actually moved by.	✓	
HH-ST-02 <code>stHhShareLedgerMovesWithItsNetworkLegs</code> The class total moves by exactly the net change of its per-network legs.	✓	
HH-ST-03 <code>stHhSnapshotFlagCostsANonce</code> A network's snapshot flag is never set without spending that network's nonce.	✓	
HH-ST-04 <code>stHhDeficitCountFollowsTheShortRows</code> The deficit count moves by exactly the change in the number of holdings in shortfall.	✓	

Variable Transitions

Properties	Status	Mutations
HH-VT-01 <code>vtHhSnapshotSequenceStepsByAtMostOne</code> A network's snapshot sequence advances one step at a time and never rewinds.	✓	
HH-VT-02 <code>vtHhJournalSequenceStepsByAtMostOne</code> A pool's journal numbering advances one step at a time and never rewinds.	✓	
HH-VT-03 <code>vtHhAccountStampRecordsTheCurrentBlock</code> A ledger row's activity stamp, when it moves, records the block the posting happened in.	✓	
HH-VT-04 <code>vtHhHoldingFlowHistoryNeverShrinks</code> What a pool has received and paid out of an asset only ever grow.	✓	
HH-VT-05 <code>vtHhNetworkShareHistoryNeverShrinks</code> What a network has minted and burned of a share class only ever grow.	✓	
HH-VT-06 <code>vtHhLedgerRowTotalsNeverShrink</code> A ledger row's running debit and credit totals only ever grow.	✓	

High Level

Properties	Status	Mutations
HH-HL-01 <code>hlHhTransferConservesTheClassTotal</code> Moving shares between networks never changes the pool's total share supply.	✓	
HH-HL-02 <code>hlHhTransferDebitsOriginByTheSentAmount</code> The burn leg lands on the origin network at exactly the size the outbound message announces.	✓	
HH-HL-03 <code>hlHhTransferSendsTheAmountItMints</code> The amount announced to the destination chain is the amount actually minted on its ledger.	✓	
HH-HL-04 <code>hlHhTransferKeepsTheOriginNetwork</code> The outbound execute message names the network the shares were burned on.	✓	
HH-HL-05 <code>hlHhTransferKeepsTheTargetNetwork</code> The outbound execute message is addressed to the network the caller asked for.	✓	
HH-HL-06 <code>hlHhHooklessTransferKeepsTheReceiver</code> With no bridging hook, the transfer names the destination party the caller asked for.	✓	
HH-HL-07 <code>hlHhHooklessTransferKeepsTheAmount</code> With no bridging hook, the transfer sends the size the caller asked for.	✓	
HH-HL-08 <code>hlHhHooklessTransferKeepsTheRefund</code> With no bridging hook, the transfer keeps the caller's refund address.	✓	
HH-HL-09 <code>hlHhTransferSendsExactlyOneMessage</code> One transfer emits exactly one outbound message.	✓	
HH-HL-10 <code>hlHhTransferSendsAnExecuteTransfer</code> The message a transfer emits is the execute transfer for the destination network.	✓	
HH-HL-11 <code>hlHhAssetReportLandsGrossOnItsSide</code> An asset flow reported by a spoke lands on the side it was reported on at its full gross size.	✓	
HH-HL-12 <code>hlHhAssetReportLeavesTheOppositeSideAlone</code> The holding counters stay gross: a reported flow never moves the opposite counter.	✓	
HH-HL-13 <code>hlHhAssetReportLeavesOtherIncreasesAlone</code> One asset's report cannot raise another asset's incoming counter.	✓	
HH-HL-14 <code>hlHhAssetReportLeavesOtherDecreasesAlone</code> One asset's report cannot raise another asset's outgoing counter.	✓	
HH-HL-15 <code>hlHhAssetReportLeavesOtherValuesAlone</code> One asset's report cannot revalue another asset's holding.	✓	
HH-HL-16 <code>hlHhShareReportLandsOnItsCounter</code> A network's share report lands on the counter it names at its full size.	✓	
HH-HL-17 <code>hlHhShareReportMovesTheClassTotalInStep</code> A share report moves the class total by the reported amount in the reported direction.	✓	
HH-HL-18 <code>hlHhRequestForwardsExactlyOnce</code> An investor request is forwarded once and only once.	✓	
HH-HL-19 <code>hlHhRequestReachesTheRegisteredManager</code>		

The forwarding call goes to the request manager registered for the asset's network, never to anyone else.	✓	
HH-HL-20 <code>hlHhRequestForwardsThePool</code> An investor request arrives at the manager under the pool it was sent for.	✓	
HH-HL-21 <code>hlHhRequestForwardsTheShareClass</code> An investor request arrives at the manager under the share class it was sent for.	✓	
HH-HL-22 <code>hlHhRequestForwardsTheAsset</code> An investor request arrives at the manager under the asset it was sent for.	✓	
HH-HL-23 <code>hlHhRequestForwardsThePayload</code> The request body reaches the manager byte for byte, so the order the manager executes is the order the spoke sent.	✓	

Reverts

Properties	Status	Mutations
HH-RV-01 <code>rvHhFileRefusesANonWardOrAnUnknownName</code> The handler is never rewired by an outsider, nor under a name it does not wire.	✓	
HH-RV-02 <code>rvHhRegisterAssetRefusesANonWardAnOversizedPrecisionOrARepet</code> An asset is registered on the hub at most once, by a ward, at a precision the arithmetic carries.	✓	
HH-RV-03 <code>rvHhRequestRefusesANonWardOrANetworkWithNoManager</code> A request whose network has no manager registered is refused rather than silently dropped.	✓	
HH-RV-04 <code>rvHhUpdateSharesRefusesANonWardAnUnknownClassOrAMissedNonce</code> A share report does not land for a stranger, for a class the hub does not know, or under any nonce but that network's next.	✓	 
HH-RV-05 <code>rvHhUpdateAssetsRefusesANonWardAMissedNonceOrAFullCounter</code> An asset report reaches the books only under that network's next nonce, so a replay is refused.	✓	
HH-RV-06 <code>rvHhUninitializedAssetReportIsNeverStrandedByTheShortfallTally</code> On an uninitialized holding a report moves quantities only, and neither the handler nor the hub core strands it on the shortfall tally.	✓	
HH-RV-07 <code>rvHhTransferRefusesANonWardOrAnUnknownShareClass</code> A transfer naming a share class the hub does not know is refused, hook or no hook.	✓	
HH-RV-08 <code>rvHhHooklessTransferIsNeverStrandedWhileTheCountersHaveRoom</code> With no bridging hook, neither the handler nor the hub core stops a ward's transfer while the counters have room.	✓	
HH-RV-09 <code>rvHhRelyRefusesANonWard</code> The roll of parties that may feed the hub is extended by a sitting ward alone.	✓	
HH-RV-10 <code>rvHhDenyRefusesANonWard</code> The roll of parties that may feed the hub is trimmed by a sitting ward alone.	✓	
HH-RV-11 <code>rvHhRequestIsNeverRefusedOnANetworkThatHasAManager</code>		

A request reaches the manager the network registered, and the core itself never turns it away.	✓	
--	---	---

Reachability

Properties	Status	Mutations
HH-RC-01 <code>rcHhRegisterAssetIsReachable</code> Witnesses a spoke announcing an asset the hub has never seen, the row going live.	✓	
HH-RC-02 <code>rcHhRegisterAssetAtTheDecimalsCeilingIsReachable</code> Witnesses an asset registering at eighteen decimals, the ceiling the hub still accepts.	✓	
HH-RC-03 <code>rcHhRegisterAssetAtZeroDecimalsIsReachable</code> Witnesses an asset registering with zero decimals, the floor of the range the hub bounds only from above.	✓	
HH-RC-04 <code>rcHhRequestIsReachable</code> Witnesses an investor request reaching a live request manager under the pool it names.	✓	
HH-RC-05 <code>rcHhRequestWithAnEmptyPayloadIsReachable</code> Witnesses a bare notification, a request carrying no body at all, still being forwarded to the pool's request manager.	✓	
HH-RC-06 <code>rcHhAssetDepositOnALiveHoldingIsReachable</code> Witnesses a spoke reported deposit on a live holding, the books taking a matched pair.	✓	
HH-RC-07 <code>rcHhAssetWithdrawalOnALiveHoldingIsReachable</code> Witnesses a reported withdrawal on a live holding booked, devalued and journaled in one call.	✓	
HH-RC-08 <code>rcHhAssetReportOnAnUnopenedHoldingIsReachable</code> Witnesses a report for a holding the hub never opened: counters take it, nothing is journaled.	✓	
HH-RC-09 <code>rcHhAssetReportClosingTheBatchIsReachable</code> Witnesses a spoke closing its asset batch against a freshly spent nonce.	✓	
HH-RC-10 <code>rcHhEmptyAssetBatchClosingIsReachable</code> Witnesses an empty batch still being closed: a report that moves nothing at all can leave the network marked in sync.	✓	
HH-RC-11 <code>rcHhLargestAssetReportIsReachable</code> Witnesses the largest single asset report the message format can carry landing whole on an empty holding row.	✓	
HH-RC-12 <code>rcHhFirstAssetReportOfANetworkIsReachable</code> Witnesses the very first report of a network, arriving at nonce zero and opening that network's snapshot history.	✓	
HH-RC-13 <code>rcHhAssetWithdrawalEnteringDeficitIsReachable</code> Witnesses a reported withdrawal taking a holding past everything it ever received.	✓	
HH-RC-14 <code>rcHhAssetDepositLeavingDeficitIsReachable</code> Witnesses a reported deposit covering the whole shortfall.	✓	
HH-RC-15 <code>rcHhShareIssuanceIsReachable</code>	✓	

Witnesses a spoke reporting shares minted on its network.		
HH-RC-16 <code>rcHhShareRevocationIsReachable</code> Witnesses a network reporting burned shares, raising its own revocations and lowering the class total.	✓	
HH-RC-17 <code>rcHhShareRevocationDrainingTheClassIsReachable</code> Witnesses a share class drained to zero supply by one report.	✓	
HH-RC-18 <code>rcHhShareReportClosingTheBatchIsReachable</code> Witnesses a spoke closing its share batch against a freshly spent nonce.	✓	
HH-RC-19 <code>rcHhMarkerOnlyShareReportIsReachable</code> Witnesses a marker only share report: a network can close its batch while reporting no share movement at all.	✓	
HH-RC-20 <code>rcHhHooklessBridgeIsReachable</code> Witnesses the plain cross network bridge, the class total standing still.	✓	
HH-RC-21 <code>rcHhBridgeReachableWithAHookRewritingTheAmount</code> Witnesses a pool's own bridging hook rewriting the transfer it was handed.	✓	
HH-RC-22 <code>rcHhBridgeLeavesAThirdNetworkAloneIsReachable</code> Witnesses a bridge landing on both legs while a third network keeps its own position.	✓	
HH-RC-23 <code>rcHhEmptyBridgeLegIsReachable</code> Witnesses an empty bridge leg still paying for a cross chain message.	✓	
HH-RC-24 <code>rcHhBridgeEmptyingTheOriginNetworkIsReachable</code> Witnesses a network emptied of a share class in one bridge.	✓	
HH-RC-25 <code>rcHhBridgeDrivesTheOriginNetworkNegativeIsReachable</code> Witnesses a bridge sending more shares out of a network than it ever reported receiving.	✓	
HH-RC-26 <code>rcHhFileRepointsTheHubIsReachable</code> Witnesses a ward repointing the handler at a new hub, the reference every asset report journals through.	✓	
HH-RC-27 <code>rcHhFileRepointsTheHoldingsLedgerIsReachable</code> Witnesses a ward repointing the handler at a new holdings ledger.	✓	
HH-RC-28 <code>rcHhFileRepointsTheSenderIsReachable</code> Witnesses a ward repointing the handler at a new outbound message sender.	✓	
HH-RC-29 <code>rcHhFileRepointsTheShareClassBookIsReachable</code> Witnesses a ward repointing the handler at a new share class book.	✓	
HH-RC-30 <code>rcHhRelyGrantsAWardIsReachable</code> Witnesses a ward extending the handler's authority to an account that held none.	✓	
HH-RC-31 <code>rcHhRelyThenDenyRoundTripIsReachable</code> Witnesses authority granted and taken back within one transaction.	✓	
HH-RC-32 <code>rcHhRegisterThenReportOnTheAssetIsReachable</code> Witnesses an asset announced and then carrying reported flow in the same transaction.	✓	
HH-RC-33 <code>rcHhIssueThenRevokeRestoresTheClassTotalIsReachable</code> Witnesses a network minting and then burning the same shares.	✓	

HH-RC-34 <code>rcHhSnapshotSetThenClearedIsReachable</code> Witnesses a network closing its batch and opening the next in the same transaction.	✓	
HH-RC-35 <code>rcHhBridgeAndReturnLegIsReachable</code> Witnesses a bridge and its return leg, both positions ending where they started.	✓	

Access Control

Properties	Status	Mutations
HH-AC-01 <code>acHhOnlyAWardMovesTheWardSet</code> Only an address already holding it can hand out or take back the right to feed the hub.	✓	
HH-AC-02 <code>acHhOnlyAWardMovesTheHubPointer</code> Only a ward repoints the hub an inbound message is applied against.	✓	
HH-AC-03 <code>acHhOnlyAWardMovesTheHoldingsPointer</code> Only a ward repoints the holdings ledger an inbound asset report is written to.	✓	
HH-AC-04 <code>acHhOnlyAWardMovesTheSenderPointer</code> Only a ward repoints the channel the hub answers spokes on.	✓	
HH-AC-05 <code>acHhOnlyAWardMovesTheShareClassManagerPointer</code> Only a ward repoints the share class book an inbound share report is written to.	✓	
HH-AC-06 <code>acHhOnlyAWardRegistersAnAsset</code> An asset id enters the hub's registry through the handler only for a ward.	✓	
HH-AC-07 <code>acHhOnlyAWardMovesAnAssetDecimals</code> The decimals an asset is measured in are set through the handler only for a ward.	✓	
HH-AC-08 <code>acHhOnlyAWardMovesADepositHistory</code> What a pool has received of an asset moves through the handler only for a ward.	✓	
HH-AC-09 <code>acHhOnlyAWardMovesAWithdrawalHistory</code> What a pool has paid out of an asset moves through the handler only for a ward.	✓	
HH-AC-10 <code>acHhOnlyAWardMovesACarriedValue</code> The carried value of a pool's position moves through the handler only for a ward.	✓	
HH-AC-11 <code>acHhOnlyAWardMovesTheDeficitCount</code> The count of a network's positions in shortfall moves through the handler only for a ward.	✓	
HH-AC-12 <code>acHhOnlyAWardMovesASyncMarker</code> The marker saying a network is in sync flips through the handler only for a ward.	✓	
HH-AC-13 <code>acHhOnlyAWardMovesASnapshotNonce</code> The sequence ordering a network's snapshot reports advances through the handler only for a ward.	✓	
HH-AC-14 <code>acHhOnlyAWardMovesANetworkIssuedTotal</code> The shares a network has issued move through the handler only for a ward.	✓	
HH-AC-15 <code>acHhOnlyAWardMovesANetworkRevokedTotal</code> The shares a network has burned move through the handler only for a ward.	✓	
HH-AC-16 <code>acHhOnlyAWardMovesTheClassSupply</code> The supply of a share class moves through the handler only for a ward.	✓	

HH-AC-17 <code>acHhOnlyAWardMovesALedgerDebit</code> A debit reaches a pool's ledger account on a handler entry only for a ward.	✓	
HH-AC-18 <code>acHhOnlyAWardMovesALedgerCredit</code> A credit reaches a pool's ledger account on a handler entry only for a ward.	✓	
HH-AC-19 <code>acHhOnlyAWardMovesTheJournalCounter</code> The counter numbering a pool's journal entries advances through the handler only for a ward.	✓	
HH-AC-20 <code>acHhOnlyAWardForwardsAnInvestorRequest</code> An investor order reaches the pool's request manager through the handler only for a ward.	✓	
HH-AC-21 <code>acHhOnlyAWardSendsAnOutboundMessage</code> An instruction leaves the hub for another network through the handler only for a ward.	✓	

Spoke_LinkedSpokeCore

- One symbolic pool
- One symbolic share class
- Asset ids drawn from the queue's bounded set
- ERC20 and ERC6909 tokens modeled in CVL (standard, up to 3 holders, uint128 balances, 6 to 18 decimals, no share token hooks)

Valid State

Properties	Status	Mutations
SP-VS-01 <code>queuedAssetRowImpliesRegisteredAsset</code> A queued asset row belongs to an asset the registry knows, so the flush can always resolve what it is reporting.	✓	
SP-VS-02 <code>escrowedRowImpliesRegisteredAsset</code> Custody on a row means the registry knows that asset, so the escrow holds nothing it cannot name.	✓	
SP-VS-03 <code>escrowedCustodyImpliesCreatedPool</code> Custody on a row means the pool was created, so no escrow carries funds for a pool the spoke never opened.	✓	
SP-VS-04 <code>queueActivityImpliesCreatedPool</code> Queued work means the pool was created, so nothing waits to be reported for a pool the spoke never opened.	✓	
SP-VS-05 <code>registeredAssetIsLocal</code> An asset id registered through the spoke carries this spoke's own chain, so it can never name a foreign network.	✓	
SP-VS-06 <code>escrowCustodyIsBackedOrNoted</code> Custody the escrow books is covered by the tokens it holds, once what entered without a transfer is counted.	✓	

State Transitions

--	--	--

Properties	Status	Mutations
SP-ST-02 <code>stSpSpendableBalanceMoveIsQueuedForTheHub</code> Every move of the balance a pool can spend is queued for the hub at the same size and direction.	✓	
SP-ST-03 <code>stSpDrainingAQueueRowSpendsANonceAndItsSlot</code> Emptying an asset's pending report consumes its ordering nonce and releases its slot.	✓	
SP-ST-04 <code>stSpShareSupplyMovesOnlyAsReported</code> Outside a report flush and a bridge transfer, shares minted or burned on this network move the pending report by exactly the same net.	✓	
SP-ST-05 <code>stSpEscrowCustodyMovesWithItsTokens</code> The escrow stays solvent step by step: the tokens it holds and the custody it books move together.	✓	
SP-ST-06 <code>stSpNoteDepositBooksExactlyWhatItWasHanded</code> NoteDeposit books custody without a token, and books exactly the amount it was handed.	✓	
SP-ST-07 <code>stSpWithdrawMovesTokensAndBookTogether</code> A withdrawal moves the escrow's tokens and its book by the same amount.	✓	
SP-ST-08 <code>stSpWithdrawReservedMovesTokensAndBookTogether</code> A withdrawal against an earmark moves the escrow's tokens and its book by one amount, so releasing a hold does not let custody and tokens part company.	✓	
SP-ST-09 <code>rcSpWithdrawToAnotherAddressIsReachable</code> A withdrawal to another address really moves the escrow's tokens, so the claim that its tokens and its book move together stands over a path where something happens.	✓	

High Level

Properties	Status	Mutations
SP-HL-01 <code>hlSpDepositThenWithdrawRestoresCustody</code> A deposit and an equal withdrawal leave custody exactly where they found it.	✓	
SP-HL-02 <code>hlSpDepositThenWithdrawRestoresTheQueuedNet</code> A deposit and an equal withdrawal leave the pending report to the hub where they found it.	✓	
SP-HL-03 <code>hlSpReserveThenUnreserveLeavesCustodyUntouched</code> Taking a hold and releasing it never moves custody.	✓	
SP-HL-04 <code>hlSpReserveThenUnreserveRestoresTheAggregateHold</code> Releasing a hold gives back the whole aggregate hold.	✓	
SP-HL-05 <code>hlSpReserveThenUnreserveRestoresTheReserversBucket</code> Releasing a hold gives back the bucket it was taken from, never somebody else's.	✓	
SP-HL-06 <code>hlSpReserveThenUnreserveRestoresTheQueuedNet</code> Releasing a hold gives back the pending report to the hub.	✓	
SP-HL-07 <code>hlSpWithdrawReservedPaysOutTheRequestedAmount</code> Withdrawing reserved funds moves out exactly the payout the caller asked for.	✓	
SP-HL-08 <code>hlSpWithdrawReservedConsumesTheHold</code>		

Withdrawing reserved funds consumes the hold that covered them.	✓	
SP-HL-09 <code>hlSpWithdrawReservedLeavesTheQueueUntouched</code> Withdrawing reserved funds tells the hub nothing, the decrease having been queued at reservation.	✓	
SP-HL-16 <code>hlSpRevokePullsSharesFromTheCaller</code> A revoke takes the shares from the caller, never out of somebody else's balance.	✓	
SP-HL-17 <code>hlSpRevokeBurnsThePulledShares</code> The shares a revoke takes in are destroyed, not made available to somebody else.	✓	
SP-HL-18 <code>hlSpRevokeParksNoSharesOnTheSpoke</code> A revoke parks no shares on the spoke, so nothing on its way to being burned can be picked up by a rescue path.	✓	
SP-HL-19 <code>hlSpRevokeQueuesTheBurnAsADecrease</code> A revoke reports to the hub exactly the shares it burned.	✓	
SP-HL-20 <code>hlSpRegisterAssetSendsOneRegistrationAnnouncement</code> Every registration tells the hub about the asset exactly once.	✓	
SP-HL-21 <code>hlSpRegisterAssetAnnouncesTheStoredId</code> A registration announces the very id the registry keeps for the token.	✓	
SP-HL-22 <code>hlSpAssetFlushAnnouncesAnUpdateForTheFlushedRow</code> An asset flush reports under the asset row it was asked to flush.	✓	
SP-HL-23 <code>hlSpAssetFlushReportsTheRowsNetSize</code> An asset flush reports the size of the net flow the row accumulated.	✓	
SP-HL-24 <code>hlSpAssetFlushReportsTheRowsNetDirection</code> An asset flush reports the direction of the net flow the row accumulated.	✓	
SP-HL-25 <code>hlSpAssetFlushReportsTheStoredSequenceNumber</code> An asset flush reports under the sequence number the queue had stored.	✓	
SP-HL-26 <code>hlSpAssetFlushEmptiesTheRow</code> An asset flush empties the row it reported, so the same flow is never handed to the hub a second time.	✓	
SP-HL-27 <code>hlSpShareFlushAnnouncesAShareUpdate</code> A share flush always hands the hub a share update, so no flush passes without the hub hearing about it.	✓	
SP-HL-28 <code>hlSpShareFlushReportsTheQueuedNetSize</code> A share flush reports the size of the net share movement the queue accumulated.	✓	
SP-HL-29 <code>hlSpShareFlushReportsTheQueuedNetDirection</code> A share flush reports the direction of the net share movement.	✓	
SP-HL-30 <code>hlSpShareFlushReportsTheStoredSequenceNumber</code> A share flush reports under the sequence number the queue had stored.	✓	
SP-HL-31 <code>hlSpShareFlushClearsTheQueuedNet</code> A share flush clears the delta it reported, so the same share movement is never handed to the hub a second time.	✓	

SpokeHandler_LinkedSpokeRegistry

- One symbolic pool
- One symbolic share class
- ERC20 tokens modeled in CVL (standard, up to 3 holders, uint128 balances, 6 to 18 decimals, no share token hooks)

State Transitions

Properties	Status	Mutations
SH-ST-02 <code>stShVaultLinkNeverMovesItsIdentity</code> A link flag moves only on a row that ends up registered, the vault keeping what it serves.	✓	
SH-ST-03 <code>stShShareClassBindsTheRegistrarsTokenAndItsLookupBack</code> A share class that gains a token gains some registrar rather than none, and a lookup back to its own pool and class.	✓	 
SH-ST-04 <code>stShPolicySwapAdvancesTheAuthorizationNamespace</code> Changing the rule book a pool runs under advances the namespace its outstanding authorizations are filed in.	✓	
SH-ST-05 <code>stShAuthorizationsMoveOnlyWithinAStandingNamespace</code> Authorizations are granted and revoked only against the rule book in force.	✓	
SH-ST-06 <code>stShPricesNeedTheRowsTheyPriceTo</code> Prices are recorded only for things that exist, so no valuation is seeded onto a class that was never created.	✓	

Variable Transitions

Properties	Status	Mutations
SH-VT-01 <code>vtShLiveShareClassKeepsItsTokenAndRegistrar</code> While a share class carries a token, no inbound message repoints it at another.	✓	
SH-VT-02 <code>vtShShareTokenKeepsThePoolItBacks</code> A share token recorded as backing a pool keeps that record across any inbound message.	✓	

High Level

Properties	Status	Mutations
SH-HL-01 <code>hlShExecuteTransferSharesMintsExactlyTheBridgedAmount</code> The destination leg of a bridge transfer creates exactly the amount the message names.	✓	
SH-HL-02 <code>hlShExecuteTransferSharesPaysTheReceiverInFull</code> Every share the bridge leg creates reaches the receiver the message names, in full.	✓	
SH-HL-03 <code>hlShExecuteTransferSharesAddressedPastTheHandlerLeavesNoResidue</code> For a transfer addressed past the handler, the handler forwards everything and keeps none.	✓	
SH-HL-04 <code>hlShShareSupplyMovesOnlyOnTheTransferExecutor</code> No inbound message but the transfer executor can create or destroy token supply.	✓	

SH-HL-05 <code>hlShDeployAndLinkLeavesTheNewVaultLinked</code> A deploy and link message ends with the new vault linked.	✓	
SH-HL-06 <code>hlShDeployAndLinkRegistersTheVaultRowTheMessageNames</code> A deploy and link message registers the vault against the pool, class and asset it names.	✓	
SH-HL-07 <code>hlShLinkThenUnlinkRestoresTheRoutingFlag</code> A link and unlink cycle hands the routing flag back where it started.	✓	
SH-HL-08 <code>hlShLinkThenUnlinkLeavesTheVaultIdentityUntouched</code> A link and unlink cycle leaves the vault's identity untouched.	✓	
SH-HL-09 <code>hlShManagerSeatMovesOnlyOnItsRoleMessage</code> A manager seat changes only on the manager role message.	✓	
SH-HL-10 <code>hlShBridgerSeatMovesOnlyOnItsRoleMessage</code> A bridger seat changes only on the bridger role message.	✓	
SH-HL-11 <code>hlShAddPoolAsksTheEscrowFactoryExactlyOnce</code> Registering a pool asks the escrow factory for an escrow exactly once, in that same message.	✓	
SH-HL-12 <code>hlShAddPoolAsksTheEscrowFactoryForThatPool</code> The escrow a pool registration asks for is that pool's own.	✓	
SH-HL-13 <code>hlShRequestCallbackIsDeliveredExactlyOnce</code> A request callback is delivered exactly once.	✓	
SH-HL-14 <code>hlShRequestCallbackReachesTheRegisteredManager</code> A request callback reaches the manager the registry holds for the pool.	✓	
SH-HL-15 <code>hlShGrantThenRevokeLeavesTheLedgerUntouched</code> A grant and its revocation routed through the handler cancel each other exactly.	✓	

Reverts

Properties	Status	Mutations
SH-RV-01 <code>rvShFileRefusesANonWardOrAnUnknownName</code> The handler's two dependencies are repointed only by a trusted caller, under a name it knows.	✓	
SH-RV-02 <code>rvShAddPoolRefusesAnUntrustedCallerOrAPoolAlreadyOpen</code> A pool is opened on the spoke once, by a caller both the handler and the registry trust.	✓	
SH-RV-03 <code>rvShAddShareClassRefusesAnUntrustedCallerANullRegistrarOrATakenSlot</code> A share class registration needs a trusted caller, an unclaimed slot and an unclaimed token.	✓	
SH-RV-04 <code>rvShUpdateManagerRefusesAnUntrustedCallerOrAPoolNeverOpened</code> A manager seat moves only by a caller both the handler and the registry trust.	✓	
SH-RV-05 <code>rvShUpdateBridgerRefusesAnUntrustedCallerOrAPoolNeverOpened</code> The right to burn shares for a transfer moves only by a caller both trust.	✓	
SH-RV-06 <code>rvShSetPolicyRefusesAnUntrustedCallerOrAPoolNeverOpened</code> A rule book is installed only by a trusted caller, on a pool the spoke has opened.	✓	
SH-RV-07 <code>rvShAuthorizeRefusesAnUntrustedCallerOrAPoolWithNoRuleBook</code> An authorization is recorded only by a trusted caller, and only for a pool with a rule book.	✓	
SH-RV-08 <code>rvShUnauthorizeRefusesAnUntrustedCallerOrAGrantThatDoesNotStand</code>		

A revocation is recorded only against a grant the pool's standing rule book actually holds.	✓	
SH-RV-09 <code>rvShUpdatePricePoolPerShareRefusesAnUntrustedCallerAnAbsentClassOrAnOlderStamp</code> A share price is taken only from a trusted caller, and never older than the price on file.	✓	
SH-RV-10 <code>rvShUpdatePricePoolPerAssetRefusesAnUntrustedCallerAnUnknownRowOrAnOlderStamp</code> An asset price is taken only from a trusted caller, and never older than the price on file.	✓	
SH-RV-11 <code>rvShSetRequestManagerRefusesAnUntrustedCallerOrAPoolNeverOpened</code> The contract answering a pool's requests is installed only by a caller both trust.	✓	
SH-RV-12 <code>rvShRequestCallbackRefusesAnUntrustedCallerOrAPoolWithNoRequestManager</code> A fulfilment from the hub is never dropped in silence: it needs a request manager on file.	✓	
SH-RV-13 <code>rvShUpdateRestrictionRefusesAnUntrustedCallerOrAClassTheSpokeDoesNotCarry</code> A transfer restriction is pushed only at a class the spoke carries.	✓	
SH-RV-14 <code>rvShExecuteTransferSharesRefusesAnAbsentClassOrAMalformedReceiver</code> The arrival leg mints only into a class the spoke carries, for a receiver that names an address.	✓	
SH-RV-15 <code>rvShUpdateVaultLinkRefusesAnUnknownOrMismatchedVault</code> Routing is switched on only for a registered vault serving exactly what the message names.	✓	
SH-RV-16 <code>rvShUpdateVaultUnlinkRefusesAnUnroutedOrMismatchedVault</code> Routing is switched off only for a vault serving exactly what the message names.	✓	
SH-RV-17 <code>rvShUpdateVaultDeployAndLinkRefusesAnUnknownAssetADeadClassOrATakenAddress</code> A deploy and link message needs a numbered asset, a class with a token, and a fresh address.	✓	
SH-RV-18 <code>rvShAddShareClassAcceptsACleanRegistration</code> A share class registration that clears every gate is carried through.	✓	
SH-RV-19 <code>rvShRelyRefusesANonWard</code> Authority over the handler is handed on only by an account that already holds it.	✓	
SH-RV-20 <code>rvShDenyRefusesANonWard</code> Authority over the handler is withdrawn only by an account that holds it.	✓	
SH-RV-21 <code>rvShUpdateManagerIsNeverRefusedOnAPoolTheHubOpened</code> The pool the hub opened is live to the handler, so a seat move on it is never turned away.	✓	
SH-RV-22 <code>rvShUnauthorizeIsNeverRefusedOnAGrantThatStands</code> The last grant a pool holds can still be revoked, so nothing stays authorised for good.	✓	
SH-RV-23 <code>rvShRequestCallbackIsNeverRefusedOnAPoolThatHasARequestManager</code> A fulfilment reaches the pool's registered request manager, and the handler itself never turns it away.	✓	

Reachability

Properties	Status	Mutations
SH-RC-01 <code>rcShAddPoolOpensThePoolIsReachable</code> Witnesses a pool opening on this spoke, its escrow raised in the same message.	✓	
SH-RC-02 <code>rcShAddShareClassBindsTheDeployedTokenIsReachable</code> Witnesses a share class bound to its token, its registrar and the lookup back to the pool.	✓	
SH-RC-03 <code>rcShPoolThenItsFirstShareClassIsReachable</code> Witnesses the two message onboarding of a pool and its first share class.	✓	
SH-RC-04 <code>rcShDeployAndLinkRegistersAndLinksIsReachable</code>		

Witnesses the one message vault onboarding, registered and already routed to.	✓	
SH-RC-05 <code>rcShLinkSwitchesOnARegisteredVaultIsReachable</code> Witnesses the routing switch being turned on for a vault left unrouted.	✓	
SH-RC-06 <code>rcShUnlinkSwitchesOffALinkedVaultIsReachable</code> Witnesses the routing switch being turned off, the vault keeping the asset it serves.	✓	
SH-RC-07 <code>rcShLinkThenUnlinkRoundTripIsReachable</code> Witnesses a whole routing cycle in one transaction, the vault genuinely in service between.	✓	
SH-RC-08 <code>rcShTransferSharesPaysTheNamedReceiverIsReachable</code> Witnesses the destination leg paying out, the handler keeping none of it.	✓	
SH-RC-09 <code>rcShTransferSharesMintsTheFirstSharesIsReachable</code> Witnesses the very first shares of a class being created on this spoke.	✓	
SH-RC-10 <code>rcShTransferSharesAtTheLargestLegIsReachable</code> Witnesses the largest single bridge leg the message format can carry.	✓	
SH-RC-11 <code>rcShTransferSharesForAnEmptyLegIsReachable</code> Witnesses an empty bridge leg being accepted and moving nothing.	✓	
SH-RC-12 <code>rcShTransferSharesAddressedToTheHandlerIsReachable</code> Witnesses a bridge leg addressed to the handler itself, the shares staying on it.	✓	
SH-RC-13 <code>rcShManagerSeatRoundTripIsReachable</code> Witnesses the balance sheet operator seat granted and withdrawn again.	✓	
SH-RC-14 <code>rcShBridgerSeatRoundTripIsReachable</code> Witnesses the bridge seat granted and withdrawn again.	✓	
SH-RC-15 <code>rcShPolicySwapAdvancesTheNamespaceIsReachable</code> Witnesses a pool moved onto a different rule book, its namespace advancing in the same message.	✓	
SH-RC-16 <code>rcShPolicyRemovalIsReachable</code> Witnesses a pool left with no rule book at all, the namespace still advancing.	✓	
SH-RC-17 <code>rcShAuthorizeFilesTheGrantInTheLiveTenureIsReachable</code> Witnesses a hub grant landing on the row the live rule book and namespace decide.	✓	
SH-RC-18 <code>rcShGrantsStackOnOneRowIsReachable</code> Witnesses two grants for one call standing outstanding at the same time.	✓	
SH-RC-19 <code>rcShGrantThenRevokeRoundTripIsReachable</code> Witnesses a grant and its withdrawal inside one transaction.	✓	
SH-RC-20 <code>rcShSharePriceMovingToALaterStampIsReachable</code> Witnesses a fresh share valuation arriving, both the mark and its moment landing.	✓	
SH-RC-21 <code>rcShSharePriceCanBeMarkedToZeroIsReachable</code> Witnesses a live share valuation marked down to zero and still read as valid.	✓	
SH-RC-22 <code>rcShSharePriceAheadOfTheLocalClockIsReachable</code> Witnesses a valuation stamped ahead of the spoke's own block still being recorded.	✓	
SH-RC-23 <code>rcShAssetPriceMovingToALaterStampIsReachable</code>	✓	

Witnesses a fresh asset valuation landing on the row of the asset it names.		
SH-RC-24 <code>rcShRequestManagerAppointmentIsReachable</code> Witnesses a pool being given the contract that answers its investment requests, where it had none.	✓	
SH-RC-25 <code>rcShRequestManagerWithdrawalIsReachable</code> Witnesses the appointment being withdrawn by naming the zero address.	✓	
SH-RC-26 <code>rcShAppointedManagerReceivesTheCallbackIsReachable</code> Witnesses an inbound request reaching the contract the pool was just given.	✓	
SH-RC-27 <code>rcShFileRepointsTheEscrowFactoryIsReachable</code> Witnesses the handler being repointed at another live escrow factory.	✓	
SH-RC-28 <code>rcShFileRepointsTheSpokeRegistryIsReachable</code> Witnesses the handler being repointed at another live registry.	✓	
SH-RC-29 <code>rcShWardRoundTripIsReachable</code> Witnesses authority over the handler extended to another party and taken back.	✓	

Access Control

Properties	Status	Mutations
SH-AC-01 <code>acShWardRosterMovesOnlyForAWard</code> Only an address already holding the ward bit extends or trims the handler's ward roster.	✓	
SH-AC-02 <code>acShRegistryPointerMovesOnlyForAWard</code> Only a ward repoints the handler at another registry.	✓	
SH-AC-03 <code>acShEscrowFactoryPointerMovesOnlyForAWard</code> Only a ward repoints the handler at another escrow factory.	✓	
SH-AC-04 <code>acShPoolRowMovesOnlyForAWard</code> Through the handler, a pool is put on this spoke's books only when a ward asks for it.	✓	
SH-AC-05 <code>acShShareClassBindingMovesOnlyForAWard</code> Through the handler, only a ward binds the token a share class runs on and its registrar.	✓	
SH-AC-06 <code>acShTokenLookupMovesOnlyForAWard</code> Through the handler, only a ward moves the lookup naming the pool and class a share token belongs to.	✓	
SH-AC-07 <code>acShManagerSeatMovesOnlyForAWard</code> Through the handler, only a ward hands out or withdraws a manager seat on the spoke side.	✓	
SH-AC-08 <code>acShBridgerSeatMovesOnlyForAWard</code> Through the handler, only a ward hands out or withdraws a bridger seat on the spoke side.	✓	
SH-AC-09 <code>acShPolicyRowMovesOnlyForAWard</code> Through the handler, only a ward moves the rule book a pool runs under and its namespace.	✓	
SH-AC-10 <code>acShAuthorizationLedgerMovesOnlyForAWard</code> No inbound path grants or revokes an authorization entry for a caller without the ward bit.	✓	

SH-AC-11 <code>acShSharePriceMovesOnlyForAWard</code> Through the handler, only a ward moves a share class price and the moment it was computed.	✓	
SH-AC-12 <code>acShAssetPriceMovesOnlyForAWard</code> Through the handler, only a ward moves an asset price and the moment it was computed.	✓	
SH-AC-13 <code>acShVaultRowMovesOnlyForAWard</code> Through the handler, only a ward moves a vault's registry row, identity and routing flag alike.	✓	
SH-AC-14 <code>acShRequestManagerPointerMovesOnlyForAWard</code> Through the handler, only a ward repoints the request manager a pool routes its requests to.	✓	
SH-AC-15 <code>acShShareSupplyMovesOnlyForAWard</code> Share token supply moves through the handler only for a ward.	✓	
SH-AC-16 <code>acShHolderBalanceMovesOnlyForAWard</code> No holder's share balance moves through the handler for an unauthorised caller.	✓	
SH-AC-17 <code>acShRequestCallbackGoesOutOnlyForAWard</code> The handler hands a request to a pool's request manager only for a ward.	✓	

SpokeHandler_LinkedRegistryFactory

- One symbolic pool
- One symbolic share class
- ERC20 tokens modeled in CVL (standard, up to 3 holders, uint128 balances, 6 to 18 decimals, no share token hooks)

State Transitions

Properties	Status	Mutations
SH-ST-01 <code>stShPoolRowNeedsTheEscrowDeployBehindIt</code> A pool row appears only where the handler held the factory seat that lets it deploy that pool's escrow.	✓	

MessageDispatcher

Access Control

Properties	Status	Mutations
MD-AC-01 <code>acMdWardSetMovesOnlyForAWard</code> Control of the dispatcher is handed out or taken away only by an address that already holds it.	✓	
MD-AC-02 <code>acMdWiringMovesOnlyForAWard</code> Only a ward can rewire the contracts the dispatcher routes through.	✓	
MD-AC-03 <code>acMdNoUnwardedCallerReachesANeighbour</code> A caller without the ward bit reaches no neighbour at all.	✓	

PoolEscrowFactory

High Level

Properties	Status	Mutations
PEF-HL-01 <code>hlPefEscrowAddressNeverMoves</code> A pool's escrow address does not move, so consumers resolving it at different times agree.	✓	
PEF-HL-02 <code>hlPefEscrowAddressIsCallerIndependent</code> A pool's escrow address does not depend on who asks.	✓	

Access Control

Properties	Status	Mutations
PEF-AC-01 <code>acPefFactoryStateMovesOnlyForAWard</code> The factory's wiring and ward roll move only for a caller that already holds the bit.	✓	
PEF-AC-02 <code>acPefMintingAnEscrowRefusesANonWard</code> A pool's escrow does not deploy for a stranger to the factory.	✓	

Mutation Testing

A mutant is a deliberate fault planted in the source under [mutations/](#); a rule catches it by flipping to Violated on the mutated source, and one mutant may be caught by several rules. `./certora/mutations/run_mutation.sh <target> <id>` plants it, runs the rule and restores the source. Each entry names its fault and shows the diff; the Mutations column of the property tables links to it.

Band	Mutants	Rules that catch them	Entries
Accounting	56	73	mutations/Accounting/README.md
Holdings	84	87	mutations/Holdings/README.md
Hub	71	98	mutations/Hub/README.md
HubHandler	72	101	mutations/HubHandler/README.md
HubRegistry	80	104	mutations/HubRegistry/README.md
MessageDispatcher	4	3	mutations/MessageDispatcher/README.md
MultiAdapter	69	96	mutations/MultiAdapter/README.md
PoolEscrow	46	52	mutations/PoolEscrow/README.md
PoolEscrowFactory	4	4	mutations/PoolEscrowFactory/README.md
ShareClassManager	64	85	mutations/ShareClassManager/README.md
SnapshotQueue	61	75	mutations/SnapshotQueue/README.md
Spoke	73	107	mutations/Spoke/README.md
SpokeHandler	78	92	mutations/SpokeHandler/README.md

SpokeLinked	23	26	mutations/SpokeLinked/README.md
SpokeRegistry	106	141	mutations/SpokeRegistry/README.md

Reproducing the Results

The Certora Prover runs either remotely on Certora's cloud or locally from a build. Both modes share the setup below.

Prerequisites

For Ubuntu 24.04; a step-by-step walkthrough is in this [setup tutorial](#).

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install [pipx](#), which keeps Python CLI tools in isolated environments

```
sudo apt install pipx
pipx ensurepath
```

3. Install the Certora CLI, pinned to the prover version of this report

```
pipx install certora-cli==8.16.2
```

4. Install solc-select and the compiler this project builds with

```
pipx install solc-select
solc-select install 0.8.28
solc-select use 0.8.28
```

5. Create the versioned symlink the confs name. They reference the compiler as `solc0.8.28` without the dash, while solc-select installs a generic `solc`:

```
mkdir -p ~/.local/bin
ln -sf ~/.solc-select/artifacts/solc-0.8.28/solc-0.8.28 ~/.local/bin/solc0.8.28
```

Check that `~/.local/bin` is on your `PATH`, and add it if not:

```
echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

6. From the repository root, fetch the library sources the contracts import. They are git submodules, and `--recursive` reaches the nested ones the confs name by path:

```
git submodule update --init --recursive
```

Remote Execution

Set up a Certora key, available free through Certora's [Discord](#) or their website:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

Note: a local prover, if installed, takes priority. To force the cloud, add `--server production`:

```
certoraRun certora/confs/Accounting/single_pool_valid_state.conf --server production
```

Local Execution

Follow the build instructions in the [CertoraProver repository \(v8.16.2\)](#). Once built, the local prover takes priority over the cloud by default.

Running the Suites

The flags below appear in the commands that follow, and each answers one question:

Flag	What it does	When to reach for it
<code>--rule <name></code>	runs one property instead of the conf	a property that did not decide, so it gets the budget rather than sharing it
<code>--exclude_rule <names></code>	runs the conf without those properties	the rest of a conf whose one heavy property is being run separately

Accounting

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Accounting/multi_pool_state_transitions.conf
certoraRun certora/confs/Accounting/single_pool_access_control.conf
certoraRun certora/confs/Accounting/single_pool_high_level.conf
certoraRun certora/confs/Accounting/single_pool_reachability.conf
certoraRun certora/confs/Accounting/single_pool_reverts.conf
certoraRun certora/confs/Accounting/single_pool_state_transitions.conf
certoraRun certora/confs/Accounting/single_pool_valid_state.conf
certoraRun certora/confs/Accounting/single_pool_variable_transitions.conf
```

Holdings

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Holdings/multi_pool_state_transitions.conf
certoraRun certora/confs/Holdings/multi_share_class_valid_state.conf
certoraRun certora/confs/Holdings/single_pool_access_control.conf
certoraRun certora/confs/Holdings/single_pool_high_level.conf
certoraRun certora/confs/Holdings/single_pool_reachability.conf
certoraRun certora/confs/Holdings/single_pool_reverts.conf
certoraRun certora/confs/Holdings/single_pool_state_transitions.conf
certoraRun certora/confs/Holdings/single_pool_valid_state.conf
certoraRun certora/confs/Holdings/single_pool_variable_transitions.conf
```

[ShareClassManager](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/ShareClassManager/multi_pool_state_transitions.conf
certoraRun certora/confs/ShareClassManager/multi_pool_valid_state.conf
certoraRun certora/confs/ShareClassManager/multi_share_class_state_transitions.conf
certoraRun certora/confs/ShareClassManager/multi_share_class_valid_state.conf
certoraRun certora/confs/ShareClassManager/single_pool_access_control.conf
certoraRun certora/confs/ShareClassManager/single_pool_high_level.conf
certoraRun certora/confs/ShareClassManager/single_pool_reachability.conf
certoraRun certora/confs/ShareClassManager/single_pool_reverts.conf
certoraRun certora/confs/ShareClassManager/single_pool_state_transitions.conf
certoraRun certora/confs/ShareClassManager/single_pool_valid_state.conf
certoraRun certora/confs/ShareClassManager/single_pool_variable_transitions.conf
```

[HubRegistry](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/HubRegistry/multi_pool_state_transitions.conf
certoraRun certora/confs/HubRegistry/single_pool_access_control.conf
certoraRun certora/confs/HubRegistry/single_pool_high_level.conf
certoraRun certora/confs/HubRegistry/single_pool_reachability.conf
certoraRun certora/confs/HubRegistry/single_pool_reverts.conf
certoraRun certora/confs/HubRegistry/single_pool_state_transitions.conf
certoraRun certora/confs/HubRegistry/single_pool_valid_state.conf
certoraRun certora/confs/HubRegistry/single_pool_variable_transitions.conf
```

[Spoke](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Spoke/access_control.conf
certoraRun certora/confs/Spoke/high_level.conf
certoraRun certora/confs/Spoke/reachability.conf
certoraRun certora/confs/Spoke/reverts.conf
certoraRun certora/confs/Spoke/state_transitions.conf
certoraRun certora/confs/Spoke/variable_transitions.conf
```

[SpokeRegistry](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/SpokeRegistry/access_control.conf
certoraRun certora/confs/SpokeRegistry/high_level.conf
certoraRun certora/confs/SpokeRegistry/reachability.conf
certoraRun certora/confs/SpokeRegistry/reverts.conf
certoraRun certora/confs/SpokeRegistry/state_transitions.conf
certoraRun certora/confs/SpokeRegistry/valid_state.conf
certoraRun certora/confs/SpokeRegistry/variable_transitions.conf
```

[SnapshotQueue](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/SnapshotQueue/multi_pool_state_transitions.conf
certoraRun certora/confs/SnapshotQueue/single_pool_access_control.conf
certoraRun certora/confs/SnapshotQueue/single_pool_high_level.conf
certoraRun certora/confs/SnapshotQueue/single_pool_reachability.conf
certoraRun certora/confs/SnapshotQueue/single_pool_reverts.conf
certoraRun certora/confs/SnapshotQueue/single_pool_state_transitions.conf
certoraRun certora/confs/SnapshotQueue/single_pool_valid_state.conf
certoraRun certora/confs/SnapshotQueue/single_pool_variable_transitions.conf
```

[PoolEscrow](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/PoolEscrow/access_control.conf
certoraRun certora/confs/PoolEscrow/high_level.conf
certoraRun certora/confs/PoolEscrow/multi_row_valid_state.conf
certoraRun certora/confs/PoolEscrow/reachability.conf
certoraRun certora/confs/PoolEscrow/reverts.conf
certoraRun certora/confs/PoolEscrow/state_transitions.conf
certoraRun certora/confs/PoolEscrow/valid_state.conf
```

[MultiAdapter](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/MultiAdapter/access_control.conf
certoraRun certora/confs/MultiAdapter/high_level.conf
certoraRun certora/confs/MultiAdapter/reverts.conf
certoraRun certora/confs/MultiAdapter/state_transitions.conf
certoraRun certora/confs/MultiAdapter/valid_state.conf
certoraRun certora/confs/MultiAdapter/variable_transitions.conf
```

`reachability.conf` holds 1 rules that did not decide, so the conf runs without them and they run one at a time:

```
certoraRun certora/confs/MultiAdapter/reachability.conf \
  --exclude_rule rcMaExecuteReleasesThePayloadAfterTwoProofsIsReachable

certoraRun certora/confs/MultiAdapter/reachability.conf --rule
rcMaExecuteReleasesThePayloadAfterTwoProofsIsReachable
```

[Hub_LinkedHubCore](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Hub_LinkedHubCore/access_control.conf
certoraRun certora/confs/Hub_LinkedHubCore/high_level.conf
certoraRun certora/confs/Hub_LinkedHubCore/reachability.conf
certoraRun certora/confs/Hub_LinkedHubCore/reverts.conf
certoraRun certora/confs/Hub_LinkedHubCore/state_transitions.conf
certoraRun certora/confs/Hub_LinkedHubCore/valid_state.conf
certoraRun certora/confs/Hub_LinkedHubCore/variable_transitions.conf
```

[HubHandler_LinkedHubCore](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/HubHandler_LinkedHubCore/access_control.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/high_level.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/reachability.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/reverts.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/state_transitions.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/valid_state.conf
certoraRun certora/confs/HubHandler_LinkedHubCore/variable_transitions.conf
```

[Spoke_LinkedSpokeCore](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/Spoke_LinkedSpokeCore/high_level.conf
certoraRun certora/confs/Spoke_LinkedSpokeCore/state_transitions.conf
certoraRun certora/confs/Spoke_LinkedSpokeCore/valid_state.conf
```

[SpokeHandler_LinkedSpokeRegistry](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/access_control.conf
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/high_level.conf
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/reachability.conf
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/reverts.conf
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/state_transitions.conf
certoraRun certora/confs/SpokeHandler_LinkedSpokeRegistry/variable_transitions.conf
```

[SpokeHandler_LinkedRegistryFactory](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/SpokeHandler_LinkedRegistryFactory/state_transitions.conf
```

[MessageDispatcher](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/MessageDispatcher/access_control.conf
```

[PoolEscrowFactory](#)

Every rule in these decided, so each runs whole:

```
certoraRun certora/confs/PoolEscrowFactory/access_control.conf
certoraRun certora/confs/PoolEscrowFactory/high_level.conf
```

Resources

- [Certora Tutorials](#): official Certora documentation and guided tutorials
- AlexZoid [personal site](#) and [FV resources](#): write-ups on verifying real protocols, alongside a curated collection of specs, examples and references
- [Updraft Assembly & Formal Verification Course](#): comprehensive video course covering assembly and formal verification from the ground up
- [Find Highs Using Certora Formal Verification](#): practical guide with a companion [repo](#) of simplified examples based on real code and bugs from private audits
- [RareSkills Certora Book](#): structured tutorial covering CVL syntax, patterns, and common pitfalls